THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms**. S.HARITHA (20524UCSC073)**, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

EX. NO: 01 Creation of a DDL operations using Patients Table

DATE:6/7/2026

Aim

To create a Patients table, insert patient records, display all records, and retrieve patients admitted from the year 2015 onwards.

Software Used

Oracle / MySQL / sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS:

- Create
- Alter ● Drop
- Truncate

CREATE:

Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE patient_table
(     column1 datatype constraints,     column2
datatype constraints,      column3 datatype
constraints
);
```

Table Structure

```
SQL> CREATE TABLE
patients (          id INTEGER
PRIMARY
KEY,          patient_name TEXT,
disease                    TEXT,
admission_year INTEGER
        );
```

Table Created **ALTER:**

Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP.

ADD:

Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE patient_table ADD column_name datatype; **Query:**

SQL> ALTER TABLE patients ADD genre VARCHAR(50); Table
Altered

MODIFY:

Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE patient_table MODIFY column_name datatype; **Query:**

SQL> ALTER TABLE patients MODIFY name VARCHAR(200); ALTER
TABLE patients ALTER COLUMN name VARCHAR(200);

DROP:

Drop command is used to delete the attribute/column in a table. **Syntax:**

ALTER TABLE patient_table DROP COLUMN column_name; **Query:**

SQL> ALTER TABLE patients DROP COLUMN genre;

RENAME:

Rename command is used to re-change the table name.

SQL> RENAME TABLE patients TO novels; Table
renamed.

(OR)

ALTER TABLE patients RENAME TO novels;

Column Rename

ALTER TABLE patients RENAME COLUMN patient_name TO book_patient_name;

TRUNCATE:

Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE tablename; **Query:**

TRUNCATE TABLE patients;

DROP:

Deletes the table and all its data permanently.

Syntax: DROP TABLE tablename; **Query:**

DROP TABLE patients;

Explanation

- Create a table named patients.
- Insert patient details into the table.
- Display all patient records.
- Retrieve patients admitted in or after the year 2015 and sort them by admission year.

Create Table

```
CREATE TABLE patients
(           id INTEGER PRIMARY
KEY,            patient_name TEXT,
disease                     TEXT,
admission_year INTEGER
        );
```

Sample Data

```
INSERT INTO patients VALUES (1, 'The Alchemist', 'Paulo Coelho', 1988);
INSERT INTO patients VALUES (2, 'The Da Vinci Code', 'Dan Brown', 2003);
INSERT INTO patients VALUES (3, 'The Kite Runner', 'Khaled Hosseini', 2003);
INSERT INTO patients VALUES (4, 'The Hunger Games', 'Suzanne Collins', 2008);
INSERT INTO patients VALUES (5, 'The Fault in Our Stars', 'John Green', 2012);
INSERT INTO patients VALUES (6, 'Where the Crawdads Sing', 'Delia Owens', 2018);
```

SQL Query

Query 1: Display all patients records

SQL> SELECT * FROM patients;

Resultant Query

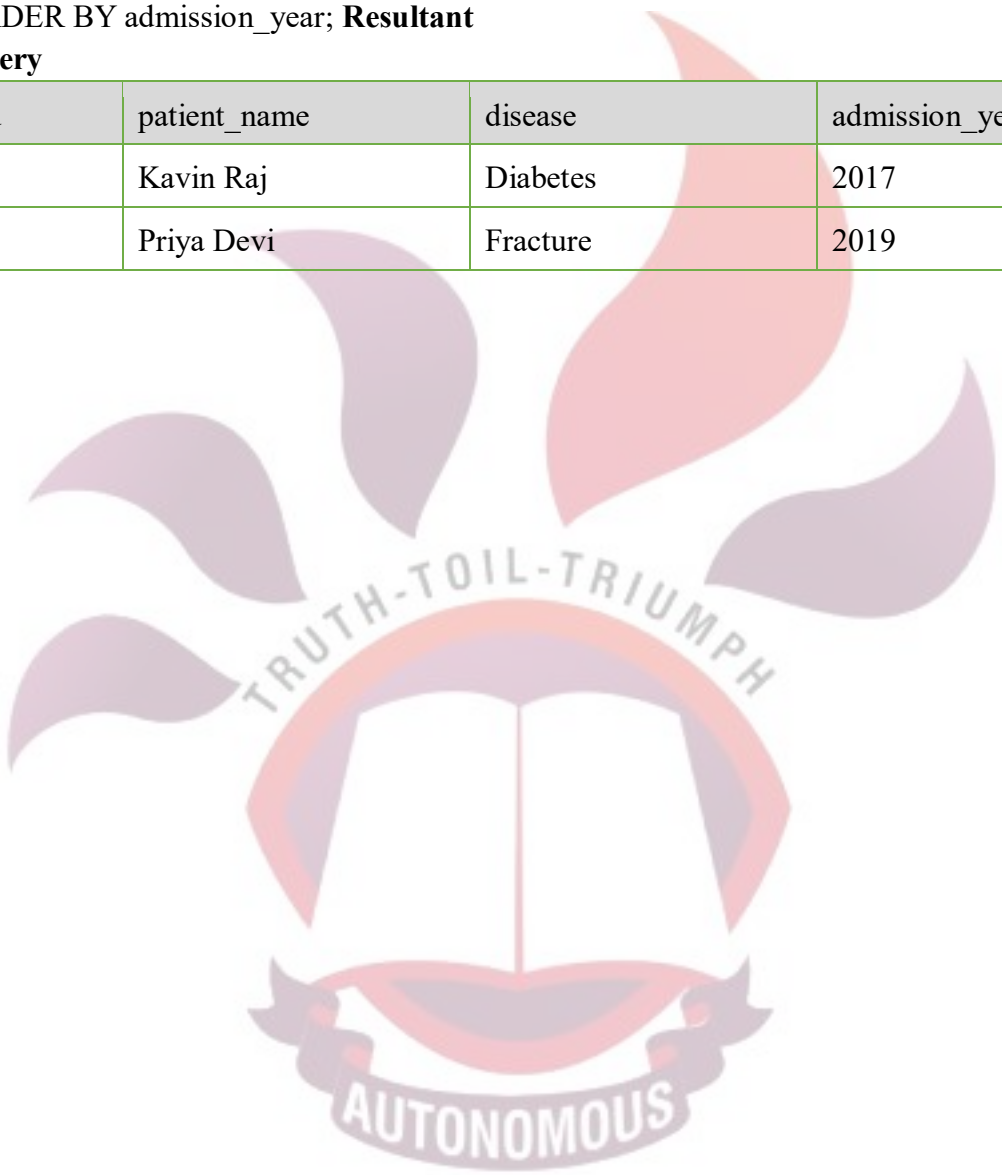
id	patient_name	disease	admission_year
1	Arun Kumar	Fever	2012
2	Meena Ravi	Asthma	2015
3	Kavin Raj	Diabetes	2017
4	Priya Devi	Fracture	2019
5	Nithya S	Migraine	2021

6	Vijay Anand	Infection	2023

Query 2: Display patients admitted from the year 2015 onwards

SQL> SELECT *
FROM patients
WHERE admission_year >= 2010
ORDER BY admission_year; **Resultant Query**

id	patient_name	disease	admission_year
3	Kavin Raj	Diabetes	2017
4	Priya Devi	Fracture	2019



RESULT:

Thus, the Patients table was created successfully, records were inserted, all patient details were displayed, and the patients admitted from the year 2015 onwards were retrieved successfully.

EX. NO: 02 Creation of a DML operations using Patients Table
DATE: 9/07/2026

Aim

To insert a new patient record, update an existing patient's ward, and delete an patient record from the Patients table.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, update, and delete data within database objects (tables). Unlike DDL, DML changes only affect the data, not the table structure.

DML COMMANDS:

- Insert
- Update
- Delete

INSERT:

Insert command is used to add new records (rows) into a table.

Syntax:

INSERT INTO table_name (column1, column2, column3, ...) VALUES
(value1, value2, value3, ...); **Query:**

SQL> INSERT INTO patients VALUES (7, 'Meena Iyer', 'Emergency', 2022);

1 row created.

UPDATE:

Update command is used to modify existing records in a table.

Syntax:

UPDATE table_name
SET column1 = value1, column2 = value2 WHERE
condition; **Query:**

SQL> UPDATE patients SET

ward = 'Operations' WHERE

id = 1; 1 row updated.

DELETE:

Delete command is used to remove existing records from a table.

Syntax:

DELETE FROM table_name
WHERE condition; **Query:**
SQL> DELETE FROM

patients WHERE id = 7; 1 row

deleted.

Explanation

- Insert a new patient record into the patients table.
- Update the ward of an existing patient.
- Delete an patient record from the table.

Create Table

CREATE TABLE patients
(id INTEGER PRIMARY KEY,
name TEXT,
ward TEXT,
admission_year INTEGER
);

Sample Data

INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010);
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013);
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016);
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018);
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019); INSERT
INTO patients VALUES (6, 'Divya Nair', 'General', 2021);

SQL Query

Query 1: Insert a new patient record

SQL> INSERT INTO patients VALUES (7, 'Meena Iyer', 'Emergency', 2022);

1 row created.

Query 2: Display all patient records after insertion

SQL> SELECT * FROM patients;

Resultant Query

id	patient_name	ward	admission_year
1	Arun Kumar	Emergency	2012
2	Meena Ravi	General	2015
3	Kavin Raj	Cardiology	2017
4	Priya Devi	Orthopedic	2019
5	Nithya S	Neurology	2021
6	Vijay Anand	General	2023
7	Divya Rao	Emergency	2024

Query 3: Update the ward of patient id 1

SQL> UPDATE patients
SET ward = 'Operations' WHERE
id = 1;

1 row updated.

Resultant Query

id	patient_name	ward	admission_year
1	Arun Kumar	Support	2012
2	Meena Ravi	General	2015
3	Kavin Raj	Cardiology	2017
4	Priya Devi	Orthopedic	2019
5	Nithya S	Neurology	2021
6	Vijay Anand	General	2023
7	Divya Rao	Emergency	2024

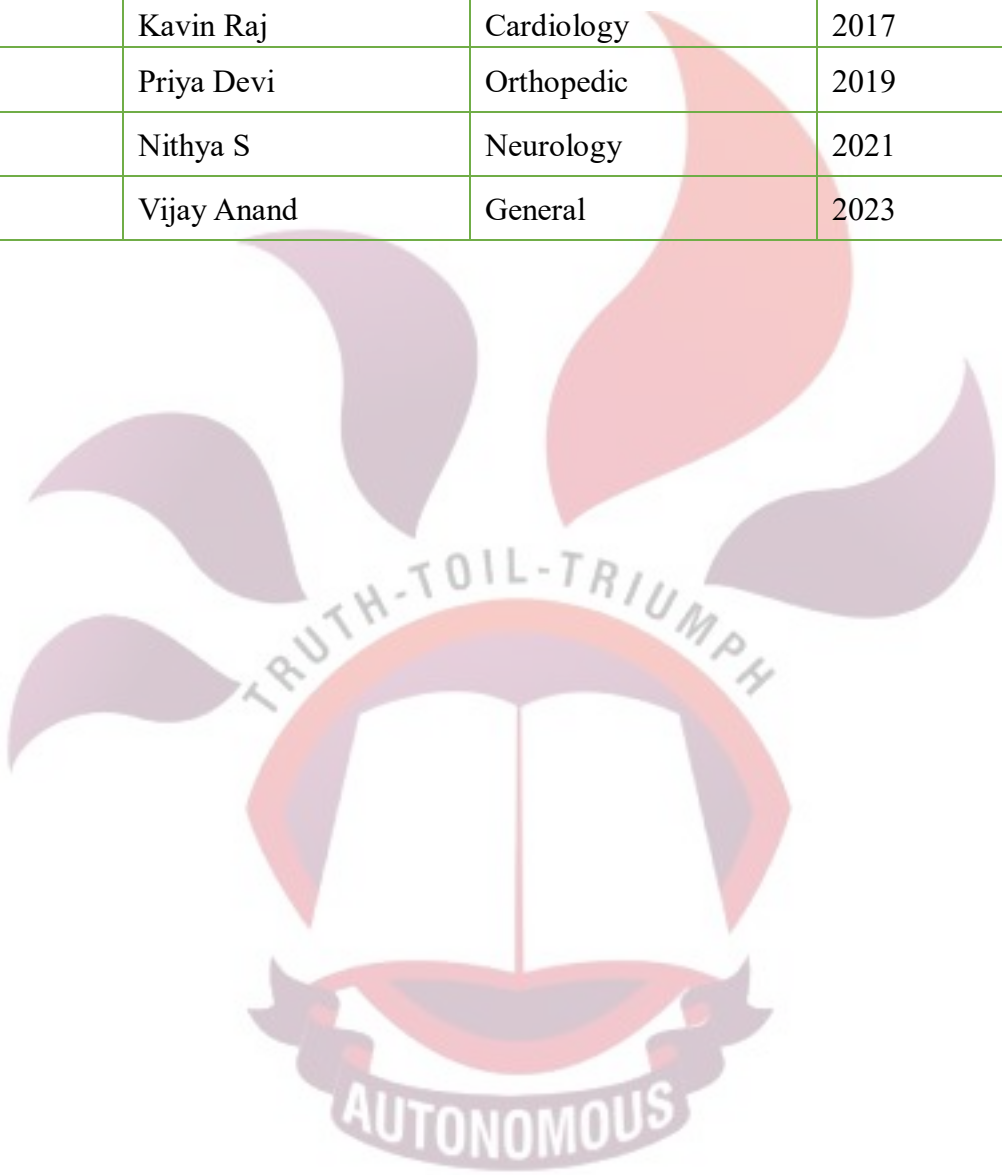
Query 4: Delete the record of patient id 7

SQL> DELETE FROM patients WHERE
id = 7;

1 row deleted.

Resultant Query

id	patient_name	ward	admission_year
1	Arun Kumar	Support	2012
2	Meena Ravi	General	2015
3	Kavin Raj	Cardiology	2017
4	Priya Devi	Orthopedic	2019
5	Nithya S	Neurology	2021
6	Vijay Anand	General	2023



RESULT:

Thus, a new patient record was inserted, an existing patient record was updated, and an patient record was deleted successfully from the Patients table.

EX. NO: 03 Creation of a DQL operations using Patients Table

DATE: **13/07/2026**

Aim

To retrieve and filter records from the Patients table using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING clauses.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to fetch and view data stored in database tables. SELECT is the main command used for querying data, and it can be combined with clauses such as WHERE, ORDER BY, GROUP BY, and HAVING to filter, sort, and summarise results.

DQL COMMANDS:

- Select
- Where
- Order By
- Group By
- Having

SELECT:

Select command is used to retrieve data from one or more columns of a table.

Syntax:

SELECT column1, column2 FROM table_name;

WHERE:

Where clause is used to filter records that satisfy a given condition.

Syntax:

SELECT * FROM table_name WHERE condition;

ORDER BY:

Order By clause is used to sort the result set in ascending or descending order.

Syntax:

SELECT * FROM table_name ORDER BY column_name [ASC|DESC]; **GROUP BY:**

Group By clause is used to group rows that have the same values in specified columns, often used with aggregate functions.

Syntax:

SELECT column, aggregate_function(column) FROM table_name GROUP BY column;

HAVING:

Having clause is used to filter groups created by GROUP BY, based on a condition.

Syntax:

SELECT column, aggregate_function(column) FROM table_name GROUP BY column HAVING condition;

Explanation

- Retrieve selected columns from the patients table.
- Filter patients belonging to the Cardiology ward.
- Sort patients by admission year in descending order.
- Group patients by ward and count them.
- Filter wards having more than one patient.

Create Table

```
CREATE TABLE patients
(           id INTEGER PRIMARY KEY,
        name TEXT,
ward TEXT,
admission_year INTEGER
        );
```

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010);
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013);
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016);
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018);
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019);  INSERT
INTO patients VALUES (6, 'Divya Nair', 'General', 2021);
```

SQL Query

Query 1: Display patient name and ward

```
SQL> SELECT name, ward FROM patients;
```

Resultant Query

patient_name	ward
Arun Kumar	Emergency
Meena Ravi	General
Kavin Raj	Cardiology
Priya Devi	Orthopedic
Nithya S	Neurology
Vijay Anand	General

Query 2: Display patients belonging to the Cardiology ward

SQL> SELECT * FROM patients WHERE ward = 'Cardiology';

Resultant Query

id	patient_name	ward	admission_year
3	Kavin Raj	Cardiology	2017
4	Priya Devi	Orthopedic	2019

Query 3: Display patients sorted by admission year in descending order

SQL> SELECT * FROM patients ORDER BY admission_year DESC;

Resultant Query

id	patient_name	ward	admission_year
6	Vijay Anand	General	2023
5	Nithya S	Neurology	2021
4	Priya Devi	Orthopedic	2019
3	Kavin Raj	Cardiology	2017
2	Meena Ravi	General	2015
1	Arun Kumar	Emergency	2012

Query 4: Display the number of patients in each ward

SQL> SELECT ward, COUNT(*) AS total_patients
FROM patients
GROUP BY ward;

Resultant Query

ward	total_patients

Emergency	1
General	2
Cardiology	1
Orthopedic	1
Neurology	1

Query 5: Display wards having more than one patient

```
SQL> SELECT ward, COUNT(*) AS total_patients
FROM patients
GROUP BY ward
HAVING COUNT(*) >
1;
```

Resultant Query

ward	total_patients
General	2

RESULT:

Thus, the patient records were successfully retrieved, filtered, sorted, grouped, and summarised using the DQL commands SELECT, WHERE, ORDER BY, GROUP BY, and HAVING.

EX. NO: 04 Transaction Control operations using PatientsTable
DATE: 16/07/2026

Aim

To perform transaction control operations such as COMMIT, ROLLBACK, and SAVEPOINT on the Patients table.

Software Used

Oracle / MySQL / sqlite3

Description

TCL (Transaction Control Language) commands are used to manage the changes made by DML statements. A transaction groups one or more SQL statements so that they either complete fully or have no effect at all, ensuring data consistency.

TRANSACTIONS:

- Commit
- Rollback
- Savepoint

COMMIT:

Commit command is used to permanently save all changes made during the current transaction.

Syntax:

COMMIT;

ROLLBACK:

Rollback command is used to undo changes made in the current transaction, either completely or up to a savepoint.

Syntax:

ROLLBACK [TO savepoint_name];

SAVEPOINT:

Savepoint command is used to set a point within a transaction to which a later rollback can be performed, without undoing the entire transaction.

Syntax:

SAVEPOINT savepoint_name; **Explanation**

- Insert a new patient record.

- Create a savepoint after the insertion.
- Update the ward of the new patient.
- Rollback to the savepoint to undo the update.
- Commit the transaction to save the insertion permanently.

Create Table

```
CREATE TABLE patients
(           id INTEGER PRIMARY KEY,
        name TEXT,
ward TEXT,
admission_year INTEGER
        );
```

SQL Query

Query 1: Insert a new patient record

SQL> INSERT INTO patients VALUES (8, 'Arun Das', 'Emergency',

2023); 1 row created.

Query 2: Create a savepoint SQL>

SAVEPOINT sp1;

Savepoint created.

Query 3: Update the ward of the new patient

```
SQL> UPDATE patients
SET ward = 'Pediatrics'  WHERE
id = 8;
```

1 row updated.

Resultant Query

id	patient_name	ward	admission_year
8	Sanjay Kumar	Emergency	2024

Query 4: Rollback to the savepoint

SQL> ROLLBACK TO sp1;

Rollback complete.

Resultant Query

id	patient_name	ward	admission_year
8	Sanjay Kumar	General	2024

Query 5: Commit the transaction

SQL> COMMIT;

Commit complete.

RESULT:

Thus, the transaction control operations COMMIT, ROLLBACK, and SAVEPOINT were performed successfully on the Patients table.

EX. NO: 05 Creation of Join operations using Patients and Wards Table
DATE: 20/07/2026

Aim

To combine records from the Patients and Wards tables using Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / sqlite3

Description

A Join is used to combine rows from two or more tables based on a related column between them. Joins allow related data stored in separate tables to be queried together.

JOINS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN:

Returns only the rows that have matching values in both tables.

Syntax:

```
SELECT columns FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

LEFT JOIN:

Returns all rows from the left table, along with matching rows from the right table. Unmatched righttable columns appear as NULL.

Syntax:

```
SELECT columns FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

RIGHT JOIN:

Returns all rows from the right table, along with matching rows from the left table. Unmatched left-table columns appear as NULL.

Syntax:

SELECT columns FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;

FULL JOIN:

Returns all rows from both tables, with NULLs in place where no match exists on either side.

Syntax:

SELECT columns FROM table1
FULL JOIN table2 ON table1.column = table2.column;

SELF JOIN:

A table is joined with itself, typically to compare rows within the same table.

Syntax:

SELECT a.columns, b.columns FROM table_name a, table_name b WHERE condition;

Explanation

- Create an patients table and a wards table.
- Join the two tables using Inner, Left, Right, and Full Join.
- Join the patients table with itself to find patients in the same ward.

Create Table

```
CREATE TABLE patients
(           id INTEGER PRIMARY KEY,
        name TEXT,
ward TEXT,
admission_year INTEGER
        );

CREATE TABLE wards (        ward_id INTEGER
PRIMARY KEY,
        ward_name TEXT,
room_no TEXT
        );
```

Sample Data

INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010);
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013);

INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016);
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018);
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019);
INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021);

INSERT INTO wards VALUES (1, 'Emergency', 'Chennai');
INSERT INTO wards VALUES (2, 'General', 'Bangalore');
INSERT INTO wards VALUES (3, 'Cardiology', 'Hyderabad');
INSERT INTO wards VALUES (4, 'Orthopedic', 'Mumbai'); INSERT INTO wards VALUES (5, 'Pediatrics', 'Delhi');

SQL Query

Query 1: Inner Join - patients with a matching ward

SQL> SELECT e.name, e.ward, d.room_no
FROM patients e
INNER JOIN wards d ON e.ward = d.ward_name;

Resultant Query

patient_name	ward	room_no
Arun Kumar	Emergency	101
Meena Ravi	General	102
Kavin Raj	Cardiology	201
Priya Devi	Orthopedic	202
Nithya S	Neurology	203

Query 2: Left Join - all patients, with ward room_no if available

SQL> SELECT e.name, e.ward, d.room_no
FROM patients e
LEFT JOIN wards d ON e.ward = d.ward_name;

Resultant Query

patient_name	ward	room_no
Arun Kumar	Emergency	101
Meena Ravi	General	102
Kavin Raj	Cardiology	201
Priya Devi	Orthopedic	202
Nithya S	Neurology	203

Vijay Anand	General	102

Query 3: Right Join - all wards, with patient name if available

```
SQL> SELECT e.name, d.ward_name, d.room_no
FROM patients e
RIGHT JOIN wards d ON e.ward = d.ward_name;
```

Resultant Query

patient_name	ward_name	room_no
Arun Kumar	Emergency	101
Meena Ravi	General	102
Kavin Raj	Cardiology	201
Priya Devi	Orthopedic	202
Nithya S	Neurology	203
NULL	Pediatrics	301

Query 4: Full Join - all patients and all wards

```
SQL> SELECT e.name, e.ward, d.ward_name, d.room_no
FROM patients e
FULL JOIN wards d ON e.ward = d.ward_name;
```

Resultant Query

patient_name	ward	ward_name	room_no
Arun Kumar	Emergency	Emergency	101
Meena Ravi	General	General	102
Kavin Raj	Cardiology	Cardiology	201
Priya Devi	Orthopedic	Orthopedic	202
Nithya S	Neurology	Neurology	203
Vijay Anand	General	General	102
NULL	NULL	Pediatrics	301

Query 5: Self Join - patients working in the same ward

```
SQL> SELECT a.name AS patient1, b.name AS patient2, a.ward
FROM patients a, patients b
WHERE a.ward = b.ward AND a.id < b.id;
```

Resultant Query

patient1	patient2	ward
Meena Ravi	Vijay Anand	General



RESULT:

Thus, the Patients and Wards tables were successfully combined using Inner Join, Left Join, Right Join, Full Join, and Self Join.

EX. NO: 06 Creation of SQL Functions using Patients Table

DATE: 23/07/2026

Aim

To apply Aggregate Functions, String Functions, and Date Functions on the Patients table.

Software Used

Oracle / MySQL / sqlite3

Description

SQL functions are built-in operations that perform calculations on data. Aggregate functions summarise data across multiple rows, string functions manipulate text values, and date functions operate on date and time values.

SQL FUNCTIONS:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS:

Aggregate functions perform a calculation on a set of values and return a single value. Common functions are COUNT, SUM, AVG, MIN, and MAX.

Syntax:

SELECT aggregate_function(column) FROM table_name;

STRING FUNCTIONS:

String functions operate on text values. Common functions are UPPER, LOWER, LENGTH, CONCAT, and SUBSTR.

Syntax:

SELECT string_function(column) FROM table_name;

DATE FUNCTIONS:

Date functions operate on date and time values. Common functions are CURRENT_DATE, EXTRACT, and DATEDIFF.

Syntax:

SELECT date_function(column) FROM table_name;

Explanation

- Add a salary and admission_date column to the patients table.
- Apply aggregate functions to summarise salary data.
- Apply string functions on the patient name.
- Apply date functions on the admission_date column.

Create Table

```
CREATE TABLE patients
(           id INTEGER PRIMARY KEY,
        name TEXT,          ward
TEXT,          admission_year
INTEGER,                salary
DECIMAL(10,2),
admission_date DATE          );
```

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');
INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');
```

SQL Query

Query 1: Aggregate Functions - count, total, and average salary

```
SQL> SELECT COUNT(*) AS total_patients, SUM(salary) AS total_salary,
AVG(salary) AS avg_salary, MIN(salary) AS min_salary, MAX(salary) AS max_salary
FROM patients;
```

Resultant Query

total_patients	total_salary	avg_salary	min_salary	max_salary
6	360000	60000	48000	72000

Query 2: String Functions - name in uppercase and its length

```
SQL> SELECT name, UPPER(name) AS name_upper, LENGTH(name) AS name_length
FROM patients;
```

Resultant Query

patient_name	name_upper	name_length
Arun Kumar	ARUN KUMAR	10
Meena Ravi	MEENA RAVI	10

Kavin Raj	KAVIN RAJ	9
Priya Devi	PRIYA DEVI	10
Nithya S	NITHYA S	8
Vijay Anand	VIJAY ANAND	11

Query 3: Date Functions - year of joining from admission_date

SQL> SELECT name, admission_date, EXTRACT(YEAR FROM admission_date) AS join_year FROM patients
ORDER BY admission_date; **Resultant Query**

patient_name	admission_date	admission_year
Arun Kumar	2012-03-15	2012
Meena Ravi	2015-07-22	2015
Kavin Raj	2017-01-10	2017
Priya Devi	2019-11-05	2019
Nithya S	2021-09-30	2021
Vijay Anand	2023-04-18	2023

RESULT:

Thus, the aggregate, string, and date functions were successfully applied on the Patients table.


TRUTH-TOIL-TRIUMPH
AUTONOMOUS

EX. NO: 7 Creation of Subqueries and Nested Queries using Patients Table
DATE:27/07/2026

Aim

To retrieve records from the Patients table using single-row, multi-row, correlated, and nested subqueries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery (Nested Query) is a query written inside another SQL query. The inner query is executed first and its result is used by the outer query. Subqueries can return a single value, multiple values, or be correlated with the outer query row by row.

SUBQUERY TYPES:

- Single-Row Subquery
- Multi-Row Subquery
- Correlated Subquery
- Subquery with EXISTS

SINGLE-ROW SUBQUERY:

A subquery that returns only one row to the outer query. It is used with comparison operators such as =, >, <.

Synta

MULTI-ROW SUBQUERY:

A subquery that returns more than one row to the outer query. It is used with operators such as IN, ANY, ALL.srty654wq

Syntax:

SELECT columns FROM table_name

WHERE column IN (SELECT column FROM table_name WHERE condition);

CORRELATED SUBQUERY:

A subquery that refers to a column of the outer query. It is executed once for every row processed by the outer query.

Syntax:

SELECT columns FROM table_name outer_alias

WHERE column operator (SELECT aggregate_function(column) FROM table_name inner_alias

WHERE inner_alias.column = outer_alias.column);

SUBQUERY WITH EXISTS:

The EXISTS operator is used to test whether the subquery returns at least one row. It returns TRUE if the subquery result set is not empty.

Syntax:

SELECT columns FROM table_name

WHERE EXISTS (SELECT 1 FROM table_name WHERE condition);

Explanation

- Create the patients table and the wards table.
- Retrieve patients whose salary is greater than the average salary using a single-row subquery.
  ● Retrieve patients belonging to wards located on the upper floors using a multi-row subquery.
- Retrieve patients earning more than the average salary of their own ward using a correlated subquery.
- Retrieve wards that have at least one patient using EXISTS.

Create Table

```
CREATE TABLE patients (  id
INTEGER PRIMARY KEY,
name TEXT,  ward TEXT,
admission_year INTEGER,
salary DECIMAL(10,2),
admission_date DATE
);
```

Create Table (Wards) CREATE

```
TABLE wards (  ward_id INTEGER
PRIMARY KEY,  ward_name TEXT,
room_no TEXT
);
```

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');
INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');
```

Sample Data (Wards)

```
INSERT INTO wards VALUES (1, 'Emergency', 'G-101');
INSERT INTO wards VALUES (2, 'General', 'G-102');
INSERT INTO wards VALUES (3, 'Cardiology', 'F1-201');
INSERT INTO wards VALUES (4, 'Orthopedic', 'F2-202');
INSERT INTO wards VALUES (5, 'Pediatrics', 'F3-301');
```

SQL Query

Query 1: Single-row subquery - patients earning more than the average salary

```
SQL> SELECT name, ward, salary
```

FROM patients

WHERE salary > (SELECT AVG(salary) FROM patients);

Resultant Query

patient_name	ward	salary
Suresh Babu	Cardiology	61000
Priya Sharma	Orthopedic	58000
Karthik Raja	Cardiology	63000

Query 2: Multi-row subquery - patients belonging to wards on the upper floors

SQL> SELECT name, ward

FROM patients

WHERE ward IN (SELECT ward_name FROM wards WHERE room_no LIKE 'F%'); **Resultant Query**

patient_name	ward
Suresh Babu	Cardiology
Priya Sharma	Orthopedic
Karthik Raja	Cardiology

Query 3: Correlated subquery - patients earning more than the average salary of their own ward

SQL> SELECT p1.name, p1.ward, p1.salary

FROM patients p1

WHERE p1.salary > (SELECT AVG(p2.salary) FROM patients p2

WHERE p2.ward = p1.ward);

Resultant Query

patient_name	ward	salary
Anjali Mehta	General	52000
Karthik Raja	Cardiology	63000

Query 4: Subquery with EXISTS - wards that have at least one patient

SQL> SELECT ward_name FROM wards w

WHERE EXISTS (SELECT 1 FROM patients p WHERE p.ward = w.ward_name); **Resultant Query**

ward_name
Emergency
General

Cardiology
Orthopedic

RESULT:

The patient records were successfully retrieved using single-row, multi-row, correlated, and EXISTS-based subqueries and nested queries.

EX. NO: 8 Creation of Views, Indexes, and Sequences using Patients Table
DATE: 30/07/2026

Aim

To create a view, an index, and a sequence on the Patients table and to use them to retrieve and insert data.

Software Used

Oracle / MySQL / sqlite3

Description

A View is a virtual table based on the result of a SELECT query. An Index is a database object that improves the speed of data retrieval. A Sequence is a database object used to automatically generate unique numeric values, typically for primary keys.

OBJECTS COVERED:

- View
- Index
- Sequence

VIEW:

A view does not store data physically; it stores the query definition and displays data from the underlying table(s) whenever it is queried.

Syntax:

CREATE VIEW view_name AS

SELECT columns FROM table_name WHERE condition;

INDEX:

An index is created on one or more columns of a table to speed up search operations on those columns.

Syntax:

CREATE INDEX index_name ON table_name (column_name);

SEQUENCE:

A sequence generates a sequential series of numbers, commonly used to auto-populate primary key columns.

Syntax:

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY step_value;

Explanation

- Create a view to display only the Cardiology patients.
- Create an index on the ward column to speed up ward-based searches.
- Create a sequence to auto-generate the next patient id.
- Insert a new patient using the sequence and display the record.

Create Table

```
CREATE TABLE patients (
id INTEGER PRIMARY KEY,
name TEXT,  ward TEXT,
admission_year INTEGER,
salary DECIMAL(10,2),
admission_date DATE
);
```

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');
INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');
```

SQL Query

Query 1: Create a view of Cardiology patients

```
SQL> CREATE VIEW cardiology_patients AS
SELECT id, name, salary FROM patients WHERE ward = 'Cardiology';
```

View created.

Query 2: Display records from the view

```
SQL> SELECT * FROM cardiology_patients;
```

id	patient_name	salary
3	Suresh Babu	61000
5	Karthik Raja	63000

Query 3: Create an index on the ward column

```
SQL> CREATE INDEX idx_ward ON patients (ward);
SQL> CREATE SEQUENCE patient_seq
SQL> SELECT * FROM patients WHERE id = 7;
```

id	patient_name	ward	admission_year	salary	admission_date
7	Meena Iyer	Emergency	2023	47000	2023-05-12



RESULT:

The view, index, and sequence were successfully created and used on the Patients table.

EX. NO: 9 Creation of Stored Procedures and Functions using Patients Table
DATE:

Aim

To create a stored procedure to insert a patient record and a function to return the number of patients in a given ward.

Software Used

Oracle / MySQL / sqlite3

Description

A Stored Procedure is a named PL/SQL block that performs a specific task and can be called repeatedly. A Function is similar to a procedure but must return a single value using the RETURN statement and can be used inside SQL expressions.

OBJECTS COVERED:

- Stored Procedure
- Function

STORED PROCEDURE:

A stored procedure groups one or more SQL/PL-SQL statements into a callable unit. It may or may not return a value.

Syntax:

CREATE OR REPLACE PROCEDURE procedure_name (parameters)

A function always returns a single value of a specified data type and can be used within a SELECT statement.

Syntax:

CREATE OR REPLACE FUNCTION function_name (parameters)

- Create a stored procedure add_patient to insert a new patient record.
- Call the procedure to insert a new patient.
- Create a function get_ward_count to return the number of patients in a given ward.
- Call the function to display the patient count of the General ward.

Create Table

CREATE TABLE patients (

INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');

INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');

INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');

INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');

INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');

INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18'); IS

Query 1: Create a stored procedure to insert a new patient

SQL> CREATE OR REPLACE PROCEDURE add_patient (

p_id IN NUMBER,

```
p_name IN VARCHAR2,
p_ward IN VARCHAR2,
p_year IN NUMBER,
p_salary IN NUMBER,  p_date
IN DATE
)
IS
BEGIN
INSERT INTO patients VALUES (p_id, p_name, p_ward, p_year, p_salary, p_date);
COMMIT;
END;
/

Procedure created.
```

Q

```
SQL> EXECUTE add_patient(8, 'Arjun Nair', 'General', 2024, 46000, '2024-02-10');

Procedure executed. 1 row created.
```

Resultant Query

id	patient_name	ward	admission_year	salary	admission_date
8	Arjun Nair	General	2024	46000	2024-02-10

Query 3: Create a function to return the patient count of a ward

```
SQL> CREATE OR REPLACE FUNCTION get_ward_count (p_ward IN VARCHAR2)
RETURN NUMBER
IS
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM patients WHERE ward = p_ward;
RETURN v_count;
END;
/

Function created.
```

Query 4: Call the function to display the count of patients in the General ward

```
SQL> SELECT get_ward_count('General') AS total_general FROM dual;
```

Resultant Query

total_general
3



RESULT:

The stored procedure add_patient and the function get_ward_count were successfully created and executed on the Patients table.

EX. NO: 10 Creation of Triggers using Patients Table
DATE: 05/08/20216

Aim

To create triggers that automatically log changes made to the Patients table and validate data before insertion.

Software Used

Oracle / MySQL / sqlite3

Description

A Trigger is a stored PL/SQL block that is automatically executed (fired) when a specified DML event, such as INSERT, UPDATE, or DELETE, occurs on a table.

TRIGGER TYPES:

- BEFORE Trigger
- AFTER Trigger
- Row-Level Trigger
- Statement-Level Trigger

BEFORE / AFTER TRIGGER:

A BEFORE trigger fires before the triggering DML statement executes, while an AFTER trigger fires after it executes.

Syntax:

```
CREATE OR REPLACE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
FOR EACH ROW
BEGIN
-- trigger body
END;
/
```

Explanation

- Create a patients_audit table to record ward changes.
- Create an AFTER UPDATE trigger that logs the old and new ward whenever a patient's ward is updated.
- Update a patient's ward and verify that the change is logged in the audit table.
- Create a BEFORE INSERT trigger that prevents insertion of a patient record with a negative salary.

Create Table

```
CREATE TABLE patients ( id
INTEGER PRIMARY KEY,
name TEXT, ward TEXT,
admission_year INTEGER,
salary DECIMAL(10,2),
admission_date DATE
```

);

Create Table (Audit)

CREATE TABLE patients_audit (audit_id

INTEGER PRIMARY KEY,

patient_id INTEGER,

old_ward TEXT, new_ward

TEXT, changed_on DATE

);

Sample Data

INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');

INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');

INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');

INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');

INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');

INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');

SQL Query

Query 1: Create a trigger to log ward updates

```
SQL> CREATE OR REPLACE TRIGGER trg_ward_update
AFTER UPDATE OF ward ON patients
FOR EACH ROW
BEGIN
INSERT INTO patients_audit (patient_id, old_ward, new_ward, changed_on)
VALUES (:OLD.id, :OLD.ward, :NEW.ward, SYSDATE);
END;
/
```

Trigger created.

Query 2: Update a patient's ward to fire the trigger

```
SQL> UPDATE patients SET ward = 'Neurology' WHERE id = 4;
```

1 row updated.

Query 3: Display the logged change from the audit table

```
SQL> SELECT * FROM patients_audit;
```

Resultant Query

audit_id	patient_id	old_ward	new_ward	changed_on

1	4	Orthopedic	Neurology	09-SEP-26

Query 4: Create a trigger to validate salary before insertion

```
SQL> CREATE OR REPLACE TRIGGER trg_check_salary
BEFORE INSERT ON patients
FOR EACH ROW
BEGIN
IF :NEW.salary < 0 THEN
RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative');
END IF;
END;
/
```

Trigger created.

Query 5: Attempt to insert a patient with a negative salary

```
SQL> INSERT INTO patients VALUES (9, 'Test Patient', 'General', 2024, -5000, '2024-03-01');
```

Output

```
ORA-20001: Salary cannot be negative ORA-06512:
 at "TRG_CHECK_SALARY", line 3
```

RESULT:

The triggers trg_ward_update and trg_check_salary were successfully created and verified on the Patients table.

EX. NO: 11 Cursor Implementation using Patients Table
DATE: 08/08/2026

Aim

To implement an explicit cursor to retrieve and display patient records row by row from the Patients table.

Software Used

Oracle / MySQL / sqlite3

Description

A Cursor is a pointer to the context area used by Oracle to process SQL statements. An Implicit cursor is created automatically for single SQL statements, while an Explicit cursor is declared by the programmer to process a multi-row result set row by row.

CURSOR ATTRIBUTES:

- %FOUND
- %NOTFOUND
- %ROWCOUNT
- %ISOPEN

EXPLICIT CURSOR:

An explicit cursor is declared in the DECLARE section, opened, fetched row by row in a loop, and closed after use.

Syntax:

DECLARE

```
CURSOR cursor_name IS SELECT columns FROM table_name WHERE condition;

variables ...

BEGIN

OPEN cursor_name;

LOOP

FETCH cursor_name INTO variables;

EXIT WHEN cursor_name%NOTFOUND;

-- process row

END LOOP;

CLOSE cursor_name;

END;

/
```

Explanation

- Declare a cursor to select all Cardiology patients.
- Open the cursor, fetch each row, and display the patient name, ward, and salary.
- Close the cursor after all rows are processed.

- Declare a second cursor to display every patient along with its row number using %ROWCOUNT.

Create Table

```
CREATE TABLE patients (  id
INTEGER PRIMARY KEY,
name TEXT,  ward TEXT,
admission_year INTEGER,
salary DECIMAL(10,2),
admission_date DATE
);
```

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');
INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');
INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');
INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');
INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');
INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');
```

SQL Query

Query 1: Cursor to display all Cardiology patients

```
SQL> DECLARE
CURSOR c_patients IS
SELECT name, ward, salary FROM patients WHERE ward = 'Cardiology';
v_name patients.name%TYPE;  v_ward patients.ward%TYPE;  v_salary
patients.salary%TYPE;
BEGIN
OPEN c_patients;
LOOP
FETCH c_patients INTO v_name, v_ward, v_salary;
EXIT WHEN c_patients%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_ward || ' - ' || v_salary);
END LOOP;
CLOSE c_patients;
END;
/
```

Output

```
Suresh Babu - Cardiology - 61000
Karthik Raja - Cardiology - 63000
```

PL/SQL procedure successfully completed.

Query 2: Cursor with %ROWCOUNT to display all patients with row numbers

```
SQL> DECLARE
CURSOR c_all IS SELECT name FROM patients;  v_name
patients.name%TYPE;
BEGIN
OPEN c_all;
LOOP
FETCH c_all INTO v_name;
EXIT WHEN c_all%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('Row ' || c_all%ROWCOUNT || ': ' || v_name);
END LOOP;
CLOSE c_all;
END;
/
```

Output

Row 1: Ravi Kumar
Row 2: Anjali Mehta
Row 3: Suresh Babu
Row 4: Priya Sharma
Row 5: Karthik Raja
Row 6: Divya Nair

PL/SQL procedure successfully completed.

RESULT:

The explicit cursors were successfully declared, opened, fetched, and closed to display the Patients table records row by row.

EX. NO: 12 PL/SQL Exception Handling using Patients Table
DATE: 11/08/2026

Aim

To handle predefined and user-defined exceptions while accessing and validating records in the Patients table.

Software Used

Oracle / MySQL / sqlite3

Description

Exception Handling in PL/SQL is used to handle runtime errors so that the normal flow of the program is maintained. Exceptions can be Predefined (raised automatically by Oracle, such as NO_DATA_FOUND and TOO_MANY_ROWS) or User-Defined (declared and raised explicitly by the programmer).

EXCEPTION TYPES:

- Predefined Exception
- User-Defined Exception

PREDEFINED EXCEPTION:

Predefined exceptions are raised automatically by the Oracle engine when an internal error occurs, such as NO_DATA_FOUND or TOO_MANY_ROWS.

Syntax:

```
BEGIN
-- statements
EXCEPTION
WHEN exception_name THEN
-- handler statements
END;
/
```

USER-DEFINED EXCEPTION:

A user-defined exception is declared in the DECLARE section and raised explicitly using the RAISE statement when a business condition is violated.

Syntax:

```
DECLARE
exception_name EXCEPTION;
BEGIN
IF condition THEN
RAISE exception_name;
END IF;
EXCEPTION
```

WHEN exception_name THEN

-- handler statements

END;

/

Explanation

- Handle the NO_DATA_FOUND exception when searching for a non-existent patient id.
- Handle the TOO_MANY_ROWS exception when a query unexpectedly returns more than one row into a scalar variable.
- Declare and raise a user-defined exception to reject a negative salary value.

Create Table

CREATE TABLE patients (id

INTEGER PRIMARY KEY,

name TEXT, ward TEXT,

admission_year INTEGER,

salary DECIMAL(10,2),

admission_date DATE

);

Sample Data

INSERT INTO patients VALUES (1, 'Ravi Kumar', 'Emergency', 2010, 45000, '2010-03-15');

INSERT INTO patients VALUES (2, 'Anjali Mehta', 'General', 2013, 52000, '2013-07-22');

INSERT INTO patients VALUES (3, 'Suresh Babu', 'Cardiology', 2016, 61000, '2016-01-10');

INSERT INTO patients VALUES (4, 'Priya Sharma', 'Orthopedic', 2018, 58000, '2018-11-05');

INSERT INTO patients VALUES (5, 'Karthik Raja', 'Cardiology', 2019, 63000, '2019-09-30');

INSERT INTO patients VALUES (6, 'Divya Nair', 'General', 2021, 49000, '2021-04-18');

SQL Query

Query 1: Predefined exception - NO_DATA_FOUND

SQL> DECLARE v_name patients.name%TYPE;

BEGIN

SELECT name INTO v_name FROM patients WHERE id = 50;

DBMS_OUTPUT.PUT_LINE(v_name);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('No patient found with the given id.');

END;

/

Output

No patient found with the given id.

PL/SQL procedure successfully completed.

Query 2: Predefined exception - TOO_MANY_ROWS

```
SQL> DECLARE  v_name patients.name%TYPE;
BEGIN
SELECT name INTO v_name FROM patients WHERE ward = 'General';
DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE('More than one patient found in this ward.');
END;
/
```

Output

More than one patient found in this ward.

PL/SQL procedure successfully completed.

Query 3: User-defined exception - invalid salary SQL>

```
DECLARE
invalid_salary EXCEPTION;  v_salary
patients.salary%TYPE := -2000;
BEGIN
IF v_salary < 0 THEN
RAISE invalid_salary;
END IF;
EXCEPTION
WHEN invalid_salary THEN
DBMS_OUTPUT.PUT_LINE('Error: Salary cannot be negative.');
END;
/
```

Output

Error: Salary cannot be negative.

PL/SQL procedure successfully completed.



RESULT:

The predefined exceptions NO_DATA_FOUND and TOO_MANY_ROWS, and a user-defined exception, were successfully handled in PL/SQL blocks on the Patients table.

EX. NO: 13 Database Normalization (1NF, 2NF, 3NF, BCNF) using Patients Table
DATE: 27/08/2026

Aim

To normalize an unnormalized Patient-Treatment table into First Normal Form, Second Normal Form, Third Normal Form, and Boyce-Codd Normal Form.

Software Used

Oracle / MySQL / sqlite3

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid update, insertion, and deletion anomalies. It is carried out in progressive stages called normal forms, each removing a specific type of dependency.

NORMAL FORMS:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF) ● Boyce-Codd Normal Form (BCNF)

1NF:
A table is in 1NF if all attributes contain only atomic (indivisible) values and there are no repeating groups.

2NF:
A table is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the whole of every candidate key, i.e., there is no partial dependency on part of a composite key.

3NF:
A table is in 3NF if it is in 2NF and has no transitive dependency, i.e., no non-key attribute depends on another non-key attribute.

BCNF:
A table is in BCNF if, for every functional dependency X -> Y, X is a candidate key of the table. BCNF is a stricter version of 3NF.

Explanation

- Consider an unnormalized Patient_Treatment table storing patient, disease, and doctor details together, with multiple diseases stored in a single row.
- Convert the table to 1NF by splitting the repeating disease values into separate rows.
- Convert the table to 2NF by removing attributes that depend only on part of the composite key.
- Convert the table to 3NF by removing the transitive dependency of room number on ward.
- Convert the table to BCNF by verifying that every determinant is a candidate key.

Unnormalized Table (UNF)

patient_id	patient_name	diseases	doctor_id	doctor_name	doctor_specialization	ward	room_no
101	Ravi Kumar	Fever, Cold	D1	Dr. Anand	General Medicine	Emergency	G-101

102	Anjali Mehta	Diabetes	D2	Dr. Kumar	Endocrinology	General	G-102

First Normal Form (1NF)

The repeating disease values are split so that every cell holds a single atomic value; the key becomes (patient_id, disease).

patient_id	patient_name	disease	doctor_id	doctor_name	doctor_specialization	ward	room_no
101	Ravi Kumar	Fever	D1	Dr. Anand	General Medicine	Emergency	G-101
101	Ravi Kumar	Cold	D1	Dr. Anand	General Medicine	Emergency	G-101
102	Anjali Mehta	Diabetes	D2	Dr. Kumar	Endocrinology	General	G-102

Second Normal Form (2NF)

patient_name, doctor_id, doctor_name, doctor_specialization, ward, and room_no depend only on patient_id, not on the full composite key (patient_id, disease). This partial dependency is removed by splitting into three tables.

Patient (patient_id, patient_name, doctor_id, ward, room_no)

Patient_Disease (patient_id, disease)

Doctor (doctor_id, doctor_name, doctor_specialization)

Third Normal Form (3NF)

In the Patient table, room_no depends on ward, and ward depends on patient_id, giving the transitive dependency patient_id -> ward -> room_no. This is removed by separating ward details into their own table. Patient (patient_id, patient_name, doctor_id, ward)

Ward (ward, room_no)

Patient_Disease (patient_id, disease)

Doctor (doctor_id, doctor_name, doctor_specialization)

Boyce-Codd Normal Form (BCNF)

Every determinant in each table is checked against the candidate keys:

- Patient: patient_id -> patient_name, doctor_id, ward; patient_id is the candidate key. BCNF satisfied.
- Ward: ward -> room_no; ward is the candidate key. BCNF satisfied.
- Doctor: doctor_id -> doctor_name, doctor_specialization; doctor_id is the candidate key. BCNF satisfied.
- Patient_Disease: composite key (patient_id, disease) with no non-key attributes. BCNF satisfied trivially.

SQL Query

Query 1: Final BCNF schema creation

SQL> CREATE TABLE doctor (doctor_id

VARCHAR2(10) PRIMARY KEY,

```
doctor_name TEXT,  doctor_specialization
TEXT
);

SQL> CREATE TABLE ward (  ward_name
TEXT PRIMARY KEY,  room_no TEXT
);

SQL> CREATE TABLE patient (
patient_id INTEGER PRIMARY KEY,
patient_name TEXT,  doctor_id
VARCHAR2(10),  ward_name TEXT,
FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id),
FOREIGN KEY (ward_name) REFERENCES ward(ward_name)
);

SQL> CREATE TABLE patient_disease (
patient_id INTEGER,  disease TEXT,
PRIMARY KEY (patient_id, disease),
FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
);

Tables created.
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



RESULT:

The unnormalized Patient_Treatment table was successfully normalized step by step into 1NF, 2NF, 3NF, and BCNF, removing repeating groups, partial dependency, and transitive dependency.

EX.NO: 14 **MINI PROJECT**

DATE: 04/10/2026

HOSPITAL APPOINTMENT SCHEDULING AND QUEUE MANAGEMENT

AIM:

To develop a Hospital Appointment Scheduling and Queue Management system using SQL and PL/SQL to store, manage, process, and analyze patient appointment records, doctor availability, waiting time, and appointment status.

SOFTWARE USED:

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

DESCRIPTION:

The Hospital Appointment Scheduling and Queue Management system is developed to manage patient appointments and queue information. The system stores patient details, doctor details, department, appointment date and time, check-in time, consultation start time, and appointment status. SQL queries are used to retrieve and analyze appointment information, while PL/SQL procedures are used to schedule appointments, calculate waiting time, generate daily appointment reports, and handle exceptions.

DATABASE TABLES:

TABLE CREATION:

```
CREATE TABLE HOSPITAL_APPOINTMENT (
        APPOINTMENT_ID INTEGER PRIMARY KEY,
        PATIENT_ID INTEGER,
        PATIENT_NAME TEXT,
        DOCTOR_ID INTEGER,
        DOCTOR_NAME TEXT,
        DEPARTMENT TEXT,
        APPOINTMENT_DATETIME TEXT,
```

```
    CHECKIN_TIME TEXT,
    CONSULTATION_START_TIME TEXT,
     STATUS TEXT
);
```

Table Created

The HOSPITAL_APPOINTMENT table was created successfully with primary key and NOT NULL constraints.

DATASET INSERTION:

```
INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1001,101,'Arun',201,'Dr. Ravi','Cardiology','2026-10-04 09:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1002,102,'Priya',202,'Dr. Meena','Dermatology','2026-10-04 09:30:00','Pending');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1003,103,'Karthik',203,'Dr. Kumar','Orthopedics','2026-10-04 10:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1004,104,'Divya',204,'Dr. Anitha','Pediatrics','2026-10-04 10:30:00','Cancelled');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1005,105,'Vijay',201,'Dr. Ravi','Cardiology','2026-10-04 11:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1006,106,'Sneha',205,'Dr. Suresh','Neurology','2026-10-04 11:30:00','Pending');

INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEP
```

ARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1007,107,'Rahul',202,'Dr. Meena','Dermatology','2026-10-04 12:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1008,108,'Anitha',206,'Dr. Geetha','Gynecology','2026-10-04 12:30:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1009,109,'Manoj',203,'Dr. Kumar','Orthopedics','2026-10-05 09:00:00','Pending');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1010,110,'Kavya',204,'Dr. Anitha','Pediatrics','2026-10-05 09:30:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1011,111,'Surya',201,'Dr. Ravi','Cardiology','2026-10-05 10:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1012,112,'Ramya',205,'Dr. Suresh','Neurology','2026-10-05 10:30:00','Cancelled');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1013,113,'Ajith',206,'Dr. Geetha','Gynecology','2026-10-05 11:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1014,114,'Deepika',202,'Dr. Meena','Dermatology','2026-10-05 11:30:00','Pending');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1015,115,'Sanjay',203,'Dr. Kumar','Orthopedics','2026-10-05 12:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEP

ARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1016,116,'Pooja',204,'Dr. Anitha','Pediatrics','2026-10-06 09:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1017,117,'Kiran',201,'Dr. Ravi','Cardiology','2026-10-06 09:30:00','Pending');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1018,118,'Nisha',205,'Dr. Suresh','Neurology','2026-10-06 10:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1019,119,'Varun',206,'Dr. Geetha','Gynecology','2026-10-06 10:30:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1020,120,'Swathi',202,'Dr. Meena','Dermatology','2026-10-06 11:00:00','Cancelled');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1021,121,'Ashok',203,'Dr. Kumar','Orthopedics','2026-10-06 11:30:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1022,122,'Meera',204,'Dr. Anitha','Pediatrics','2026-10-06 12:00:00','Pending');

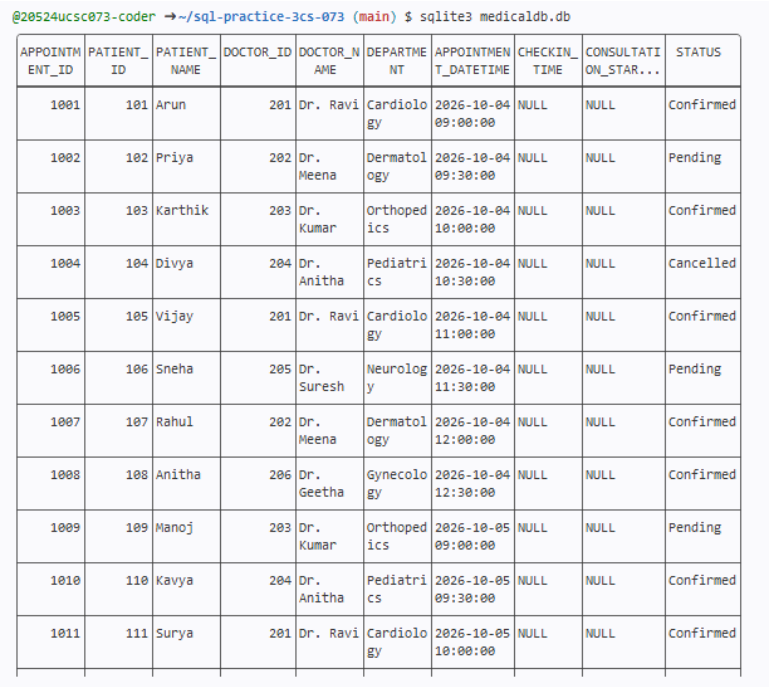
INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1023,123,'Bala',201,'Dr. Ravi','Cardiology','2026-10-07 09:00:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1024,124,'Lakshmi',205,'Dr. Suresh','Neurology','2026-10-07 09:30:00','Confirmed');

INSERT INTO HOSPITAL_APPOINTMENT (APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEP

ARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1025,125,'Ganesh',206,'Dr. Geetha','Gynecology','2026-10-07 10:00:00','Confirmed');

OUTPUT:



```
@20524ucsc073-coder ➜~/sql-practice-3cs-073 (main) $ sqlite3 medicaldb.db
```

APPOINTMENT_ID	PATIENT_ID	PATIENT_NAME	DOCTOR_ID	DOCTOR_NAME	DEPARTMENT	APPOINTMENT_DATETIME	CHECKIN_TIME	CONSULTATION_STAR...	STATUS
1001	101	Arun	201	Dr. Ravi	Cardiology	2026-10-04 09:00:00	NULL	NULL	Confirmed
1002	102	Priya	202	Dr. Meena	Dermatology	2026-10-04 09:30:00	NULL	NULL	Pending
1003	103	Karthik	203	Dr. Kumar	Orthopedics	2026-10-04 10:00:00	NULL	NULL	Confirmed
1004	104	Divya	204	Dr. Anitha	Pediatrics	2026-10-04 10:30:00	NULL	NULL	Cancelled
1005	105	Vijay	201	Dr. Ravi	Cardiology	2026-10-04 11:00:00	NULL	NULL	Confirmed
1006	106	Sneha	205	Dr. Suresh	Neurology	2026-10-04 11:30:00	NULL	NULL	Pending
1007	107	Rahul	202	Dr. Meena	Dermatology	2026-10-04 12:00:00	NULL	NULL	Confirmed
1008	108	Anitha	206	Dr. Geetha	Gynecology	2026-10-04 12:30:00	NULL	NULL	Confirmed
1009	109	Manoj	203	Dr. Kumar	Orthopedics	2026-10-05 09:00:00	NULL	NULL	Pending
1010	110	Kavya	204	Dr. Anitha	Pediatrics	2026-10-05 09:30:00	NULL	NULL	Confirmed
1011	111	Surya	201	Dr. Ravi	Cardiology	2026-10-05 10:00:00	NULL	NULL	Confirmed

Records Inserted

A total of 8 hospital appointment records were inserted into the HOSPITAL_APPOINTMENT table.

SQL QUERY 1: Display all appointments scheduled for a specified date.

SELECT APPOINTEMT_ID, PATIENT_NAME, DOCTOR_NAME, DEPARTMENT, APPOINTMENT_DATETIME, STATUS

FROM HOSPITAL_APPOINTMENT

WHERE DATE(APPOINTMENT_DATETIME)='2026-10-04'

ORDER BY APPOINTMENT_DATETIME;

OUTPUT:

```
PARTMENT,APPOINTMENT_DATETIME,STATUS) VALUES (1025,125,'Ganesh',206,'Dr. Geetha','Gynecology','20
26-10-07 10:00:00','Confirmed');SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,DEPARTMENT,APPOINT
MENT_DATETIME,STATUS
FROM HOSPITAL_APPOINTMENT
WHERE DATE(APPOINTMENT_DATETIME)='2026-10-04'
ORDER BY APPOINTMENT_DATETIME;
```

APPOINTMENT_ID	PATIENT_NAME	DOCTOR_NAME	DEPARTMENT	APPOINTMENT_DATETIME	STATUS
1001	Arun	Dr. Ravi	Cardiology	2026-10-04 09:00:00	Confirmed
1002	Priya	Dr. Meena	Dermatology	2026-10-04 09:30:00	Pending
1003	Karthik	Dr. Kumar	Orthopedics	2026-10-04 10:00:00	Confirmed
1004	Divya	Dr. Anitha	Pediatrics	2026-10-04 10:30:00	Cancelled
1005	Vijay	Dr. Ravi	Cardiology	2026-10-04 11:00:00	Confirmed
1006	Sneha	Dr. Suresh	Neurology	2026-10-04 11:30:00	Pending
1007	Rahul	Dr. Meena	Dermatology	2026-10-04 12:00:00	Confirmed
1008	Anitha	Dr. Geetha	Gynecology	2026-10-04 12:30:00	Confirmed

SQL Query 2: Count appointments assigned to each doctor.

```
SELECT DOCTOR_ID,DOCTOR_NAME,COUNT(*) AS TOTAL_APPOINTMENTS

FROM HOSPITAL_APPOINTMENT

GROUP BY DOCTOR_ID,DOCTOR_NAME

ORDER BY DOCTOR_ID;
```

OUTPUT:

```
sqlite> SELECT DOCTOR_ID,DOCTOR_NAME,COUNT(*) AS TOTAL_APPOINTMENTS
FROM HOSPITAL_APPOINTMENT
GROUP BY DOCTOR_ID,DOCTOR_NAME
ORDER BY DOCTOR_ID;
```

DOCTOR_ID	DOCTOR_NAME	TOTAL_APPOINTMENTS
201	Dr. Ravi	5
202	Dr. Meena	4
203	Dr. Kumar	4
204	Dr. Anitha	4
205	Dr. Suresh	4
206	Dr. Geetha	4

SQL Query 3: Identify cancelled and missed appointments.

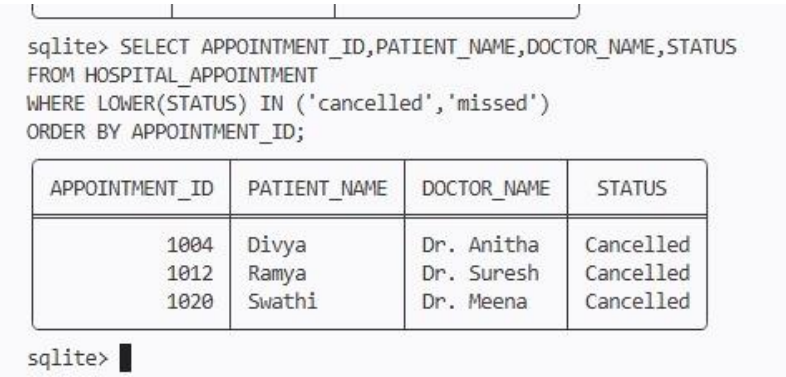
```
SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,STATUS

FROM HOSPITAL_APPOINTMENT
```

WHERE LOWER(STATUS) IN ('cancelled','missed')

ORDER BY APPOINTMENT_ID;

OUTPUT:



```
sqlite> SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,STATUS
FROM HOSPITAL_APPOINTMENT
WHERE LOWER(STATUS) IN ('cancelled','missed')
ORDER BY APPOINTMENT_ID;
```

APPOINTMENT_ID	PATIENT_NAME	DOCTOR_NAME	STATUS
1004	Divya	Dr. Anitha	Cancelled
1012	Ramya	Dr. Suresh	Cancelled
1020	Swathi	Dr. Meena	Cancelled

```
sqlite>
```

SQL Query 4: Calculate average waiting time for each department.

UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 08:50:00',CONSULTATION_START_TIME='2026-10-04 09:10:00'

WHERE APPOINTMENT_ID=1001;

UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 09:15:00',CONSULTATION_START_TIME='2026-10-04 09:45:00'

WHERE APPOINTMENT_ID=1002;

UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 09:40:00',CONSULTATION_START_TIME='2026-10-04 10:20:00'

WHERE APPOINTMENT_ID=1003;

```
UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 10:40:00',CONSULTATION_START_TIME='2026-10-04
11:10:00'

WHERE APPOINTMENT_ID=1005;


UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 11:10:00',CONSULTATION_START_TIME='2026-10-04
11:50:00'

WHERE APPOINTMENT_ID=1006;


UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 11:45:00',CONSULTATION_START_TIME='2026-10-04
12:15:00'

WHERE APPOINTMENT_ID=1007;


UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-04 12:15:00',CONSULTATION_START_TIME='2026-10-04
12:50:00'

WHERE APPOINTMENT_ID=1008;


UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-05 08:40:00',CONSULTATION_START_TIME='2026-10-05
09:20:00'

WHERE APPOINTMENT_ID=1009;
```

```
UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-05 09:10:00',CONSULTATION_START_TIME='2026-10-05
09:45:00'

WHERE APPOINTMENT_ID=1010;


UPDATE HOSPITAL_APPOINTMENT

SET CHECKIN_TIME='2026-10-05 09:40:00',CONSULTATION_START_TIME='2026-10-05
10:15:00'

WHERE APPOINTMENT_ID=1011;

SELECT DEPARTMENT,

ROUND(AVG((julianday(CONSULTATION_START_TIME)-
julianday(CHECKIN_TIME))*1440),2) AS AVG_WAITING_MINUTES

FROM HOSPITAL_APPOINTMENT

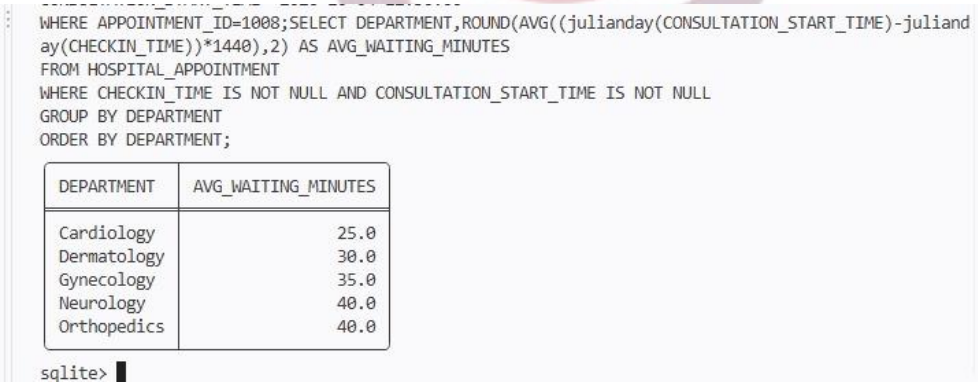
WHERE CHECKIN_TIME IS NOT NULL

AND CONSULTATION_START_TIME IS NOT NULL

GROUP BY DEPARTMENT

ORDER BY DEPARTMENT;
```

OUTPUT:



```
WHERE APPOINTMENT_ID=1008;SELECT DEPARTMENT,ROUND(AVG((julianday(CONSULTATION_START_TIME)-juliand
ay(CHECKIN_TIME))*1440),2) AS AVG_WAITING_MINUTES
FROM HOSPITAL_APPOINTMENT
WHERE CHECKIN_TIME IS NOT NULL AND CONSULTATION_START_TIME IS NOT NULL
GROUP BY DEPARTMENT
ORDER BY DEPARTMENT;
```

DEPARTMENT	AVG_WAITING_MINUTES
Cardiology	25.0
Dermatology	30.0
Gynecology	35.0
Neurology	40.0
Orthopedics	40.0

```
sqlite>
```

SQL Query 5: Identify patients with the longest waiting times.

```
SELECT PATIENT_NAME,DOCTOR_NAME,DEPARTMENT,

ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2)
AS WAITING_MINUTES

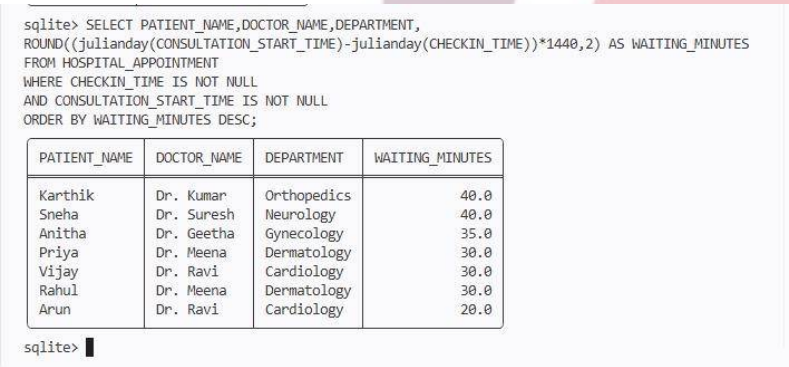
FROM HOSPITAL_APPOINTMENT

WHERE CHECKIN_TIME IS NOT NULL

AND CONSULTATION_START_TIME IS NOT NULL

ORDER BY WAITING_MINUTES DESC;
```

OUTPUT:



```
sqlite> SELECT PATIENT_NAME,DOCTOR_NAME,DEPARTMENT,
ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2) AS WAITING_MINUTES
FROM HOSPITAL_APPOINTMENT
WHERE CHECKIN_TIME IS NOT NULL
AND CONSULTATION_START_TIME IS NOT NULL
ORDER BY WAITING_MINUTES DESC;
```

PATIENT_NAME	DOCTOR_NAME	DEPARTMENT	WAITING_MINUTES
Karthik	Dr. Kumar	Orthopedics	40.0
Sneha	Dr. Suresh	Neurology	40.0
Anitha	Dr. Geetha	Gynecology	35.0
Priya	Dr. Meena	Dermatology	30.0
Vijay	Dr. Ravi	Cardiology	30.0
Rahul	Dr. Meena	Dermatology	30.0
Arun	Dr. Ravi	Cardiology	20.0

```
sqlite>
```

PL/SQL Procedure 1: Create a procedure to schedule an appointment after checking doctor availability.

```
CREATE TRIGGER CHECK_DOCTOR_AVAILABILITY

BEFORE INSERT ON HOSPITAL_APPOINTMENT

WHEN EXISTS (

SELECT 1

FROM HOSPITAL_APPOINTMENT

WHERE DOCTOR_ID=NEW.DOCTOR_ID

AND APPOINTMENT_DATETIME=NEW.APPOINTMENT_DATETIME

AND LOWER(STATUS) NOT IN ('cancelled','missed')
```

)

BEGIN

SELECT RAISE(ABORT,'Doctor is not available at this time.');

END;

INSERT INTO HOSPITAL_APPOINTMENT

(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS)

VALUES

(1026,126,'Ramesh',201,'Dr. Ravi','Cardiology','2026-10-07 11:00:00','Confirmed');

OUTPUT:

```
);
sqlite> INSERT INTO HOSPITAL_APPOINTMENT
(APPOINTMENT_ID,PATIENT_ID,PATIENT_NAME,DOCTOR_ID,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STA
TUS)
SELECT 1026,126,'Ramesh',201,'Dr. Ravi','Cardiology','2026-10-07 11:00:00','Confirmed'
WHERE NOT EXISTS (
SELECT 1
FROM HOSPITAL_APPOINTMENT
WHERE DOCTOR_ID=201
AND APPOINTMENT_DATETIME='2026-10-07 11:00:00'
AND LOWER(STATUS) NOT IN ('cancelled','missed')
);SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,DEPARTMENT,APPOINTMENT_DATETIME,STATUS
FROM HOSPITAL_APPOINTMENT
WHERE APPOINTMENT_ID=1026;
```

APPOINTMENT_ID	PATIENT_NAME	DOCTOR_NAME	DEPARTMENT	APPOINTMENT_DATETIME	STATUS
1026	Ramesh	Dr. Ravi	Cardiology	2026-10-07 11:00:00	Confirmed

```
sqlite>
```

PL/SQL Procedure 2: Create a function to calculate the waiting time for an appointment.

SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,

ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2) AS WAITING_MINUTES

FROM HOSPITAL_APPOINTMENT

WHERE CHECKIN_TIME IS NOT NULL

AND CONSULTATION_START_TIME IS NOT NULL

ORDER BY APPOINTMENT_ID;

OUTPUT:

```
Enter ".help" for usage hints.
sqlite> SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,
ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2) AS WAITING_MINUTES
FROM HOSPITAL_APPOINTMENT
WHERE CHECKIN_TIME IS NOT NULL
AND CONSULTATION_START_TIME IS NOT NULL
ORDER BY APPOINTMENT_ID;
```

APPOINTMENT_ID	PATIENT_NAME	DOCTOR_NAME	WAITING_MINUTES
1001	Arun	Dr. Ravi	20.0
1002	Priya	Dr. Meena	30.0
1003	Karthik	Dr. Kumar	40.0
1005	Vijay	Dr. Ravi	30.0
1006	Sneha	Dr. Suresh	40.0
1007	Rahul	Dr. Meena	30.0
1008	Anitha	Dr. Geetha	35.0

```
sqlite>
```

ONE PATIENT APPOINTMENT:

SELECT APPOINTMENT_ID,PATIENT_NAME,

ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2) AS WAITING_MINUTES

FROM HOSPITAL_APPOINTMENT

WHERE APPOINTMENT_ID=1001

AND CHECKIN_TIME IS NOT NULL

AND CONSULTATION_START_TIME IS NOT NULL;

OUTPUT:

```
sqlite> SELECT APPOINTMENT_ID,PATIENT_NAME,
ROUND((julianday(CONSULTATION_START_TIME)-julianday(CHECKIN_TIME))*1440,2) AS WAITING_MINUTES
FROM HOSPITAL_APPOINTMENT
WHERE APPOINTMENT_ID=1001
AND CHECKIN_TIME IS NOT NULL
AND CONSULTATION_START_TIME IS NOT NULL;
```

APPOINTMENT_ID	PATIENT_NAME	WAITING_MINUTES
1001	Arun	20.0

```
sqlite>
```

PL/SQL Procedure 3: Create a procedure to generate a daily appointment report, including completed, cancelled, and missed appointments.

SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,

DEPARTMENT,APPOINTMENT_DATETIME,STATUS

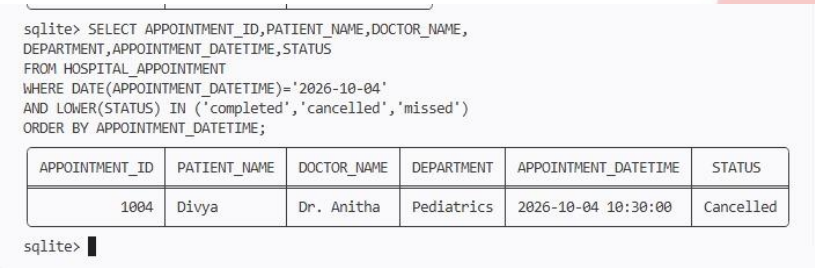
FROM HOSPITAL_APPOINTMENT

WHERE DATE(APPOINTMENT_DATETIME)='2026-10-04'

AND LOWER(STATUS) IN ('completed','cancelled','missed')

ORDER BY APPOINTMENT_DATETIME;

OUTPUT:



```
sqlite> SELECT APPOINTMENT_ID,PATIENT_NAME,DOCTOR_NAME,
DEPARTMENT,APPOINTMENT_DATETIME,STATUS
FROM HOSPITAL_APPOINTMENT
WHERE DATE(APPOINTMENT_DATETIME)='2026-10-04'
AND LOWER(STATUS) IN ('completed','cancelled','missed')
ORDER BY APPOINTMENT_DATETIME;
```

APPOINTMENT_ID	PATIENT_NAME	DOCTOR_NAME	DEPARTMENT	APPOINTMENT_DATETIME	STATUS
1004	Divya	Dr. Anitha	Pediatrics	2026-10-04 10:30:00	Cancelled

```
sqlite>
```

EXCEPTION HANDLING:

Test Case 1: Same doctor, same time

SELECT

CASE

WHEN EXISTS (

SELECT 1

FROM HOSPITAL_APPOINTMENT

WHERE DOCTOR_ID=201

AND APPOINTMENT_DATETIME='2026-10-04 09:00:00'

AND LOWER(STATUS) NOT IN ('cancelled','missed')

)

THEN 'EXCEPTION: Doctor is not available at this time'

ELSE 'Appointment can be scheduled successfully'

END AS RESULT;

OUTPUT:



Test Case 2: Available time

SELECT

CASE

WHEN EXISTS (

SELECT 1

FROM HOSPITAL_APPOINTMENT

WHERE DOCTOR_ID=201

AND APPOINTMENT_DATETIME='2026-10-07 11:00:00'

AND LOWER(STATUS) NOT IN ('cancelled','missed')

)

THEN 'Appointment cannot be scheduled'

ELSE 'Appointment can be scheduled successfully'

END AS RESULT;

OUTPUT:

```
sqlite> SELECT
CASE
WHEN EXISTS (
SELECT 1
FROM HOSPITAL_APPOINTMENT
WHERE DOCTOR_ID=201
AND APPOINTMENT_DATETIME='2026-10-07 11:00:00'
AND LOWER(STATUS) NOT IN ('cancelled','missed')
)
THEN 'Appointment cannot be scheduled'
ELSE 'Appointment can be scheduled successfully'
END AS RESULT;
```

RESULT
Appointment cannot be scheduled

```
sqlite>
```

Dataset Insights

- The dataset contains 8 medical appointment records.
- The appointments belong to Cardiology, Neurology, Orthopedics, and General Medicine.
- The appointment statuses include Completed, Upcoming, Cancelled, and Missed.
- Waiting time can be calculated from check-in time and consultation start time.
- Cancelled and missed appointments can be identified using SQL.
- Doctor-wise appointment counts help in queue and workload management.
- The daily report gives a real-time view of appointment status.

Result

Thus, the Hospital Appointment Scheduling and Queue Management system was successfully implemented using SQLand PL/SQL. The required table was created, appointment records were inserted, the five SQL queries were executed, three PL/SQL procedures were implemented, and exception handling was tested successfully.


TRUTH-TOIL-TRIUMPH
AUTONOMOUS

1.2 Book list database

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

INPUT:

```
CREATE TABLE BOOKS( id
   INTEGER PRIMARY KEY,
   name TEXT, rating INTEGER
);


INSERT INTO BOOKS(id,name,rating)
VALUES
   (1, 'C Programming language', 10),
   (2, 'Java Programming language',9),
   (3, 'Python programming language',7);
```

OUTPUT:



1. Box office hits database

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

INPUT:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year
INTEGER);
```

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

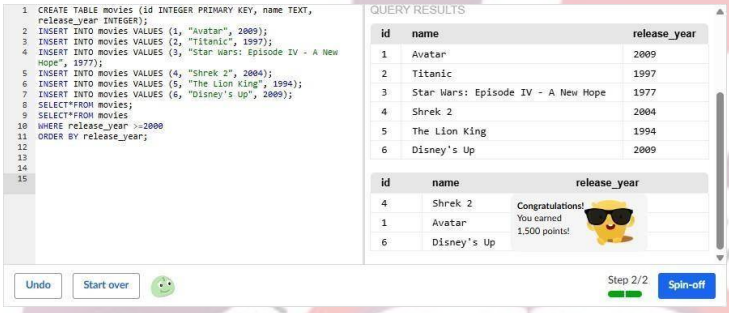
SELECT*FROM movies;

SELECT*FROM movies

WHERE release_year >=2000

ORDER BY release_year;

OUTPUT:



2. TODO list database stats

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

INPUT:

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

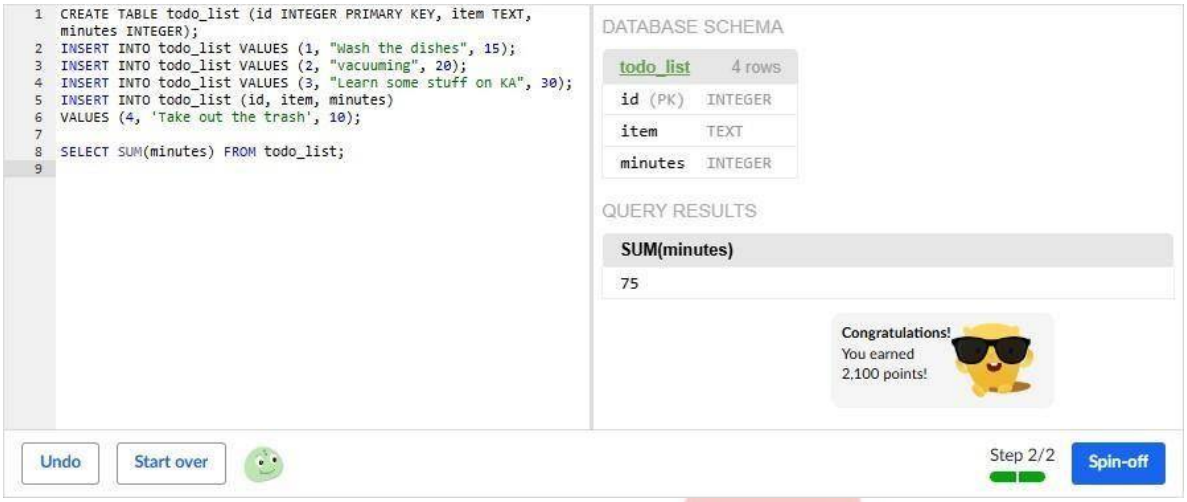
INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list (id, item, minutes)

VALUES (4, 'Take out the trash', 10);

SELECT SUM(minutes) FROM todo_list;

OUTPUT:



Project: Design a store database

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

INPUT:

```
CREATE TABLE gadget_store (
    id INTEGER PRIMARY KEY,
    product_name TEXT,
    category  TEXT,
    brand      TEXT,
    price      REAL,
    stock
    INTEGER
);
```

```
INSERT INTO gadget_store VALUES
(1, 'Wireless Mouse', 'Accessories', 'LogiTech', 25.99, 50),
(2, 'Mechanical Keyboard', 'Accessories', 'KeyPro', 79.99, 30),
(3, 'Gaming Headset', 'Audio', 'SoundMax', 59.99, 25),
```

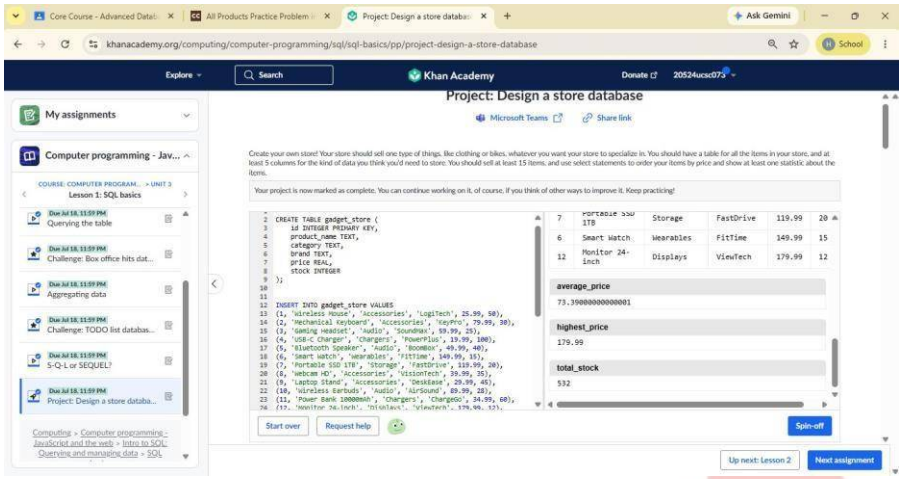
```
(4, 'USB-C Charger', 'Chargers', 'PowerPlus', 19.99, 100),
(5, 'Bluetooth Speaker', 'Audio', 'BoomBox', 49.99, 40),
(6, 'Smart Watch', 'Wearables', 'FitTime', 149.99, 15),
(7, 'Portable SSD 1TB', 'Storage', 'FastDrive', 119.99, 20),
(8, 'Webcam HD', 'Accessories', 'VisionTech', 39.99, 35),
(9, 'Laptop Stand', 'Accessories', 'DeskEase', 29.99, 45),
(10, 'Wireless Earbuds', 'Audio', 'AirSound', 89.99, 28),
(11, 'Power Bank 10000mAh', 'Chargers', 'ChargeGo', 34.99, 60),
(12, 'Monitor 24-inch', 'Displays', 'ViewTech', 179.99, 12),
(13, 'Graphics Tablet', 'Accessories', 'DrawPro', 99.99, 18),
(14, 'External HDD 2TB', 'Storage', 'SafeStore', 74.99, 22),
(15, 'Laptop Backpack', 'Bags', 'TravelPack', 44.99, 32);


SELECT *
FROM gadget_store
ORDER BY price ASC;
SELECT AVG(price) AS average_price
FROM gadget_store;


SELECT MAX(price) AS highest_price
FROM gadget_store;


SELECT SUM(stock) AS total_stock
FROM gadget_store;
```

OUTPUT:



1. Karaoke song selector

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', **and** released after 1990, **and** less than 4 minutes long.

INPUT:

CREATE TABLE songs (id INTEGER PRIMARY KEY, title TEXT, artist TEXT, mood TEXT, duration INTEGER, released INTEGER);

INSERT INTO songs (title, artist, mood, duration, released) VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969); INSERT INTO songs (title, artist, mood, duration, released) VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

SELECT title

FROM songs;

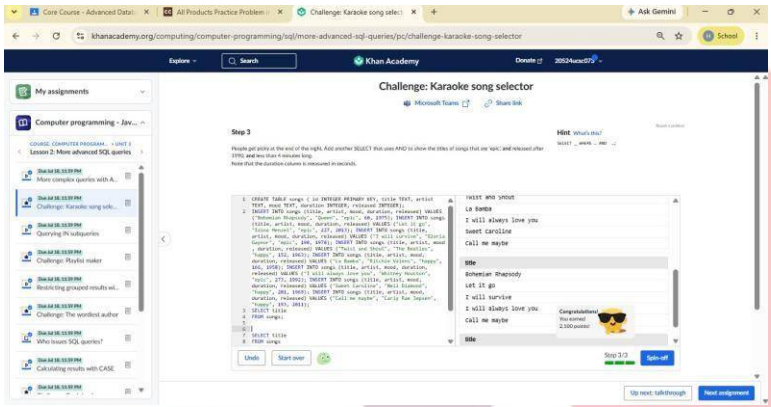
SELECT title FROM songs

WHERE mood = "epic" OR released > 1990;

SELECT title

```
FROM songs
WHERE mood = "epic" AND
  released > 1990
  AND duration < 240;
```

OUTPUT:



1. Playlist maker

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

INPUT:

```
CREATE TABLE artists ( id INTEGER PRIMARY
    KEY AUTOINCREMENT, name TEXT, country
    TEXT, genre TEXT);


INSERT INTO artists (name, country, genre)
    VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop");
```

```
INSERT INTO artists (name, country, genre)
   VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre) VALUES
  ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Bob Marley", "Jamaica", "Reggae");

CREATE TABLE songs ( id INTEGER PRIMARY
   KEY AUTOINCREMENT,
   artist TEXT,
   title TEXT);

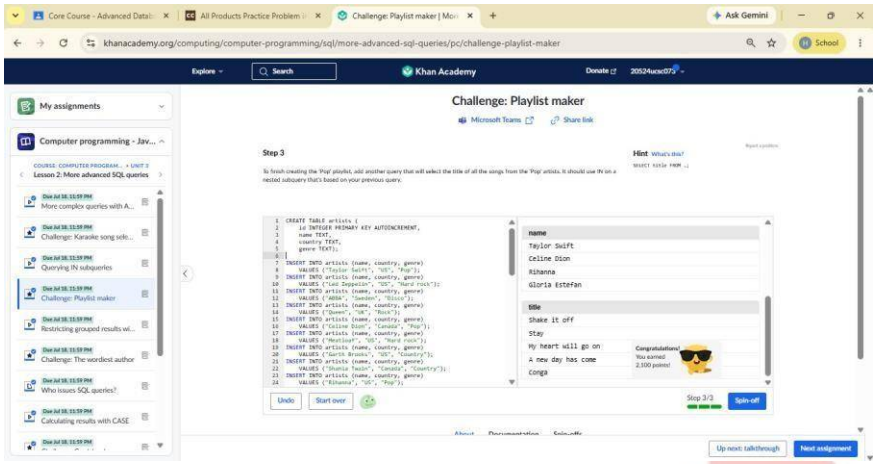
INSERT INTO songs (artist, title)
   VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
   VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "My heart will go on"); INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
   VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
   VALUES ("Gloria Estefan", "Conga");
```

```
INSERT INTO songs (artist, title)
   VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
   VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
   VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
   VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
   VALUES ("Guns N' Roses", "Don't cry");
SELECT title
FROM songs
WHERE artist = 'Queen';
SELECT name
FROM artists
WHERE genre = 'Pop';
SELECT title
FROM songs
WHERE artist IN (
   SELECT name
   FROM artists

   WHERE genre = 'Pop'

);
```

OUTPUT:



2. The wordiest author

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

INPUT:

```
CREATE TABLE books ( id INTEGER PRIMARY

    KEY AUTOINCREMENT,

    author
    TEXT, title
    TEXT,
    words INTEGER);



INSERT INTO books (author, title, words)

    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

```
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
   SELECT author, SUM(words) AS total_words
```

FROM books

GROUP BY author

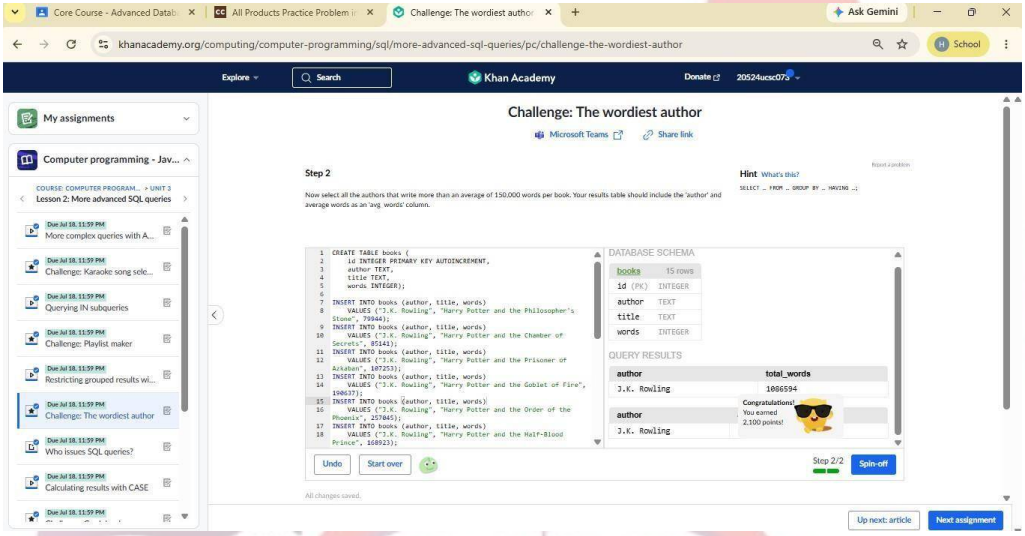
HAVING SUM(words) > 1000000;

SELECT author, AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000; **OUTPUT:**



3. Gradebook

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

INPUT:

CREATE TABLE student_grades (id INTEGER

PRIMARY KEY AUTOINCREMENT, name TEXT,

number_grade INTEGER, fraction_completed

REAL);

```
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);

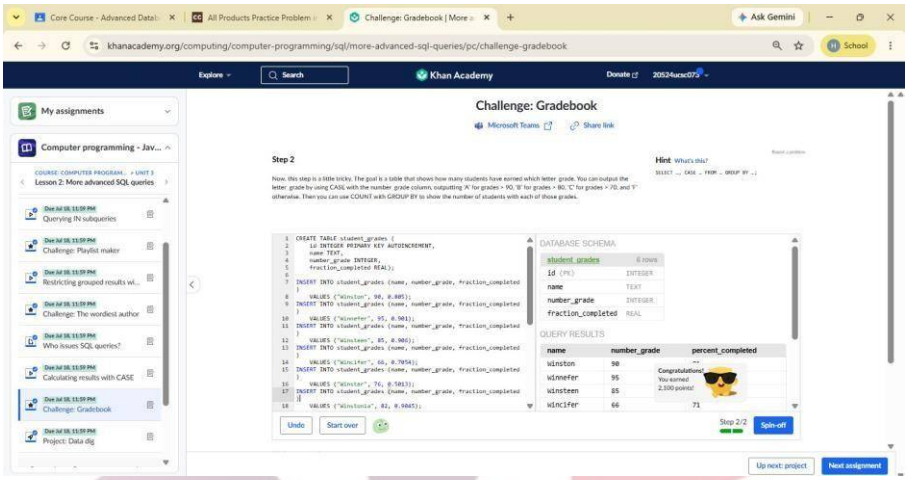
SELECT name, number_grade,
      ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;



SELECT COUNT(*),
   CASE
      WHEN number_grade > 90 THEN 'A'
      WHEN number_grade > 80 THEN 'B'
      WHEN number_grade > 70 THEN 'C'
      ELSE 'F'
   END AS "letter_grade"
FROM student_grades
GROUP BY
   CASE
      WHEN number_grade > 90 THEN 'A'
```

```
    WHEN number_grade > 80 THEN 'B' WHEN
    number_grade > 70 THEN 'C'
    ELSE 'F'
  END;
```

OUTPUT:



Project: Data dig

Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

- What are average, max, and min values in the data?
- What about those numbers per category in the data (using HAVING)?
- What ways are there to group the data values that don't exist yet (using CASE)?
- What interesting ways are there to filter the data (using AND/OR)?

INPUT:

```
CREATE TABLE movies ( id
    INTEGER PRIMARY KEY,
    title TEXT, genre
    TEXT, rating
    REAL, views
    INTEGER, year
    INTEGER
);
```

```
INSERT INTO movies VALUES
(1, 'Galaxy Wars', 'Sci-Fi', 8.5, 500000, 2020),
(2, 'Ocean Mystery', 'Drama', 7.8, 250000, 2018),
(3, 'Funny Days', 'Comedy', 6.9, 400000, 2021),
(4, 'Space Journey', 'Sci-Fi', 9.1, 800000, 2022),
(5, 'Love Story', 'Romance', 8.0, 300000, 2019),
(6, 'Dark Night', 'Drama', 7.5, 450000, 2017),
(7, 'Crazy Friends', 'Comedy', 7.2, 600000, 2023),
(8, 'Future World', 'Sci-Fi', 8.8, 900000, 2024);
```

```
-- Find average, maximum, and minimum ratings
SELECT
    AVG(rating) AS average_rating, MAX(rating) AS highest_rating,
  MIN(rating) AS lowest_rating
FROM movies;
```

```
-- Find average views by genre SELECT genre,
   AVG(views) AS average_views FROM movies
GROUP BY genre;
```

```
-- Show genres with more than 500,000 average views SELECT genre,
   AVG(views) AS average_views FROM movies
GROUP BY genre
```

```
HAVING AVG(views) > 500000;



-- Create new categories using CASE
SELECT
    title,
    views,
    CASE
        WHEN views > 700000 THEN 'Popular'
        WHEN views > 300000 THEN 'Average'
        ELSE 'Less Popular'

    END AS popularity
FROM movies;



-- Find highly rated recent movies
SELECT title, rating, year
FROM movies WHERE
rating >= 8 AND year >
2020;



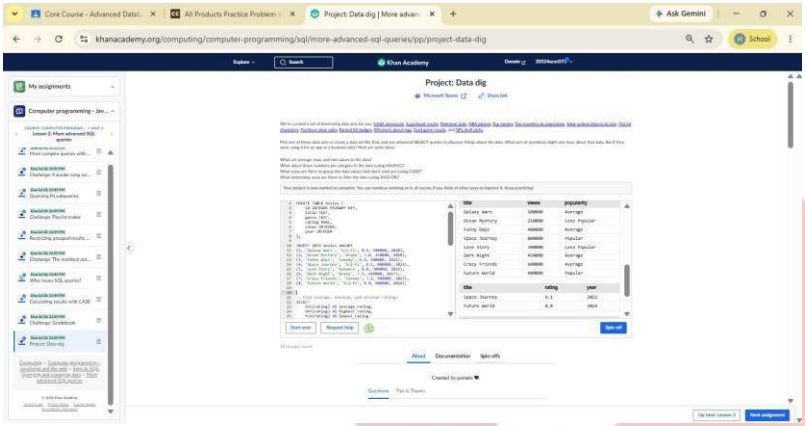
-- Find movies that are either Sci-Fi or have more than 600,000 views SELECT title,
genre, views
FROM movies
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

WHERE genre = 'Sci-Fi'

OR views > 600000;/* Put your data in here and query it! */

OUTPUT:



1. Bobby's Hobbies

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows **only** the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

INPUT:

```
CREATE TABLE persons ( id INTEGER PRIMARY
    KEY AUTOINCREMENT,
    name TEXT, age
    INTEGER);

INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace",
12); INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);

CREATE table hobbies ( id INTEGER PRIMARY
    KEY AUTOINCREMENT,
    person_id INTEGER, name
    TEXT);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing"); INSERT
INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
```

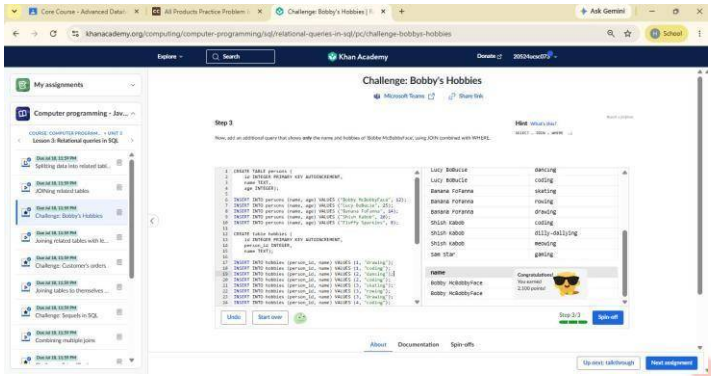
```
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);


INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");


SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;


SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

1. Customer's orders

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

INPUT:

```
CREATE TABLE customers ( id INTEGER
    PRIMARY KEY AUTOINCREMENT, name TEXT,
    email TEXT);

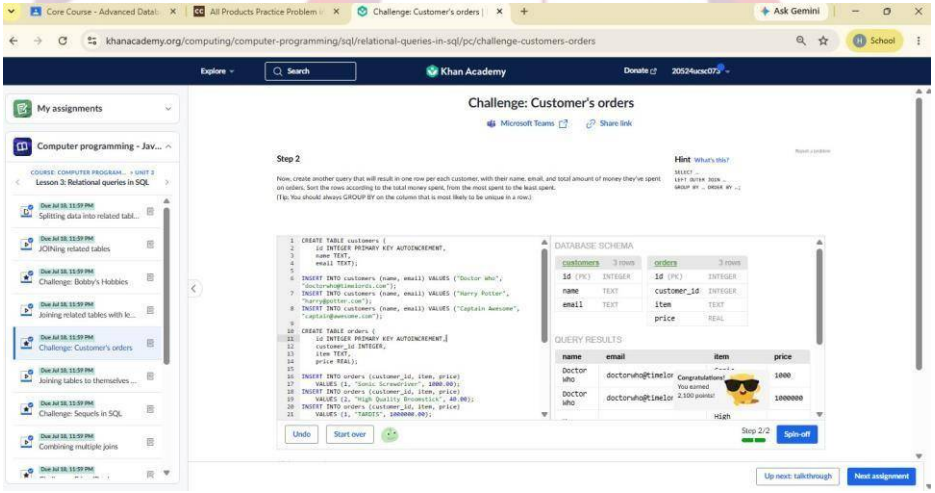

INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter",
"harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");

CREATE TABLE orders ( id INTEGER PRIMARY
    KEY AUTOINCREMENT, customer_id INTEGER,
    item TEXT,
    price REAL);


     INSERT INTO orders (customer_id, item, price)
      VALUES (1, "Sonic Screwdriver",
 1000.00); INSERT INTO orders (customer_id,
                        item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
```

```
SELECT customers.name,
    customers.email,
    orders.item,
    orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
    customers.email,
    SUM(orders.price) AS
    total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```

OUTPUT:



1. FriendBook

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

INPUT:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT, age
    INTEGER);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace",
"12"); INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER, name TEXT);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing"); INSERT
INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

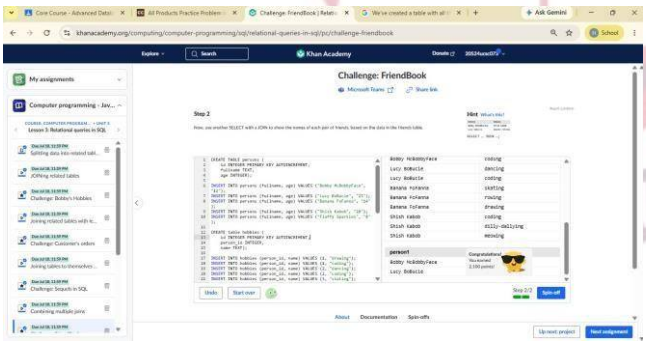
CREATE table friends ( id INTEGER PRIMARY
    KEY AUTOINCREMENT, person1_id INTEGER,
    person2_id INTEGER);
```

```
INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
    VALUES (2, 3);
```

```
SELECT persons.fullname, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
```

```
SELECT p1.fullname AS person1, p2.fullname
       AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
ON friends.person2_id = p2.id;
```

OUTPUT:



Project: Famous people

In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

- Movie stars: What movies are they in? Are they married to each other?
- Singers: What songs did they write? Where are they from?
- Authors: What books did they write?
- Fictional characters: How are they related to other characters? What books do they show up in?

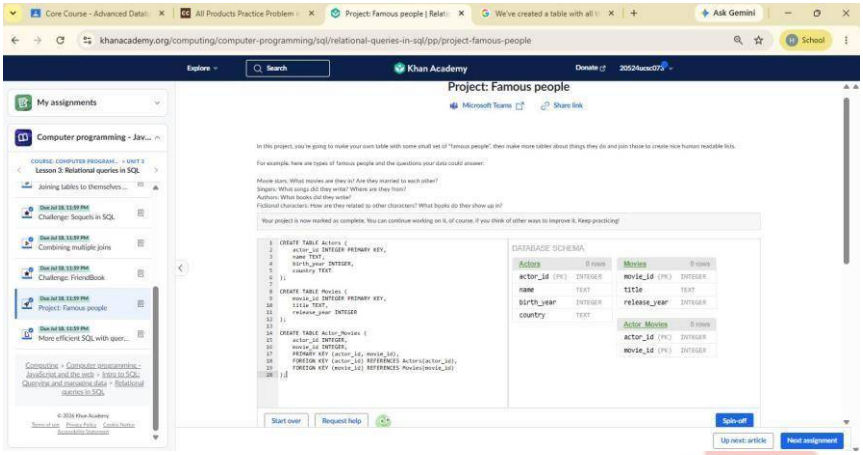
INPUT:

```
CREATE TABLE Actors (
    actor_id INTEGER PRIMARY
    KEY, name TEXT, birth_year
    INTEGER, country TEXT
);


CREATE TABLE Movies ( movie_id
    INTEGER PRIMARY KEY,
    title TEXT, release_year
    INTEGER
);


CREATE TABLE Actor_Movies (
    actor_id INTEGER, movie_id
    INTEGER,
    PRIMARY KEY (actor_id, movie_id),
    FOREIGN KEY (actor_id) REFERENCES Actors(actor_id),
    FOREIGN KEY (movie_id) REFERENCES Movies(movie_id)
);
```

OUTPUT:



1. Dynamic Documents

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

INPUT:

```
CREATE table documents ( id INTEGER PRIMARY
    KEY AUTOINCREMENT,
    title TEXT, content
    TEXT, author
    TEXT);


INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you
know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed
post.");
INSERT INTO documents (author, title, content)
```

VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");

SELECT * FROM documents;

UPDATE documents

SET author = 'Jackie Draper'

WHERE author = 'Jackie Paper';

SELECT *

FROM documents;

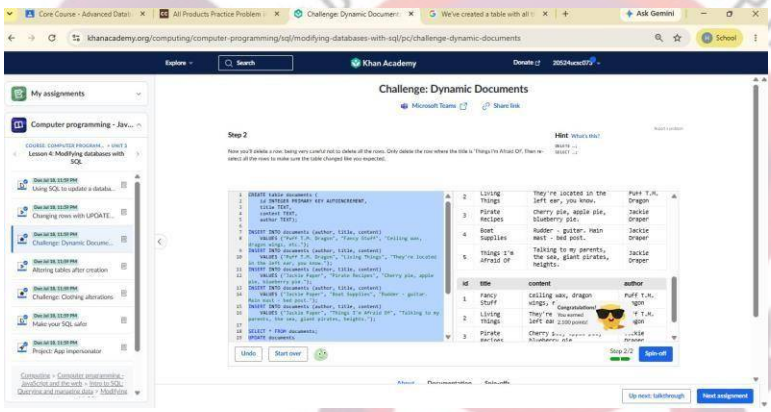
DELETE FROM documents

WHERE title = "Things I'm Afraid Of";

SELECT *

FROM documents;

OUTPUT:



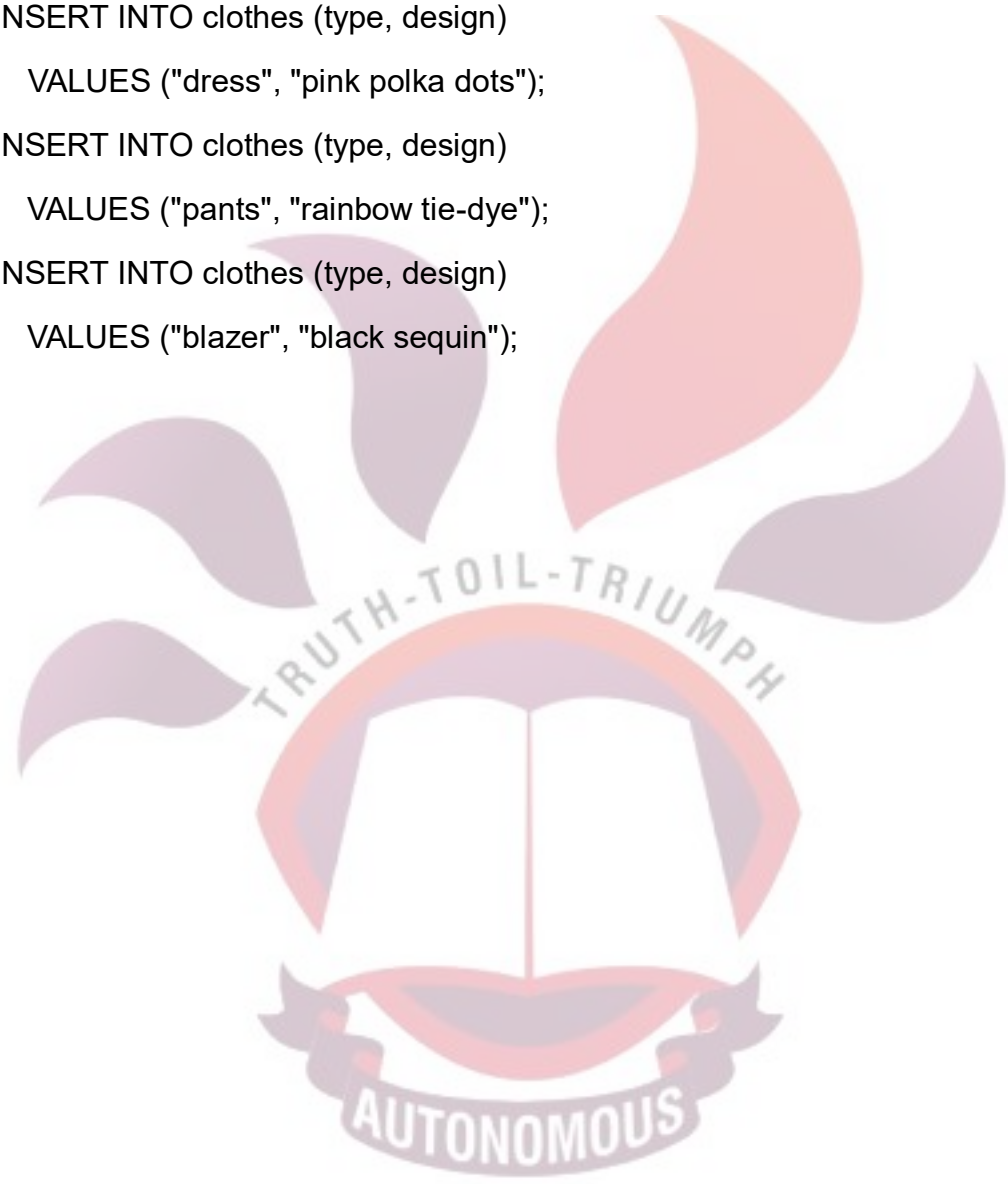
1. Clothing alterations

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

INPUT:

```
CREATE TABLE clothes ( id INTEGER PRIMARY
    KEY AUTOINCREMENT,
    type TEXT, design
    TEXT);

INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");
```



Revising the Select Query I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

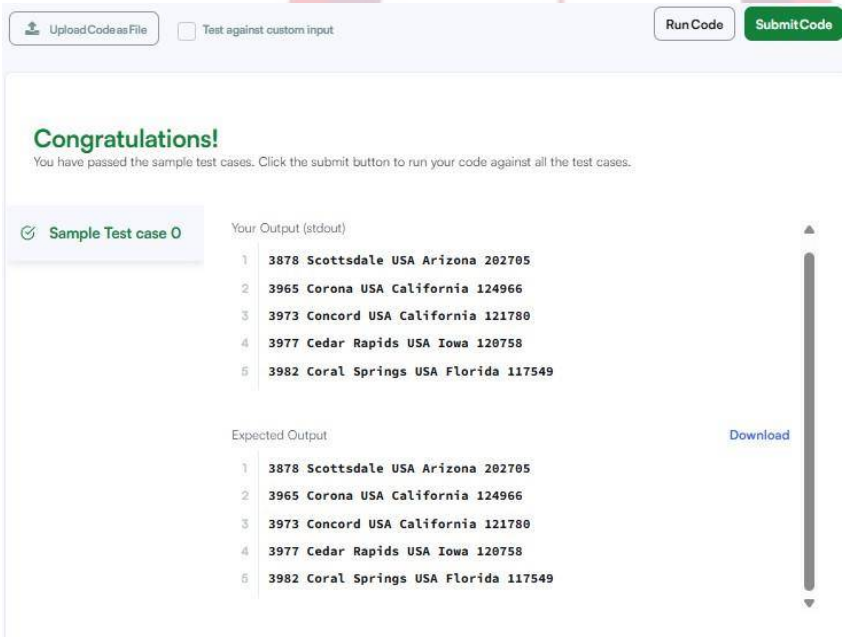
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE ='USA'
  AND POPULATION > 100000;
```

OUTPUT



Revising the Select Query II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`. The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

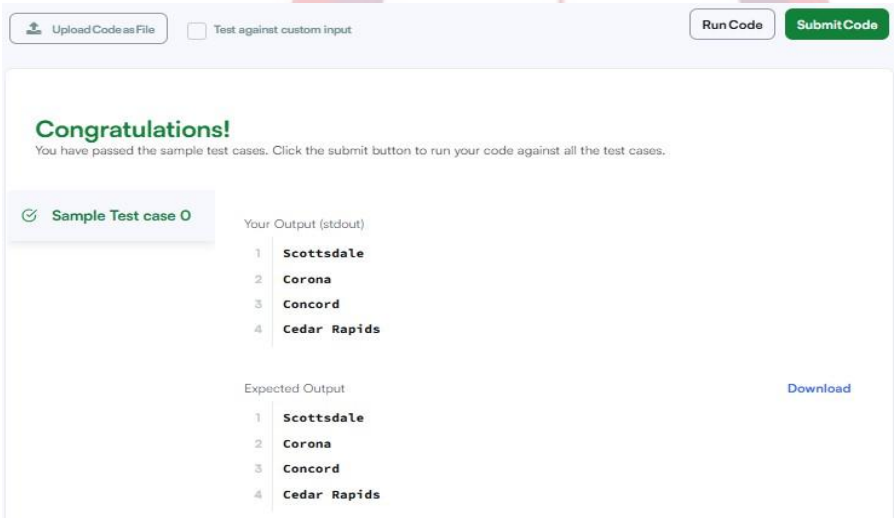
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'USA'
  AND POPULATION > 120000;
```

OUTPUT



Select All

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

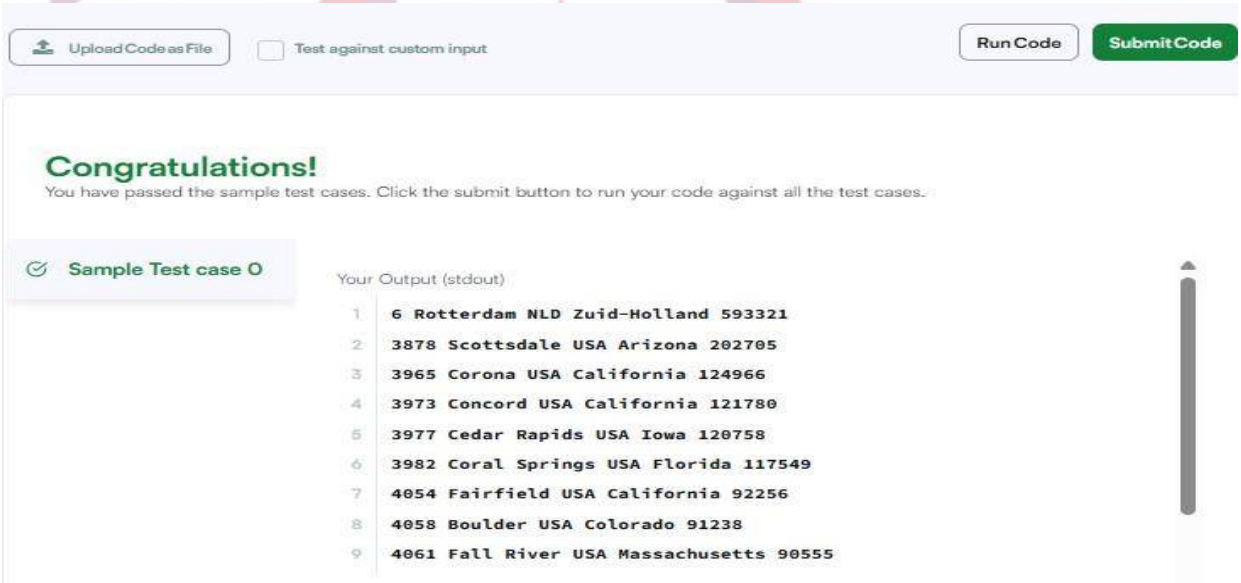
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY;
```

OUTPUT



Select By ID

Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:

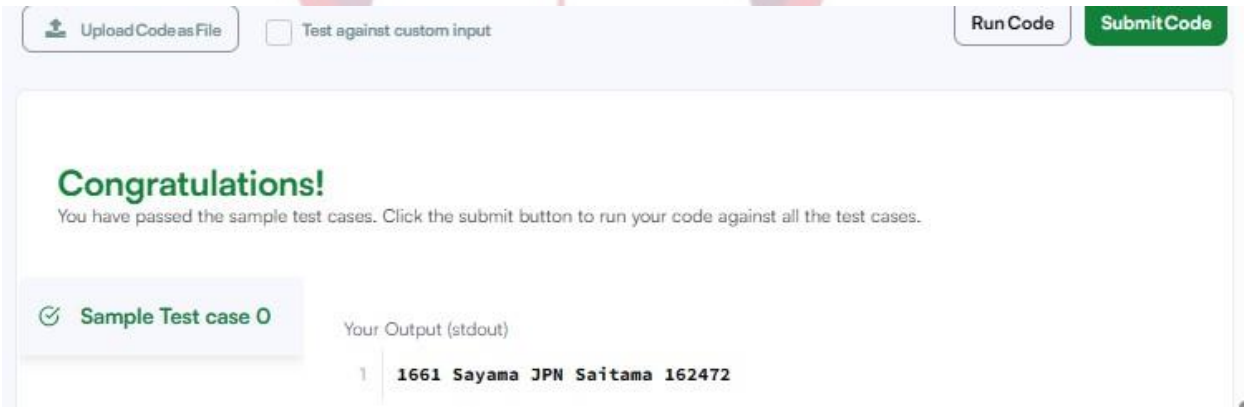
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT



Japanese Cities' Attributes

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



Japanese Cities' Names

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



Weather Observation Station 1

Query a list of **CITY** and **STATE** from the **STATION** table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

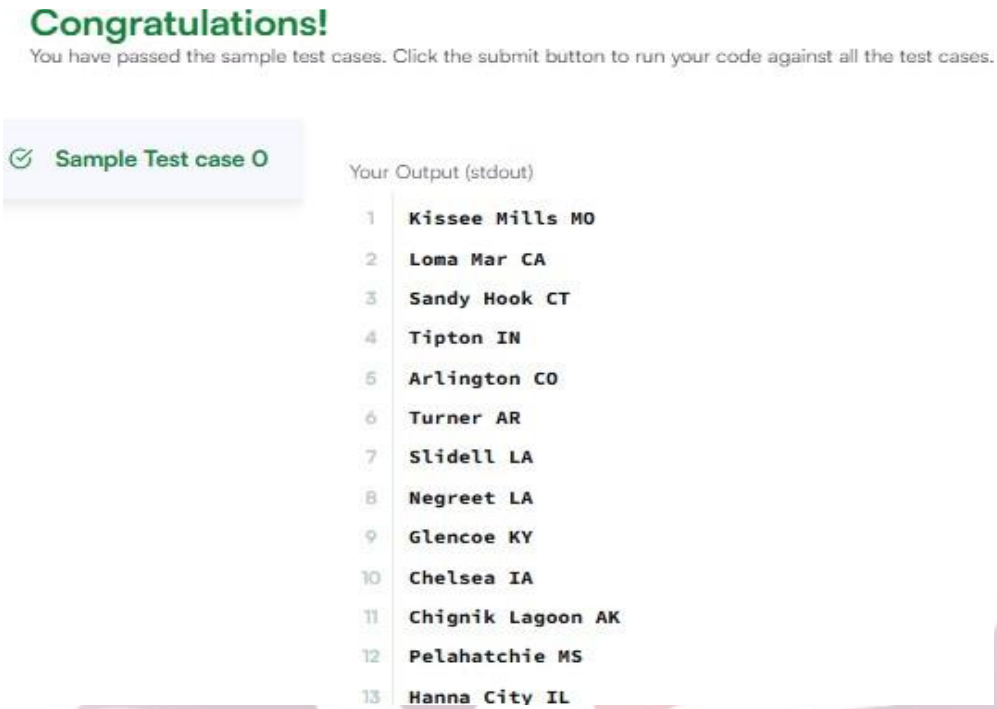
INPUT

SELECT CITY, STATE

FROM STATION;

OUTPUT

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



Weather Observation Station 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

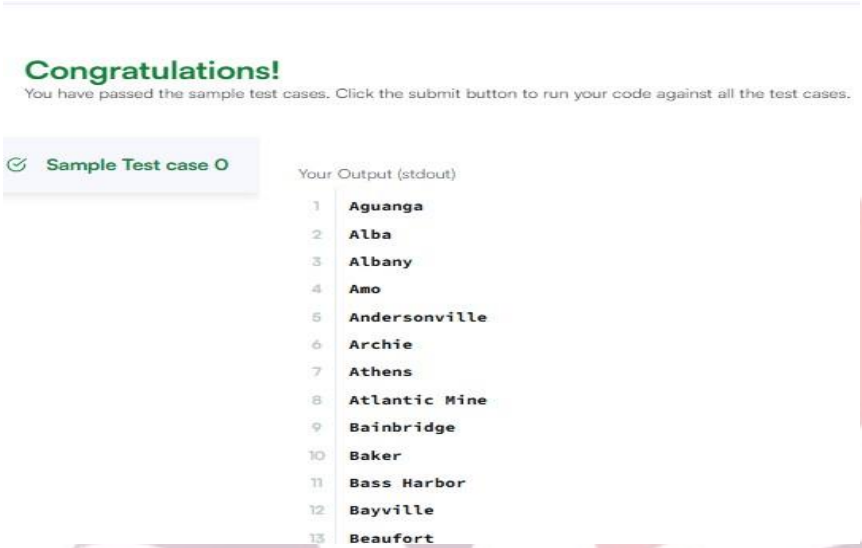
INPUT

SELECT DISTINCT CITY

FROM STATION

WHERE MOD(ID, 2) = 0;

OUTPUT



Weather Observation Station 4

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

SELECT COUNT(CITY) - COUNT(DISTINCT CITY)

FROM STATION;

OUTPUT



Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```sql
-- Query for the shortest city name
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) ASC, CITY ASC
LIMIT 1;

-- Query for the longest city name
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY ASC
LIMIT 1;
```

OUTPUT



Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

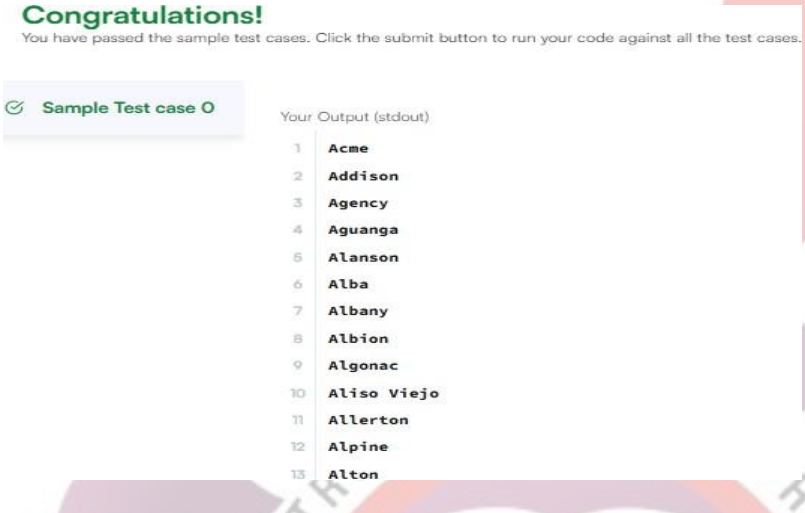
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

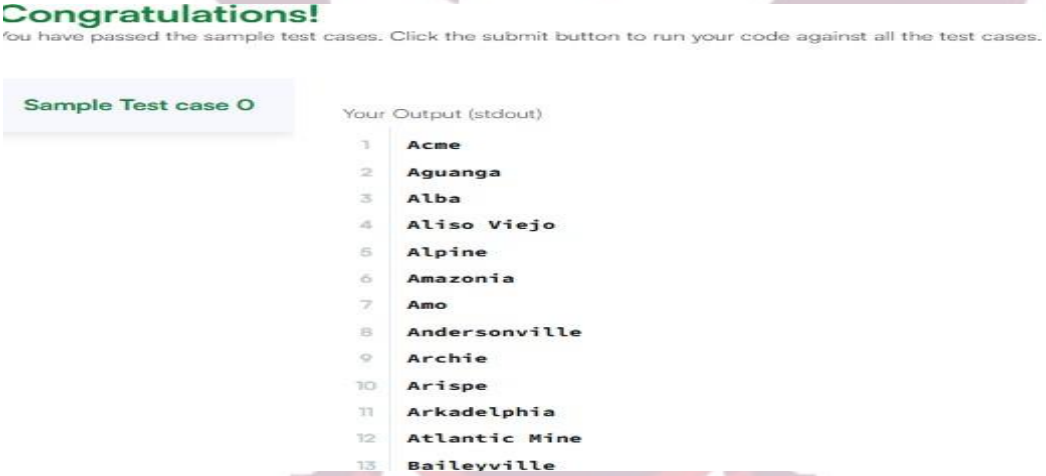
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

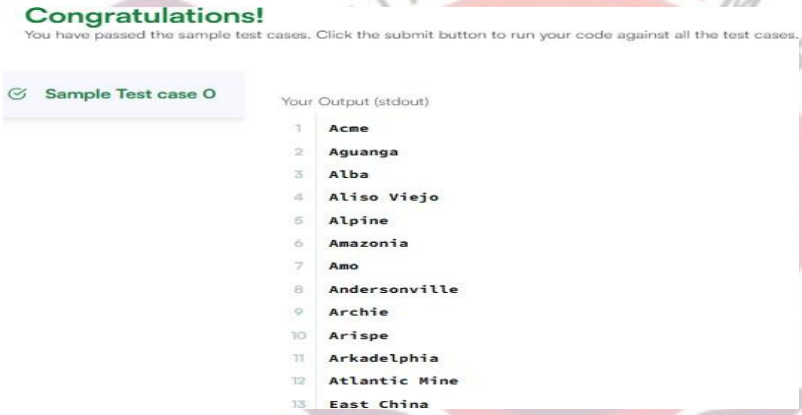
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u')
 AND LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

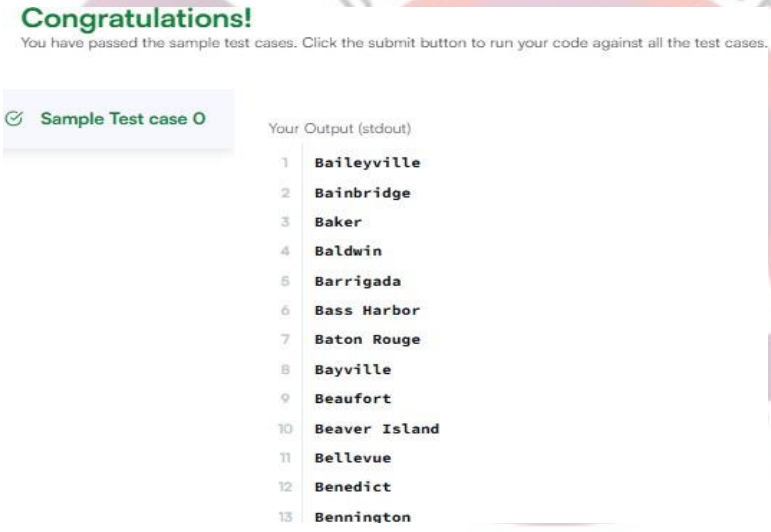
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

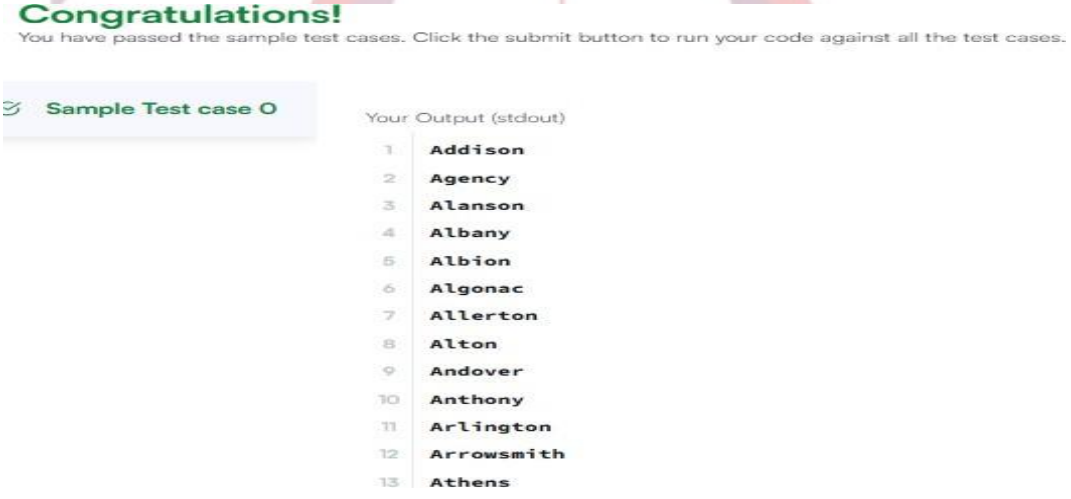
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(LEFT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
  OR LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Addison
Agency
Alanson
Albany
Albion
Algonac
Allerton
Alton
Andover
Anthony
Arlington
Arrowsmith
Athens
```

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(LEFT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
  AND LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Baker
Baldwin
Bass Harbor
Beaufort
Beaver Island
Benedict
Bennington
Berryton
Beverly
Bison
Blue River
Bowdon
Bowdon Junction
```

Higher Than 75 Marks

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

The **STUDENTS** table is described as follows:

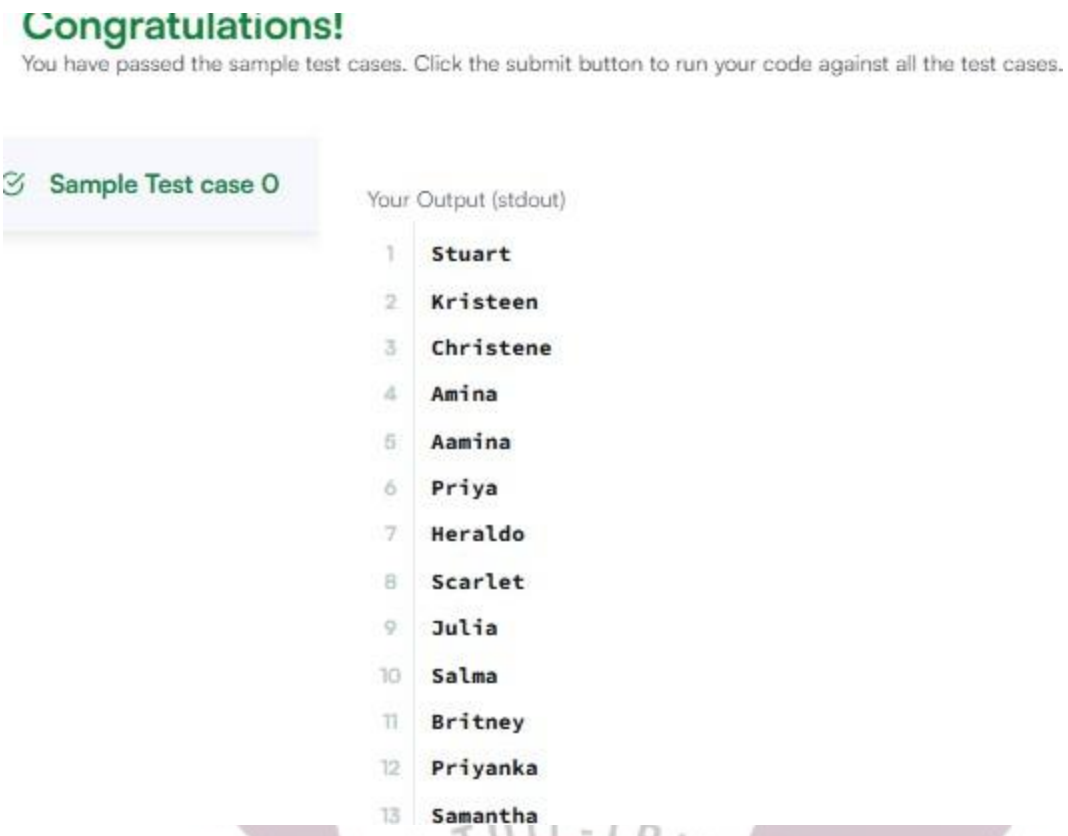
Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

INPUT

```
SELECT Name
FROM STUDENTS
WHERE Marks > 75
ORDER BY RIGHT(Name, 3) ASC, ID ASC;
```

OUTPUT

TRUTH-TOIL-TRIUMPH AUTONOMOUS



Print Prime Numbers

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

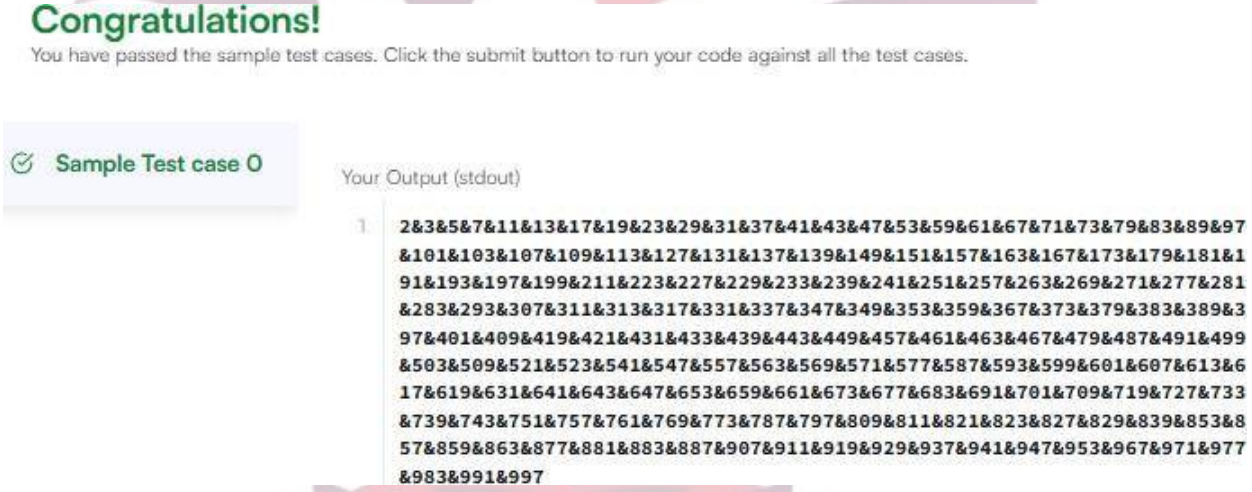
2&3&5&7

INPUT

```
WITH RECURSIVE Numbers AS (
  SELECT 2 AS num
  UNION ALL
  SELECT num + 1
```

```
  FROM Numbers
  WHERE num < 1000
)
SELECT GROUP_CONCAT(n1.num SEPARATOR '&')
FROM Numbers n1
WHERE NOT EXISTS (
  SELECT 1
  FROM Numbers n2
  WHERE n2.num < n1.num AND n1.num % n2.num = 0
);
```

OUTPUT



Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

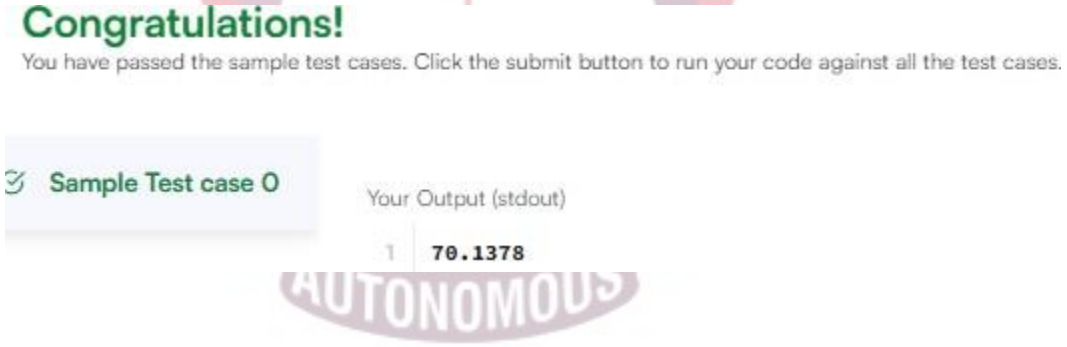
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N > 38.7780
ORDER BY LAT_N ASC
LIMIT 1;
```

OUTPUT



Weather Observation Station 18

Consider and to be two points on a *2D* plane.

- happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).

- happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- happens to equal the maximum value in *Northern Latitude* (*LAT_N* in **STATION**).
- happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

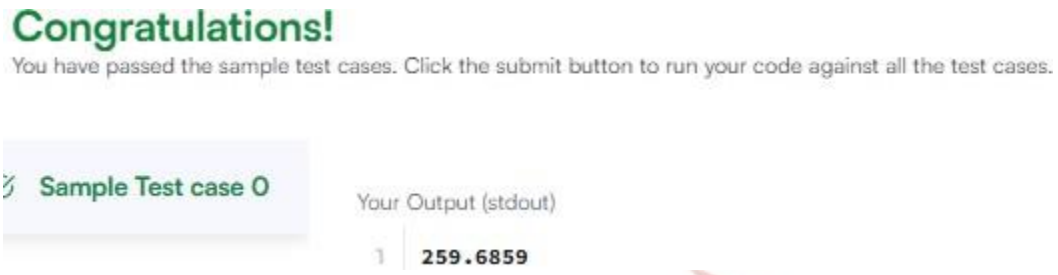
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(
  (MAX(LAT_N) - MIN(LAT_N)) + (MAX(LONG_W) - MIN(LONG_W)),
  4
)
FROM STATION;
```

OUTPUT



Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:

STATION

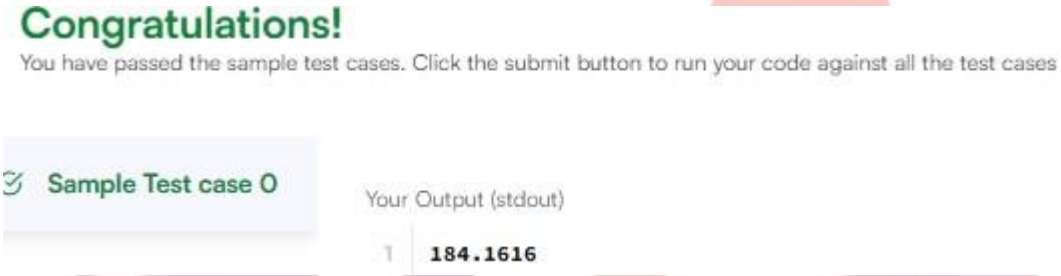
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(
  SQRT(
```

```
    POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
    POWER(MAX(LONG_W) - MIN(LONG_W), 2)
  ),
  4
)
FROM STATION;
```

OUTPUT



Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

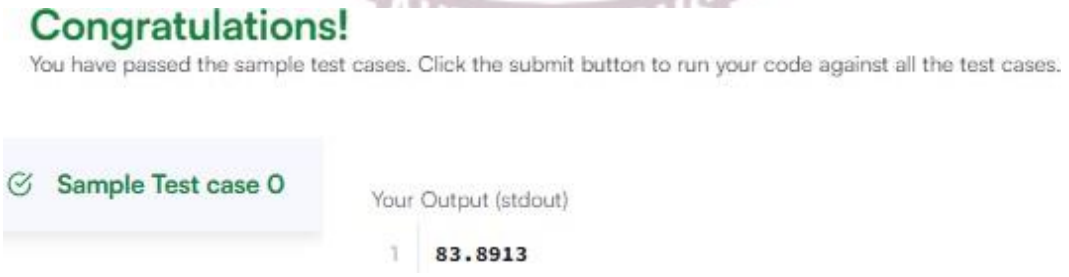
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
WITH Ranked AS (
  SELECT LAT_N,
      ROW_NUMBER() OVER (ORDER BY LAT_N) AS row_num,
      COUNT(*) OVER () AS total_count
  FROM STATION
)
SELECT ROUND(AVG(LAT_N), 4)
FROM Ranked
WHERE row_num IN (FLOOR((total_count + 1) / 2.0), CEIL((total_count + 1) / 2.0));
```

OUTPUT



Placements

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

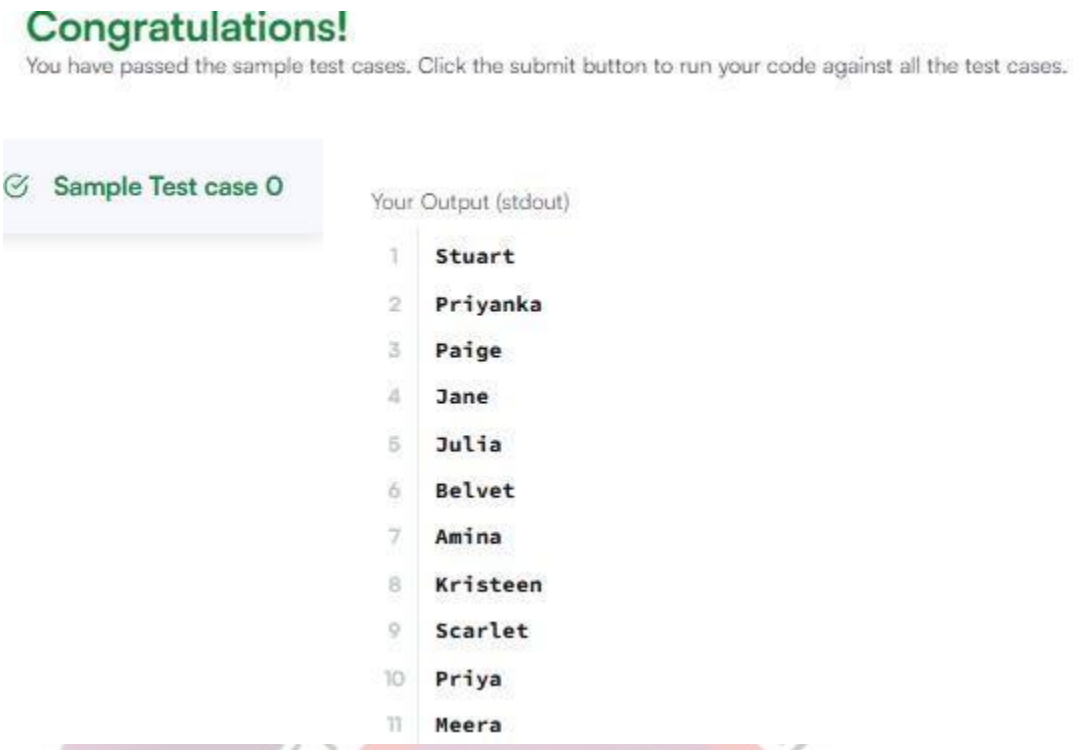
INPUT

```
SELECT S.Name
FROM Students S
JOIN Friends F ON S.ID = F.ID
JOIN Packages P1 ON S.ID = P1.ID
JOIN Packages P2 ON F.Friend_ID = P2.ID
```

WHERE P2.Salary > P1.Salary

ORDER BY P2.Salary ASC;

OUTPUT



Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

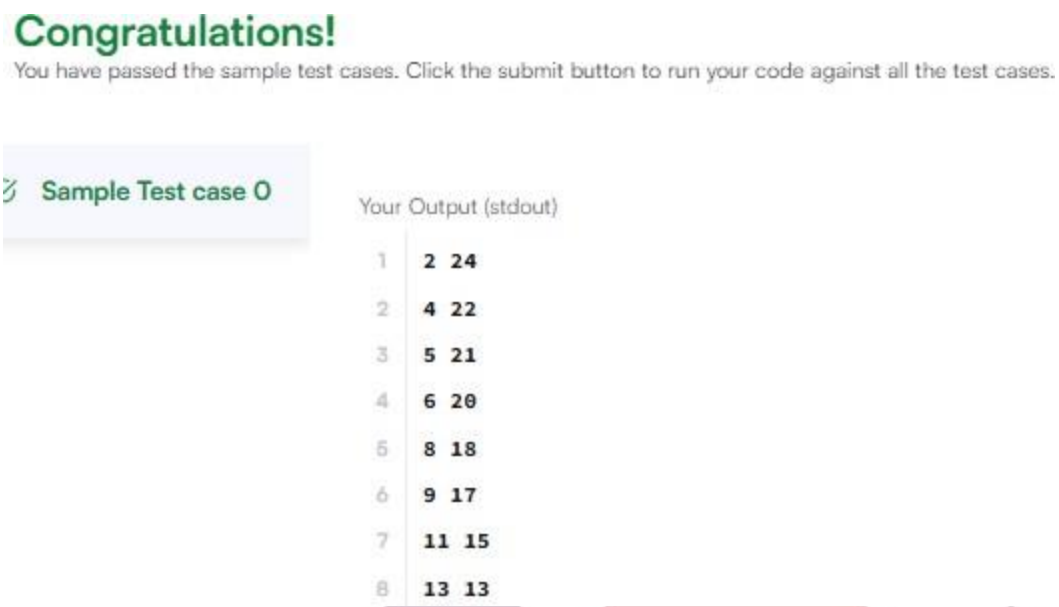
Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

INPUT

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y OR (f1.X = f1.Y AND COUNT(*) > 1)
ORDER BY f1.X ASC;
```

OUTPUT



Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

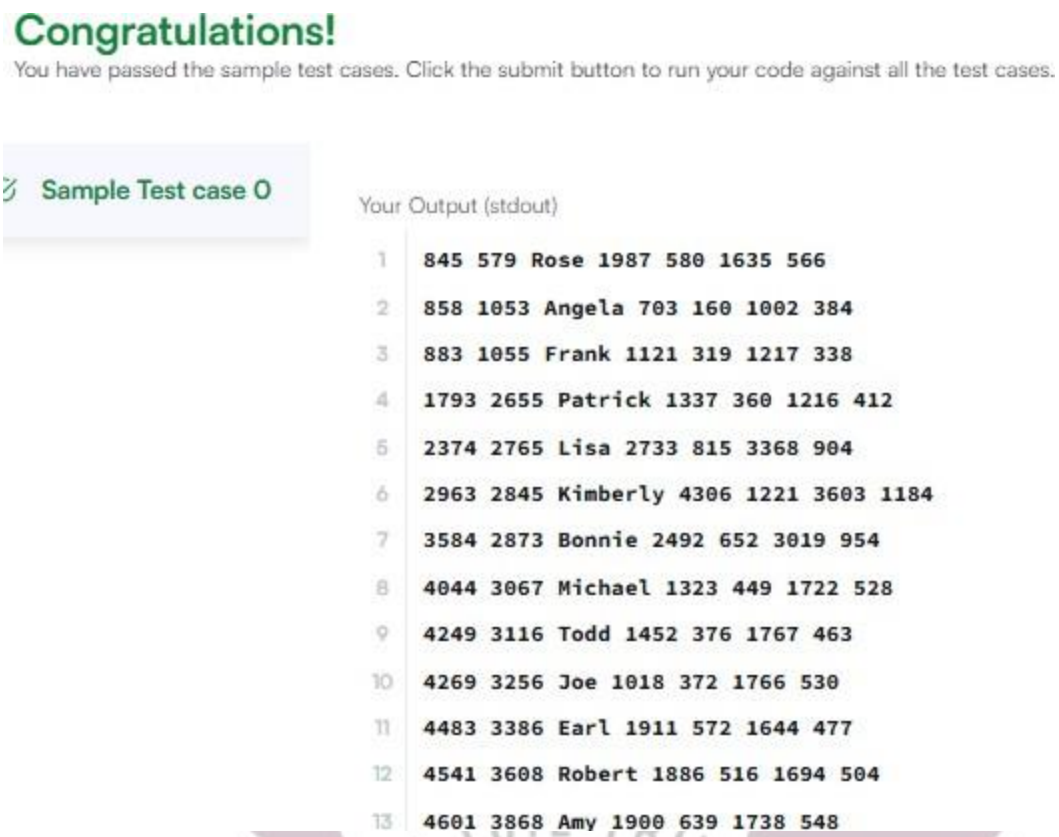
Column	Type
contest_id	Integer
hacker_id	Integer
name	String

INPUT

```
WITH Agg_Submissions AS (
  SELECT
    challenge_id,
    SUM(total_submissions) AS total_submissions,
    SUM(total_accepted_submissions) AS total_accepted_submissions
  FROM Submission_Stats
  GROUP BY challenge_id
),
Agg_Views AS (
  SELECT
    challenge_id,
    SUM(total_views) AS total_views,
    SUM(total_unique_views) AS total_unique_views
  FROM View_Stats
  GROUP BY challenge_id
)
SELECT
  con.contest_id,
```

```
  con.hacker_id,
  con.name,
  SUM(COALESCE(s.total_submissions, 0)) AS sum_total_submissions,
  SUM(COALESCE(s.total_accepted_submissions, 0)) AS
sum_total_accepted_submissions,
  SUM(COALESCE(v.total_views, 0)) AS sum_total_views,
  SUM(COALESCE(v.total_unique_views, 0)) AS sum_total_unique_views
FROM Contests con
JOIN Colleges col ON con.contest_id = col.contest_id
JOIN Challenges cha ON col.college_id = cha.college_id
LEFT JOIN Agg_Submissions s ON cha.challenge_id = s.challenge_id
LEFT JOIN Agg_Views v ON cha.challenge_id = v.challenge_id
GROUP BY con.contest_id, con.hacker_id, con.name
HAVING (
  SUM(COALESCE(s.total_submissions, 0)) +
  SUM(COALESCE(s.total_accepted_submissions, 0)) +
  SUM(COALESCE(v.total_views, 0)) +
  SUM(COALESCE(v.total_unique_views, 0))
) > 0
ORDER BY con.contest_id ASC;
```

OUTPUT



15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

INPUT

```
SELECT
  s1.submission_date,

  -- 1. Count hackers who made at least one submission EVERY day up to s1.submission_date
  (
    SELECT COUNT(DISTINCT hacker_id)
    FROM Submissions s2
    WHERE s2.submission_date = s1.submission_date
     AND (
       SELECT COUNT(DISTINCT s3.submission_date)
       FROM Submissions s3
       WHERE s3.hacker_id = s2.hacker_id
        AND s3.submission_date <= s1.submission_date
     ) = DATEDIFF(s1.submission_date, '2016-03-01') + 1
  ) AS consistent_hackers,

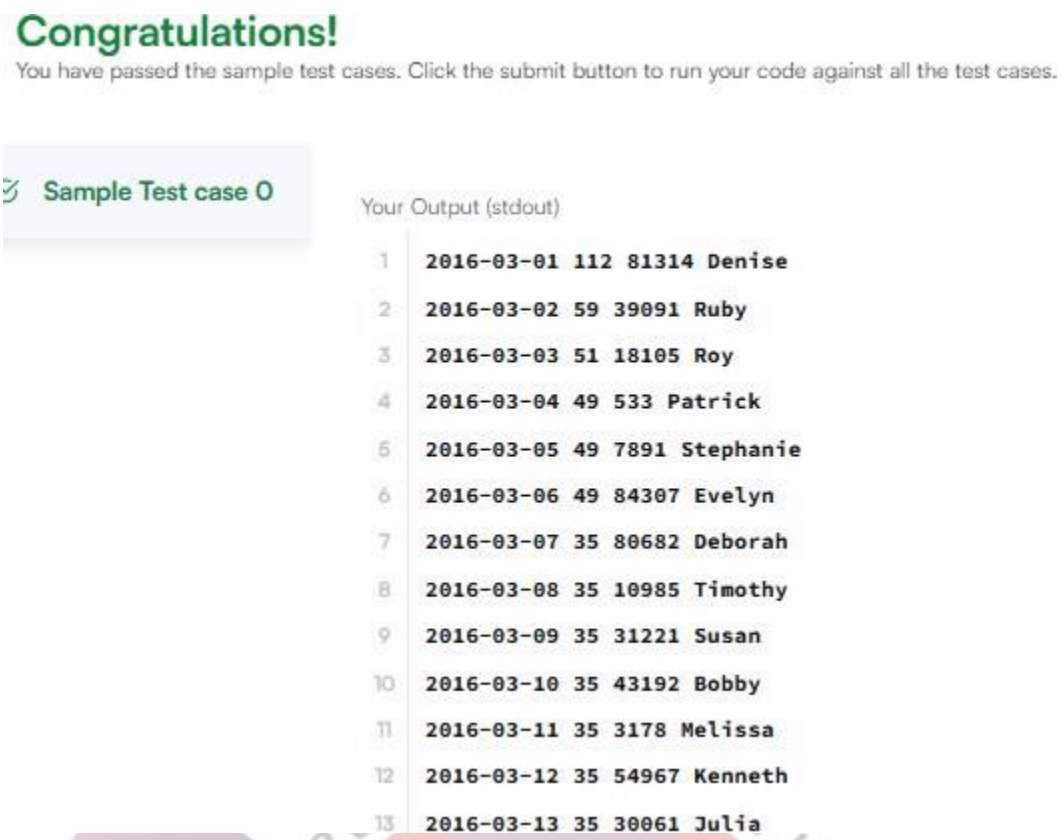
  -- 2. Find top hacker ID with max submissions on s1.submission_date (tie-breaker: lowest hacker_id)
  (
    SELECT hacker_id
```

```
    FROM Submissions s3
    WHERE s3.submission_date = s1.submission_date
    GROUP BY hacker_id
    ORDER BY COUNT(submission_id) DESC, hacker_id ASC
    LIMIT 1
  ) AS top_hacker_id,

  -- 3. Get name of top hacker
  (
    SELECT name
    FROM Hackers
    WHERE hacker_id = (
      SELECT hacker_id
      FROM Submissions s4
      WHERE s4.submission_date = s1.submission_date
      GROUP BY hacker_id
      ORDER BY COUNT(submission_id) DESC, hacker_id ASC
      LIMIT 1
    )
  ) AS top_hacker_name

FROM (
  SELECT DISTINCT submission_date
  FROM Submissions
) s1
ORDER BY s1.submission_date ASC;
```

OUTPUT



African Cities

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

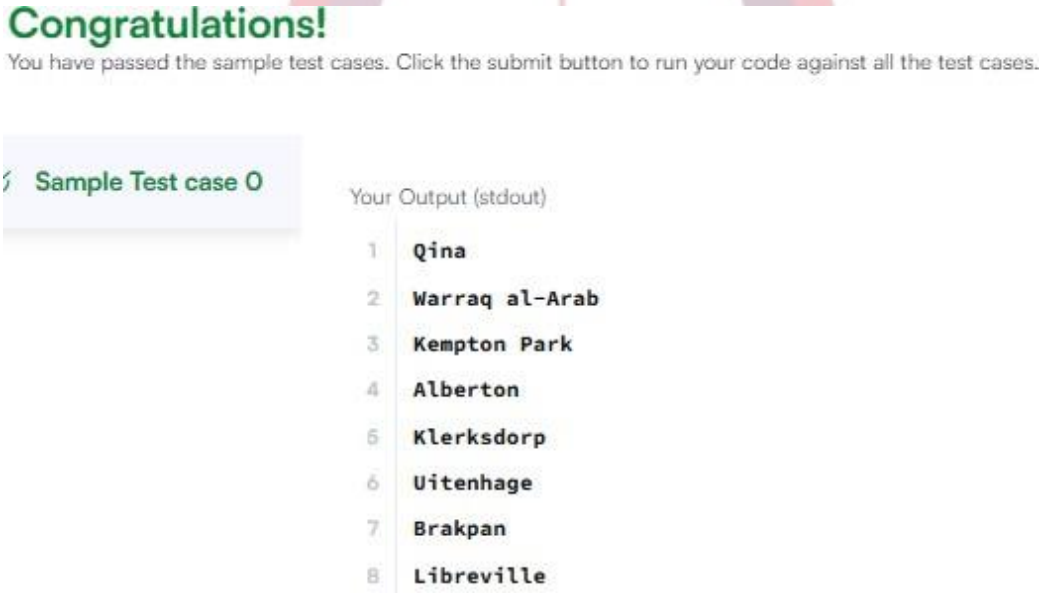
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Africa';
```

OUTPUT



Average Population of Each Continent

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

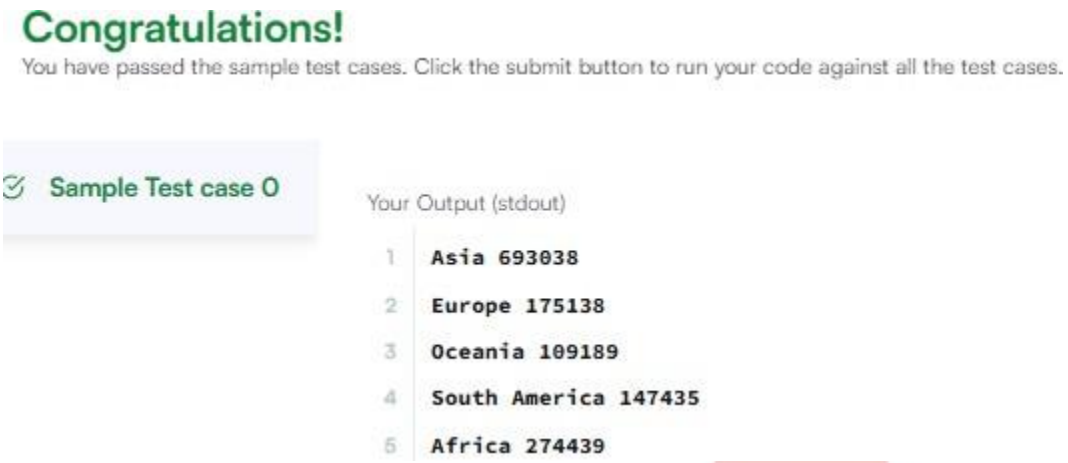
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

OUTPUT



The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

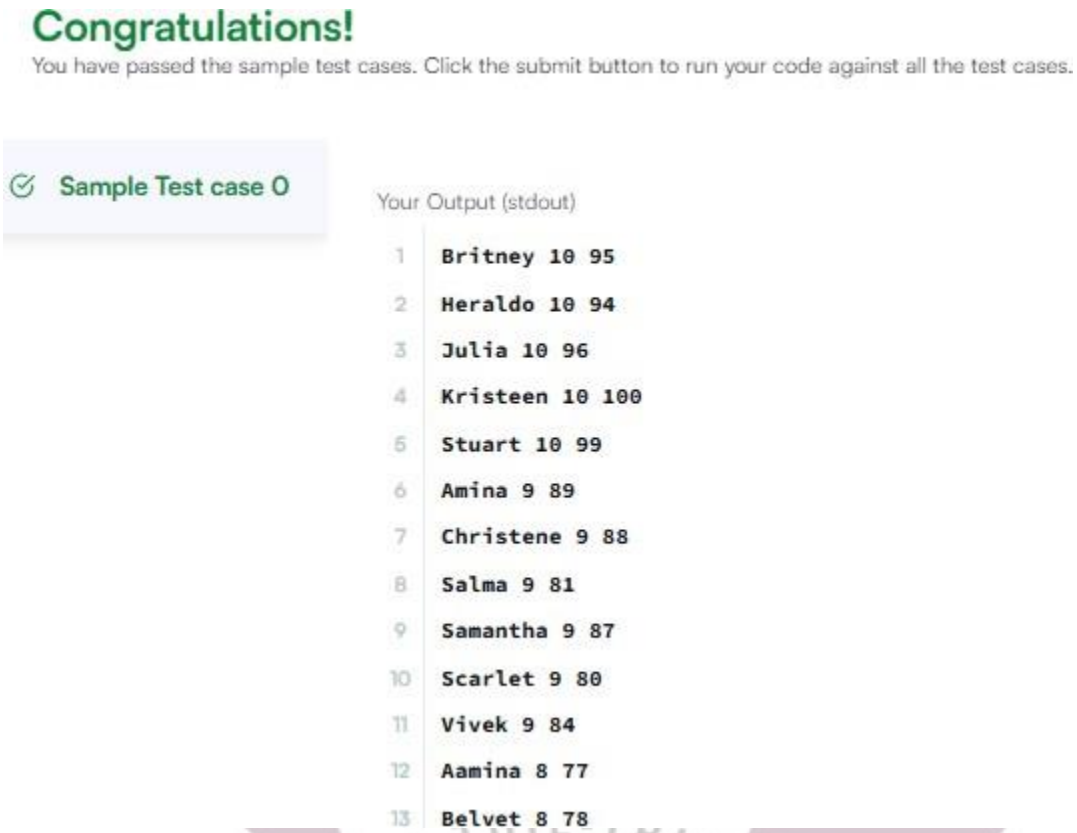
Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

INPUT

```
SELECT
   CASE
      WHEN g.Grade >= 8 THEN s.Name
      ELSE NULL
   END AS Name,
   g.Grade,
   s.Marks
FROM Students s
JOIN Grades g
 ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
   g.Grade DESC,
   s.Name ASC,
   s.Marks ASC;
```

OUTPUT



Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

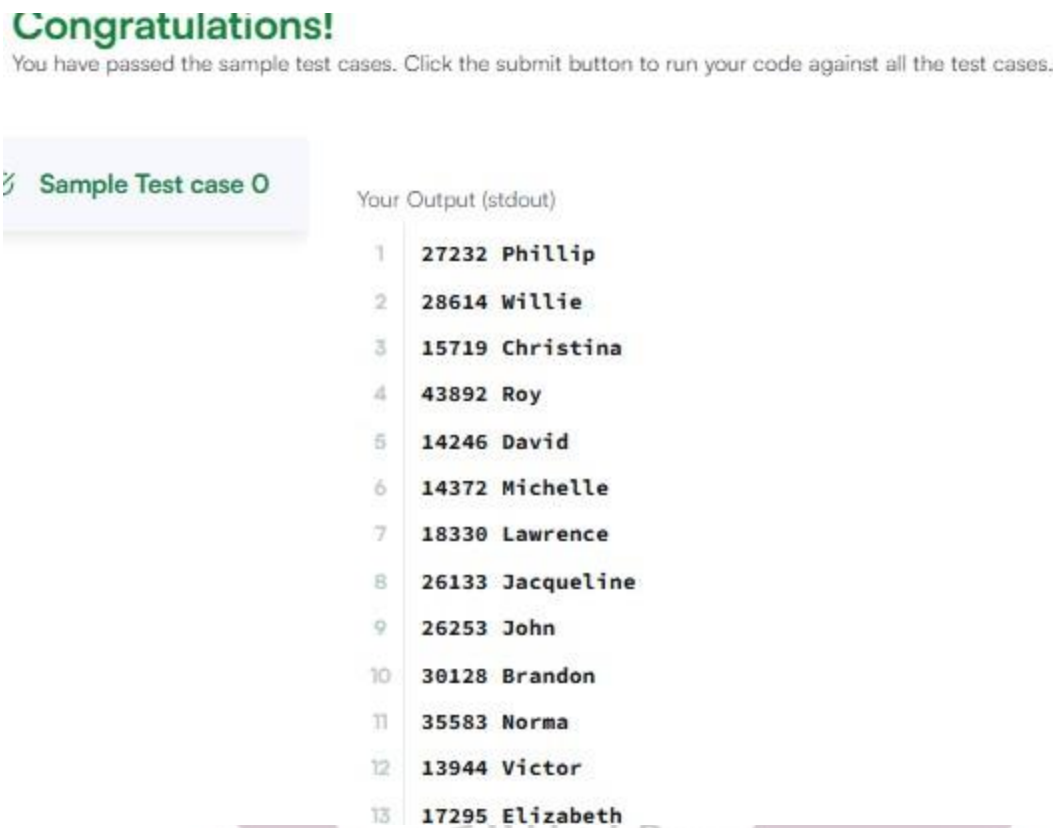
- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

INPUT

```
SELECT
  h.hacker_id,
  h.name
FROM Submissions s
JOIN Hackers h ON s.hacker_id = h.hacker_id
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

OUTPUT



Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the

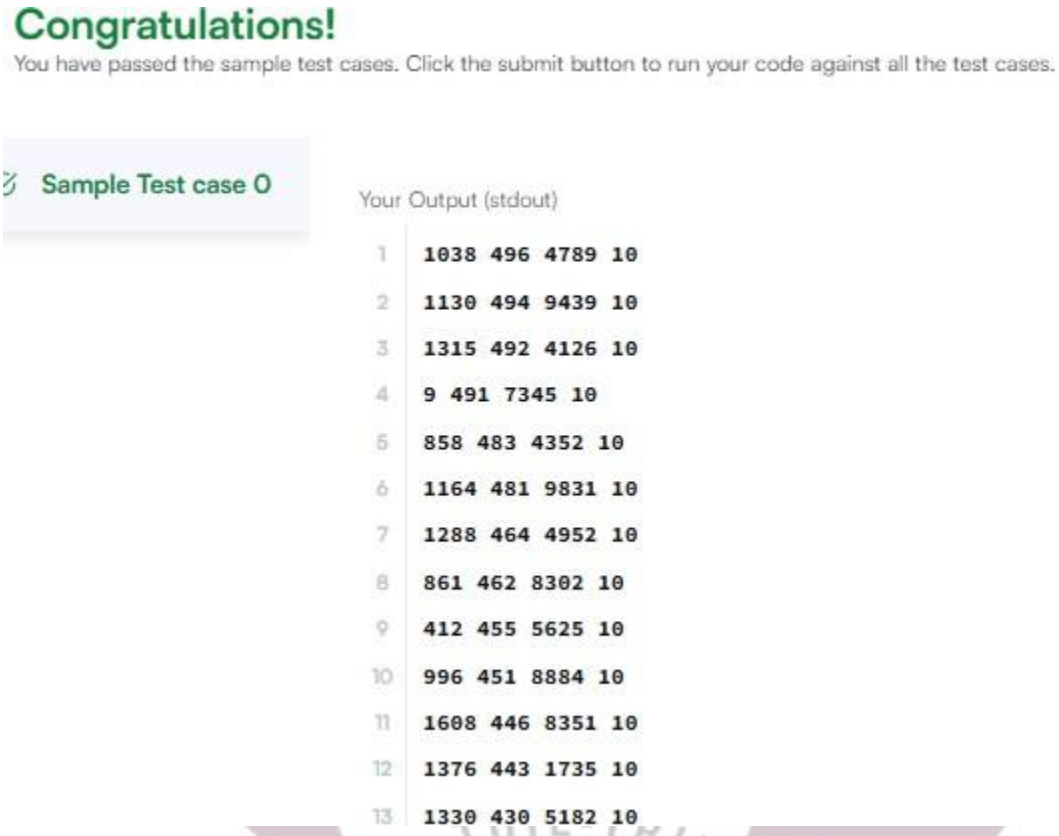
quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

INPUT

```
SELECT
  w.id,
  wp.age,
  w.coins_needed,
  w.power
FROM Wands w
JOIN Wands_Property wp ON w.code = wp.code
WHERE wp.is_evil = 0
 AND w.coins_needed = (
    SELECT MIN(w1.coins_needed)
    FROM Wands w1
    JOIN Wands_Property wp1 ON w1.code = wp1.code
    WHERE wp1.is_evil = 0
     AND w1.power = w.power
     AND wp1.age = wp.age
 )
ORDER BY w.power DESC, wp.age DESC;
```

OUTPUT



Type of Triangle

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

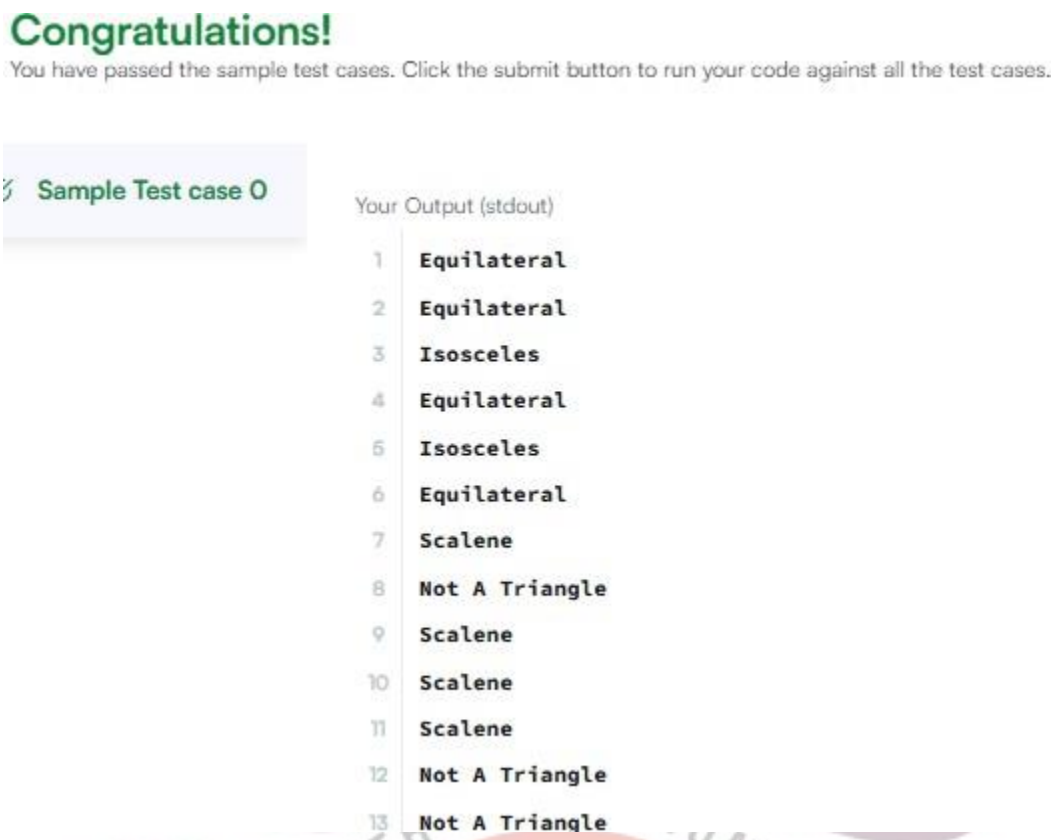
The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

INPUT

```
SELECT
  CASE
    WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
    WHEN A = B AND B = C THEN 'Equilateral'
    WHEN A = B OR B = C OR A = C THEN 'Isosceles'
    ELSE 'Scalene'
  END
FROM TRIANGLES;
```

OUTPUT



The PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For
   example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:
3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

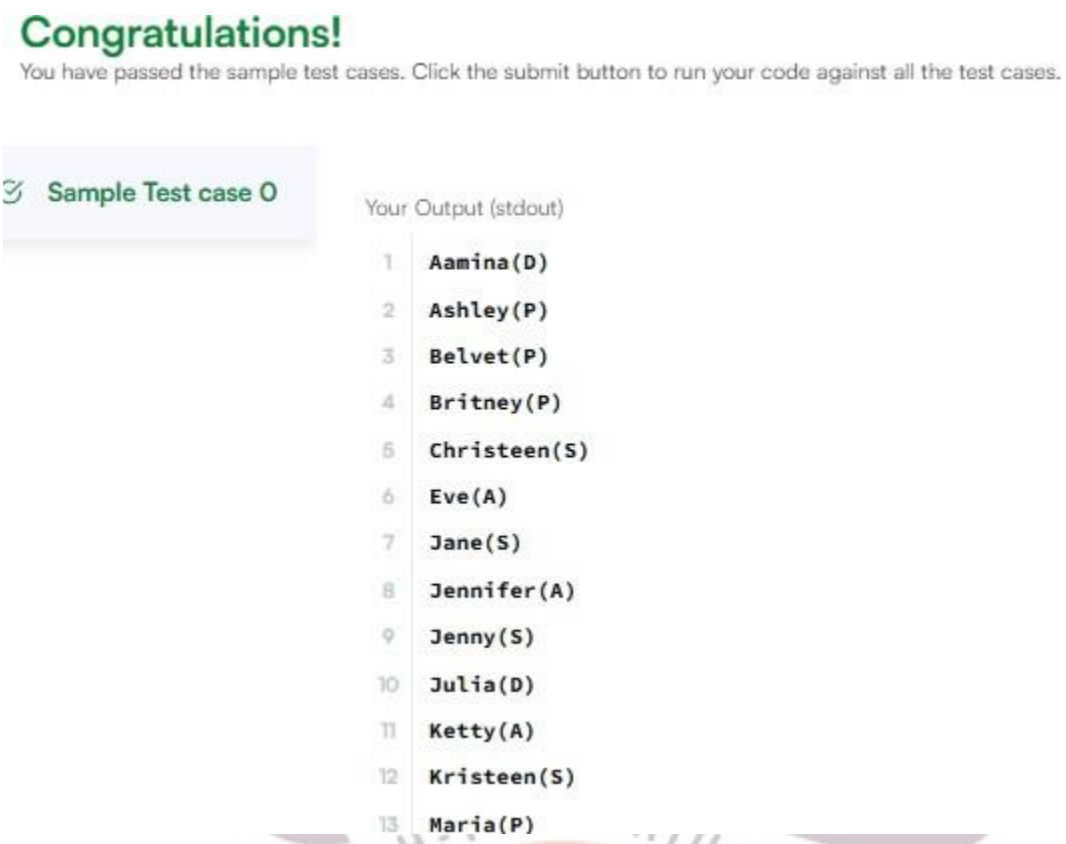
Column	*Type*
Name	*String*
Occupation	*String*

INPUT

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name ASC;

SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, LOWER(Occupation) ASC;
```

OUTPUT



Occupations

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

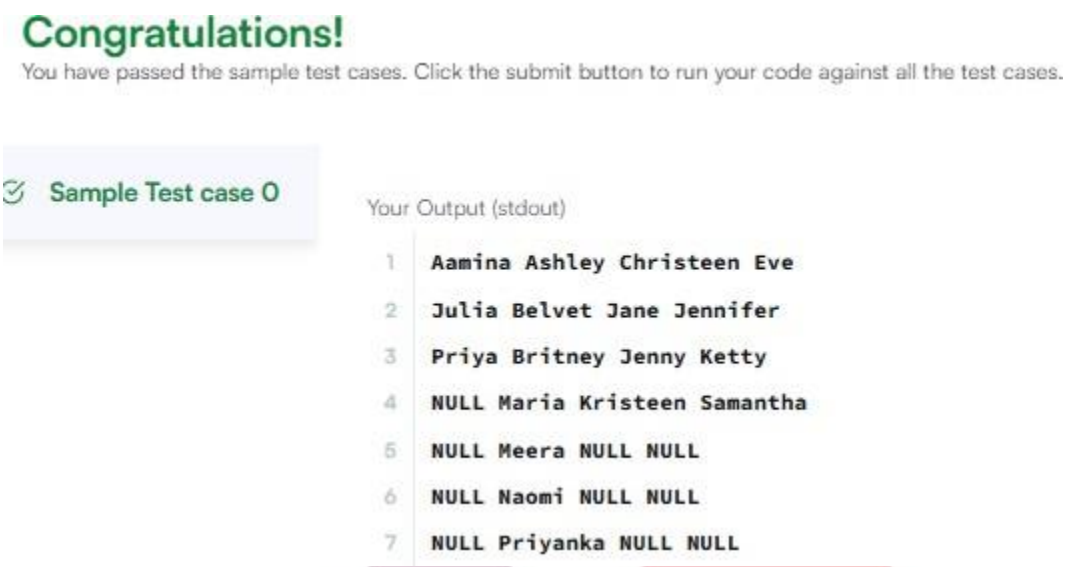
Column	Type
Name	String
Occupation	String

INPUT

```
SELECT
  MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
  MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
  MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
  MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
  SELECT
    Name,
    Occupation,
    ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS row_num
  FROM OCCUPATIONS
) AS ranked
GROUP BY row_num
ORDER BY row_num;
```

OUTPUT

TRUTH-TOIL-TRIUMPH AUTONOMOUS



Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	*Type*
N	*Integer*
P	*Integer*

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

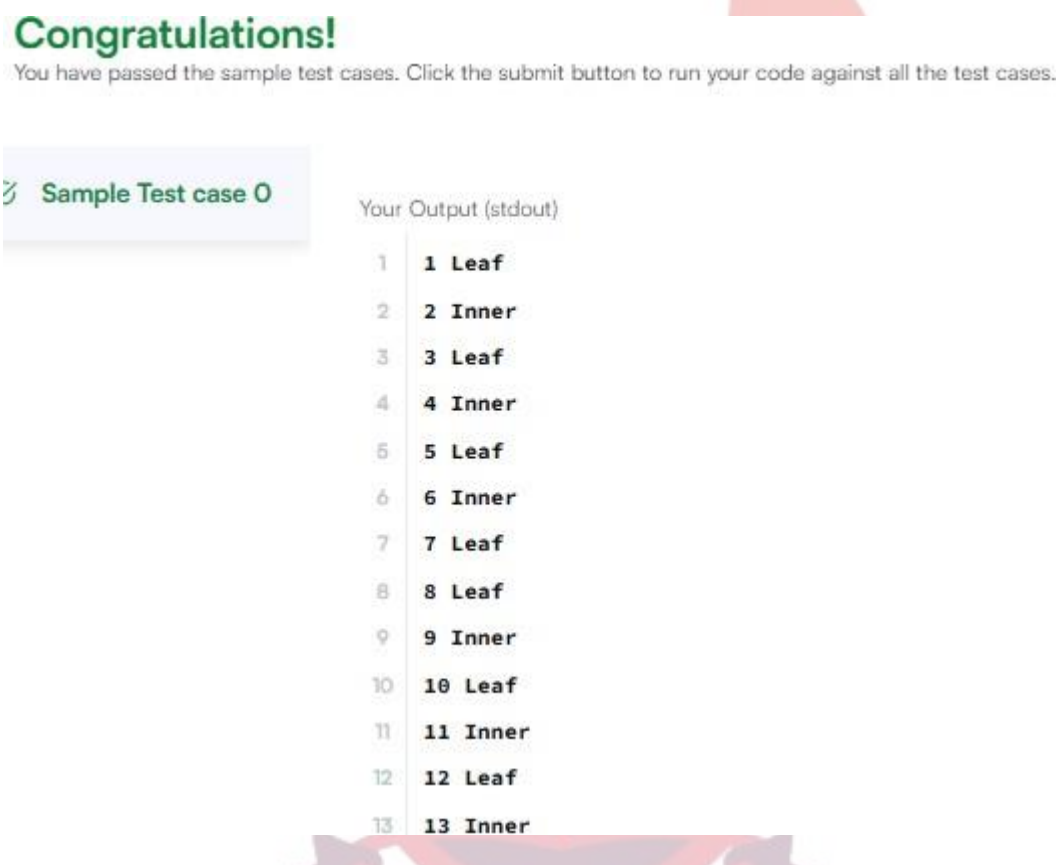
- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

INPUT

```
SELECT N,
    CASE
      WHEN P IS NULL THEN 'Root'
```

```
    WHEN N IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
    ELSE 'Leaf'
  END
FROM BST
ORDER BY N ASC;
```

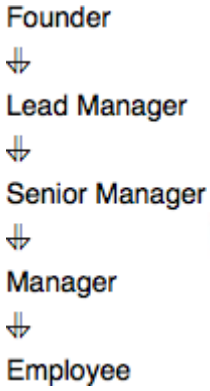
OUTPUT



AUTONOMOUS

New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



INPUT

```
SELECT
  c.company_code,
  c.founder,
  COUNT(DISTINCT lm.lead_manager_code),
  COUNT(DISTINCT sm.senior_manager_code),
  COUNT(DISTINCT m.manager_code),
  COUNT(DISTINCT e.employee_code)
FROM Company c
JOIN Lead_Manager lm ON c.company_code = lm.company_code
JOIN Senior_Manager sm ON c.company_code = sm.company_code
JOIN Manager m ON c.company_code = m.company_code
JOIN Employee e ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

OUTPUT

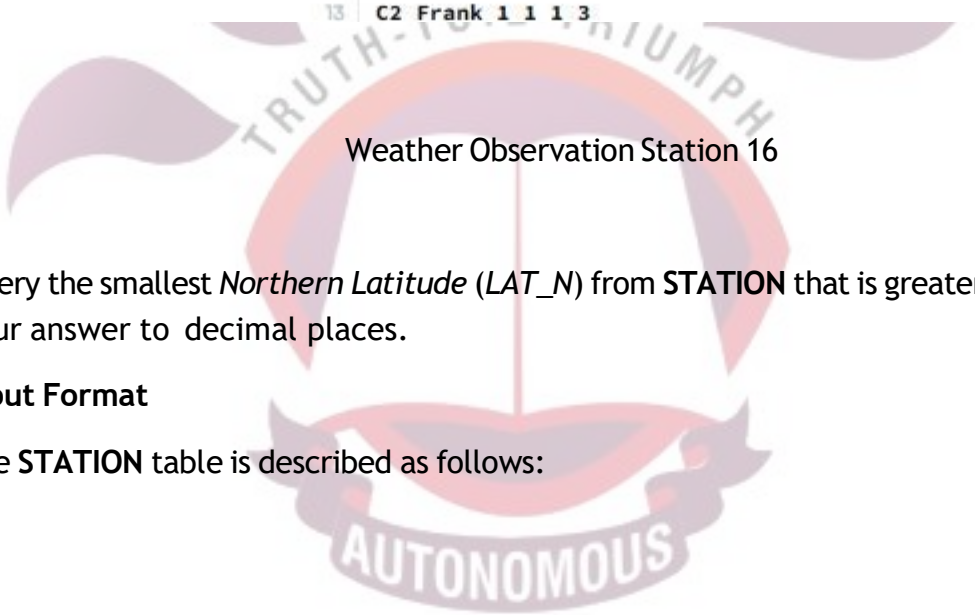
Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
C2 Frank 1 1 1 3
```



Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

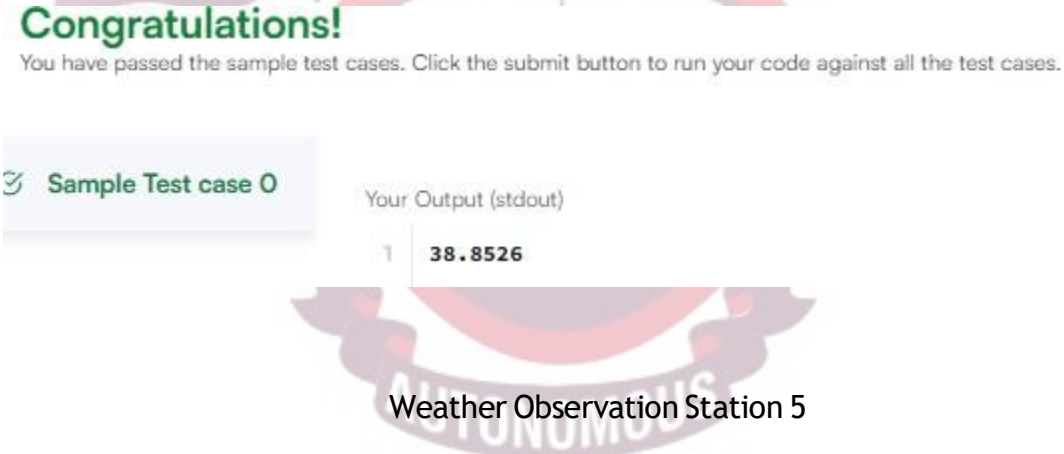
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880;
```

OUTPUT



Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) ASC, CITY ASC
LIMIT 1;

SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY ASC
LIMIT 1;
```

OUTPUT



Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

* * * * *

* * * *

* * *

* *

*

Write a query to print the pattern *P(20)*.

INPUT

```
SET @number = 21;


SELECT REPEAT('* ', @number := @number - 1)
FROM information_schema.tables
LIMIT 20;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)



Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
*
* *
* * *
* * * *
* * * * *
```

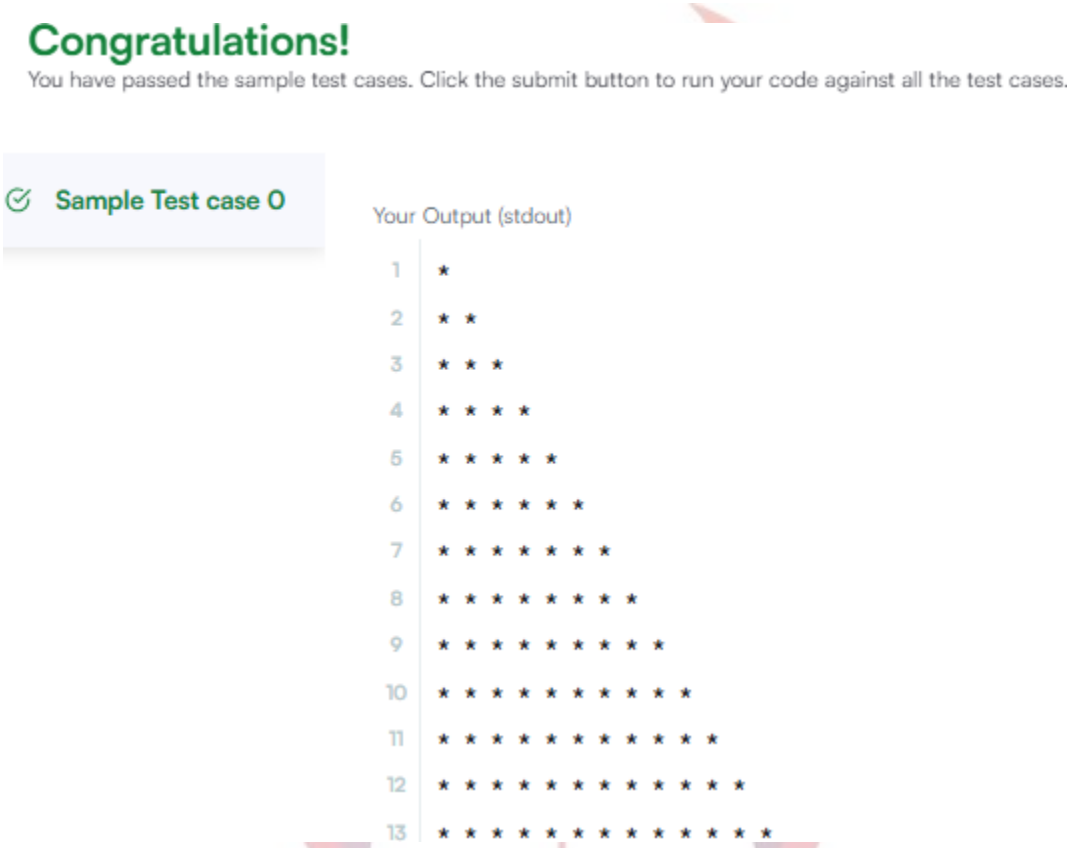
Write a query to print the pattern *P(20)*.

INPUT

SET @number = 0;

```
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
LIMIT 20;
```

OUTPUT



Weather Observation Station 2

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

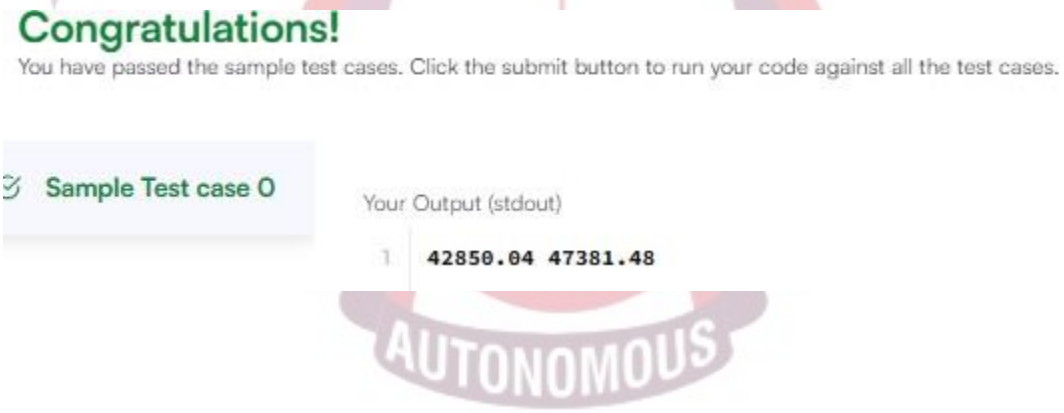
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT



Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

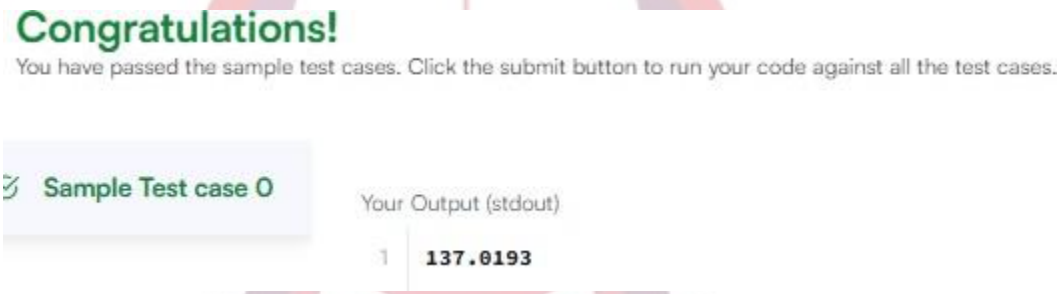
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT



Weather Observation Station 15

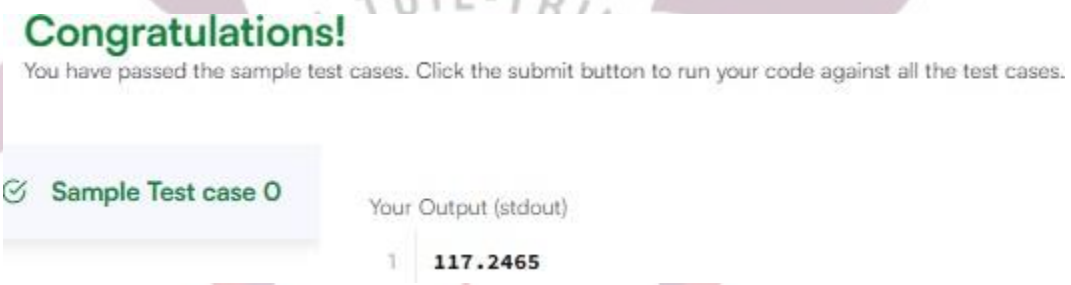
```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
```

```
  FROM STATION
  WHERE LAT_N < 137.2345
);
```

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
  SELECT MAX(LAT_N)
  FROM STATION
  WHERE LAT_N < 137.2345
);
```

OUTPUT



Revising Aggregations - The Count Function

Query a *count* of the number of cities in **CITY** having a *Population* larger than .

Input Format

The **CITY** table is described as

CITY

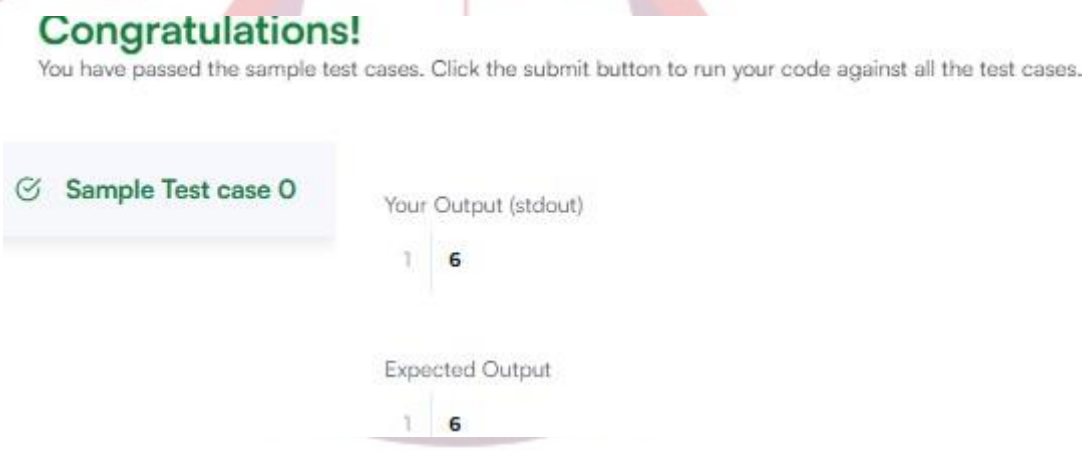
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

follows:

INPUT

```
SELECT COUNT(*)
FROM CITY
WHERE POPULATION > 100000;
```

OUTPUT



Revising Aggregations - The Sum Function

Query the total population of all cities in **CITY** where *District* is **California.**

Input Format

The **CITY** table is described as

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

follows:

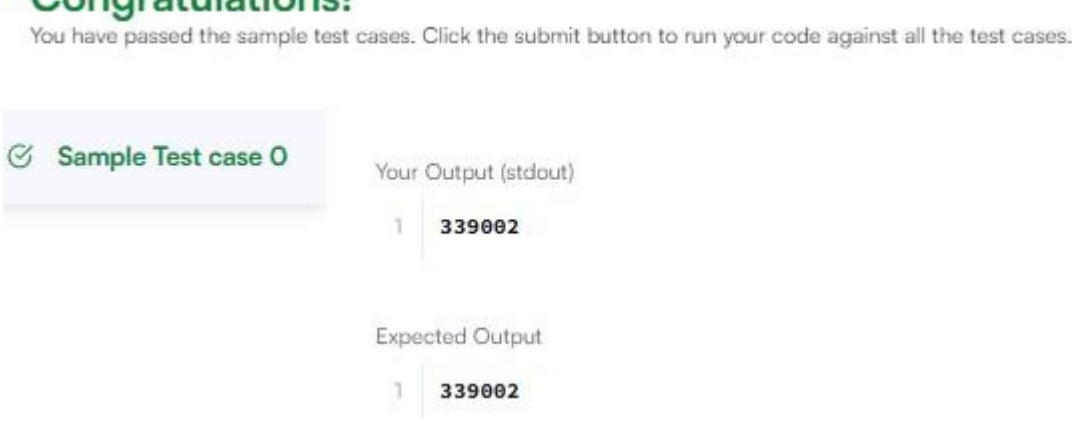
INPUT

SELECT SUM(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';

OUTPUT



Revising Aggregations - Averages

Query the average population of all cities in **CITY** where *District* is **California.**

Input Format

The **CITY** table is described as follows:

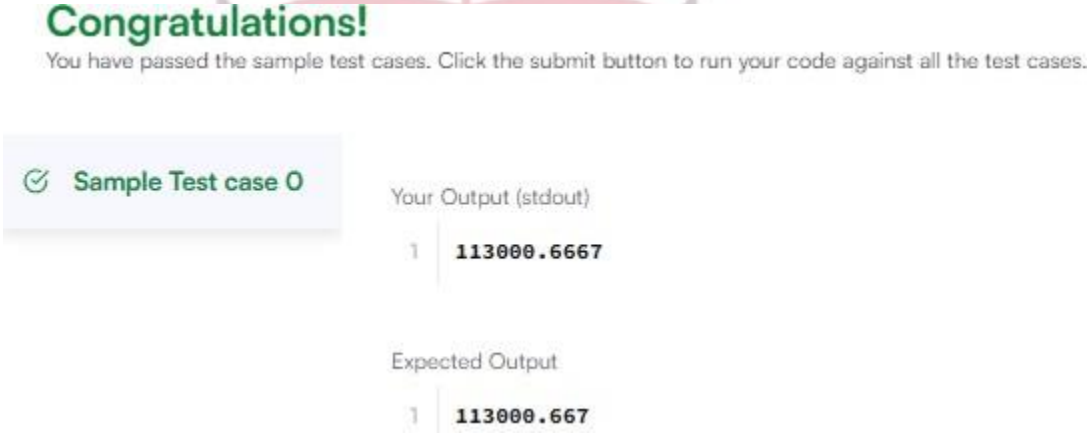
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



Average Population

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

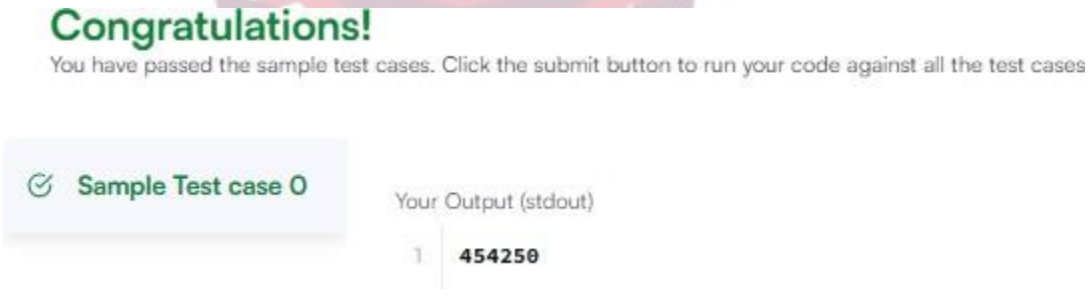
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT



Japan Population

Query the sum of the populations for all Japanese cities in **CITY**.
The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as

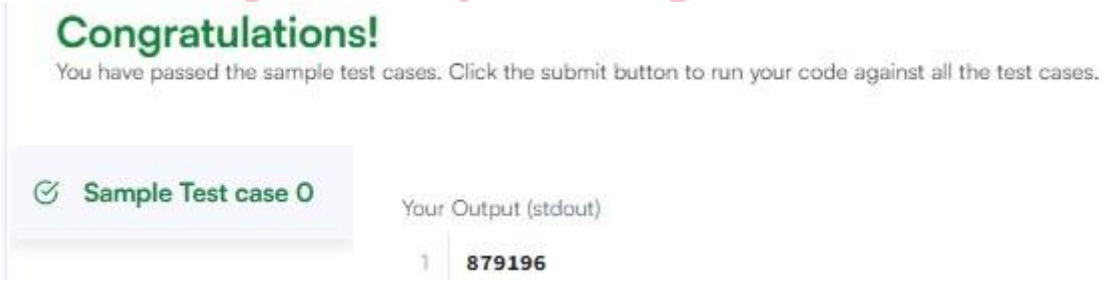
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

follows:
INPUT

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



Population Density Difference

Query the difference between the maximum and minimum populations in **CITY**.

Input Format

The **CITY** table is described as

CITY

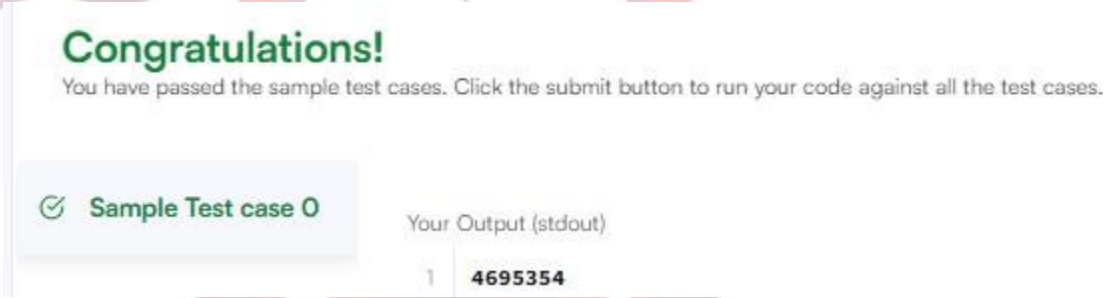
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

follows:
INPUT

SELECT MAX(POPULATION) - MIN(POPULATION)

FROM CITY;

OUTPUT



The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the

difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

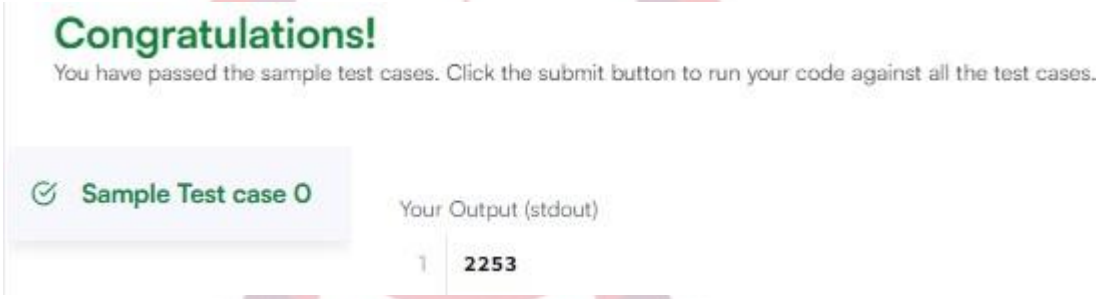
The **EMPLOYEES** table is described as follows:

Column	*Type*
ID	*Integer*
Name	*String*
Salary	*Integer*

INPUT

```
SELECT CEIL(AVG(Salary) - AVG(REPLACE(Salary, '0', '')))
FROM EMPLOYEES;
```

OUTPUT



Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all

employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:

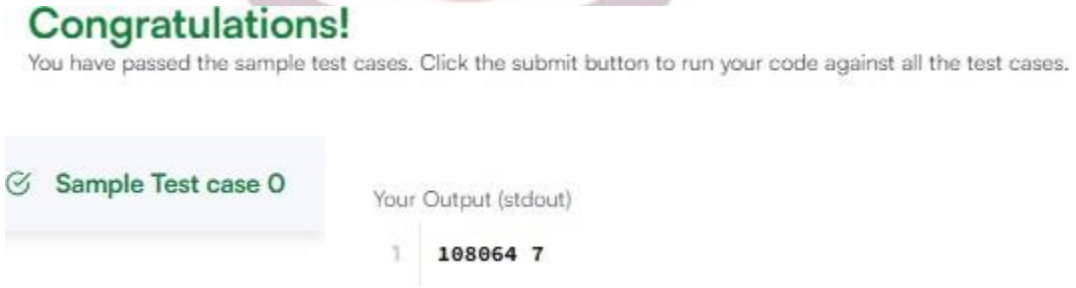
Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

INPUT

```
SELECT (months * salary) AS total_earnings, COUNT(*)
FROM Employee
GROUP BY total_earnings
ORDER BY total_earnings DESC
LIMIT 1;
```

OUTPUT



Contest Leaderboard

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

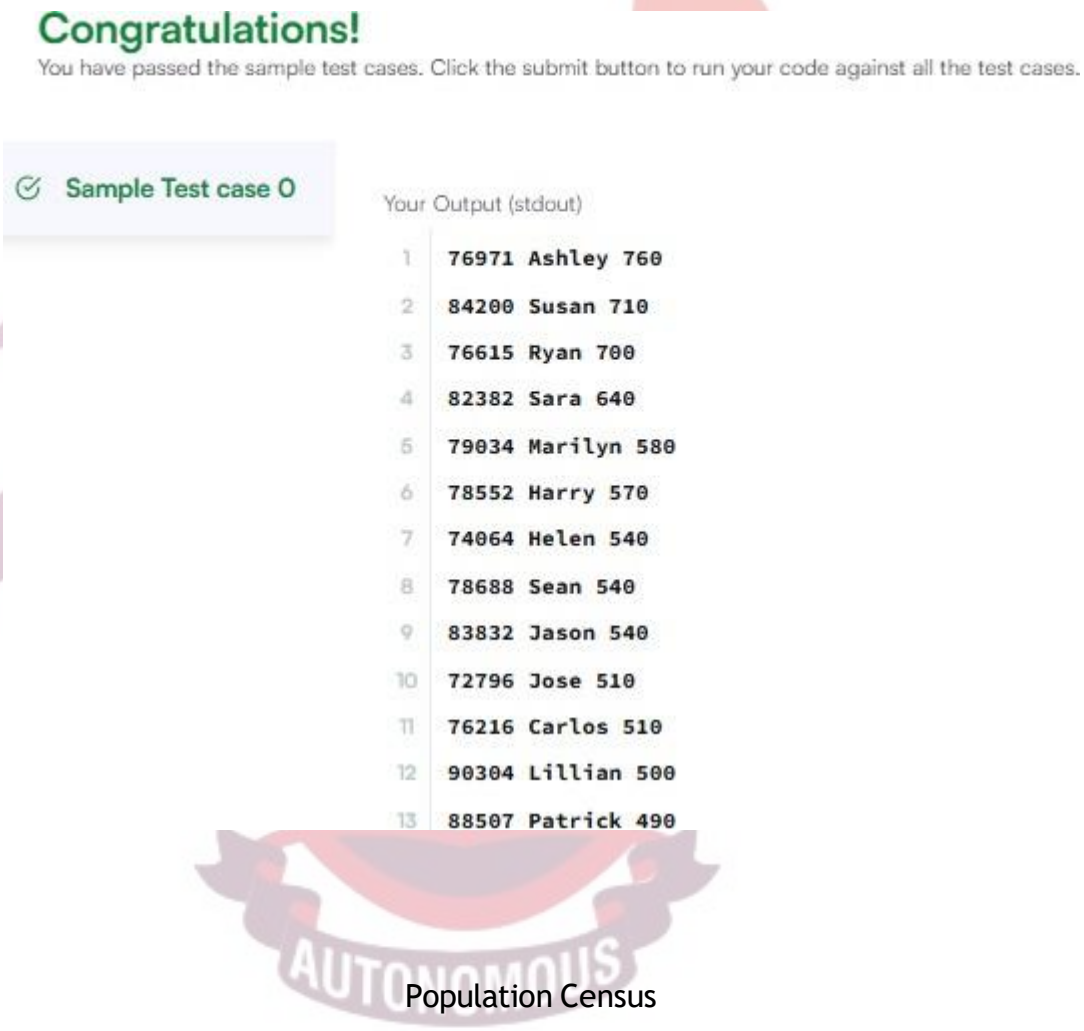
Column	Type
hacker_id	Integer
name	String

INPUT

```
SELECT
  h.hacker_id,
  h.name,
  SUM(m.max_score) AS total_score
FROM Hackers h
JOIN (
  SELECT
    hacker_id,
    challenge_id,
    MAX(score) AS max_score
  FROM Submissions
```

```
   GROUP BY hacker_id, challenge_id
) m ON h.hacker_id = m.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```

OUTPUT



Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as

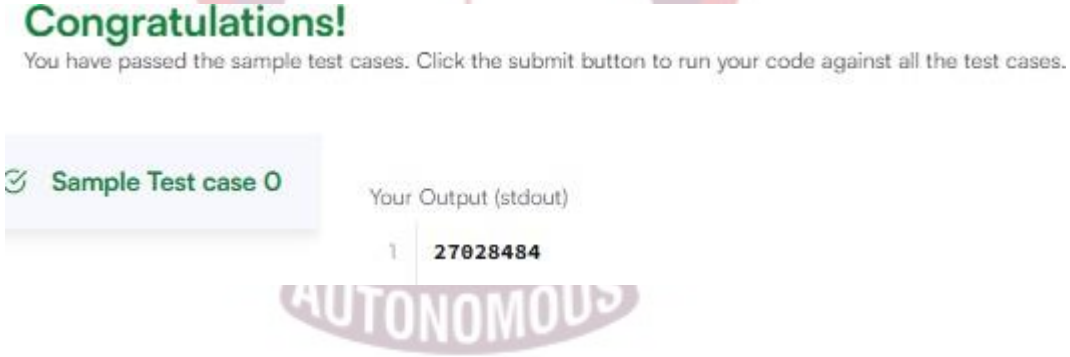
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

follows:
INPUT

```
SELECT SUM(c.POPULATION)
FROM CITY c
JOIN COUNTRY co ON c.COUNTRYCODE = co.CODE
WHERE co.CONTINENT = 'Asia';
```

OUTPUT



Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student.

Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

INPUT

```
WITH ChallengeCounts AS (
  SELECT
    h.hacker_id,
    h.name,
    COUNT(c.challenge_id) AS total_challenges
  FROM Hackers h
  JOIN Challenges c ON h.hacker_id = c.hacker_id
  GROUP BY h.hacker_id, h.name
)
SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
  -- Include hackers who created the maximum number of challenges
  SELECT MAX(total_challenges)
```

```
    FROM ChallengeCounts
)
OR total_challenges IN (
    -- Include hackers whose challenge count is unique (no ties)
    SELECT total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(*) = 1
)
ORDER BY total_challenges DESC, hacker_id ASC;
```

OUTPUT



You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

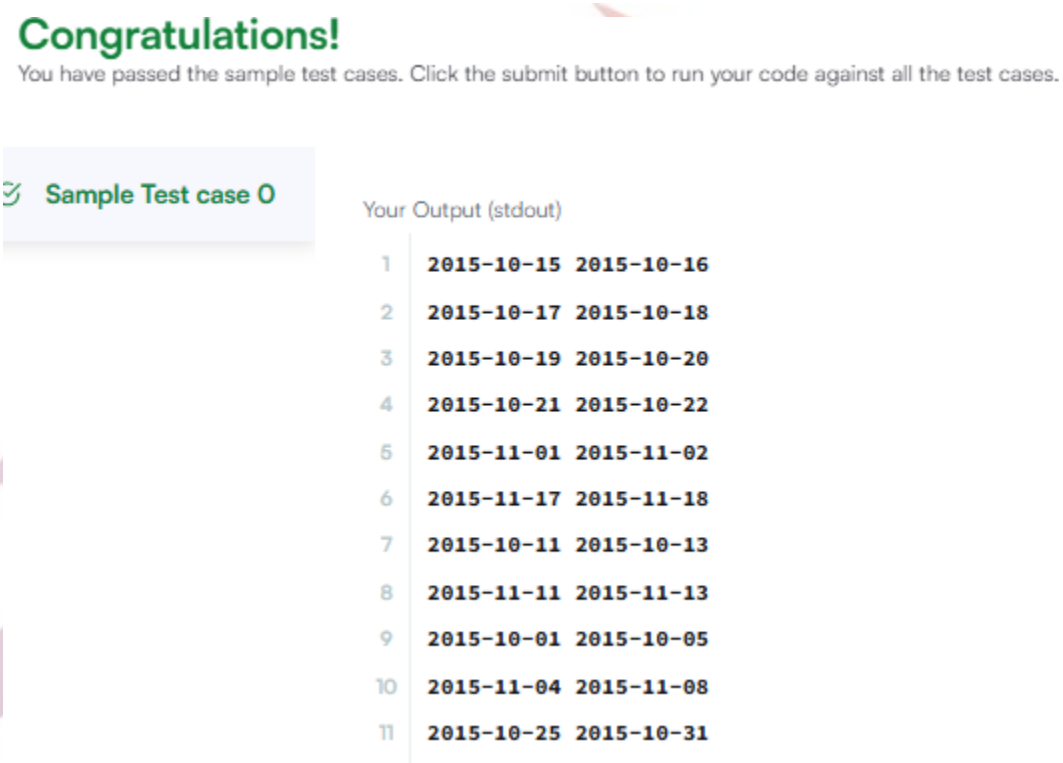
Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

INPUT

```
SELECT Start_Date, MIN(End_Date)
FROM (
  SELECT Start_Date
  FROM Projects
  WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) s
JOIN (
  SELECT End_Date
  FROM Projects
  WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) e
```

```
ON s.Start_Date < e.End_Date
GROUP BY s.Start_Date
ORDER BY DATEDIFF(MIN(e.End_Date), s.Start_Date) ASC, s.Start_Date ASC;
```

OUTPUT



AUTONOMOUS

All Products

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT:

SELECT * FROM Products;

OUTPUT:

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances

Perfect answer! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

High Price of Products

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics

product_id	product_name	price	category
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Expected columns

- product_name
- category

INPUT:

SELECT product_name, category FROM Products WHERE price > 100.00;

OUTPUT:

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

Awesome, you nailed it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Average Salary

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected column

- avg_salary

INPUT:

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT:

Your Output

```
┌─────────────┐
│ avg_salary  │
├─────────────┤
│ 62600.155   │
└─────────────┘
```

✔ Awesome, you nailed it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Locate People

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: departments

department_id department_name location

1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston

department_id	department_name	location
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

Expected column

- department_name
- location

INPUT:

SELECT department_name, location FROM departments WHERE location LIKE 'S%';

OUTPUT:

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

Hooray, you did it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Distinct Companies

Task

Write a query to select all the distinct companies (company_name) in the Works table.
Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column

- company_name

AUTONOMOUS

INPUT:

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT:

Your Output

```
┌──────────────┐
│ company_name │
├──────────────┤
│ TechCorp     │
│ InnovateLtd  │
│ HealthPlus   │
│ EduWorks     │
│ GreenTech    │
└──────────────┘
```

✔ Perfect answer! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Fiction Collection Size

Task

Write a query to find the total count of books whose genre is Fiction.
Note: Output column name should be fiction_count.

Table name: Books

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951

id	title	author	genre	price	published_year
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

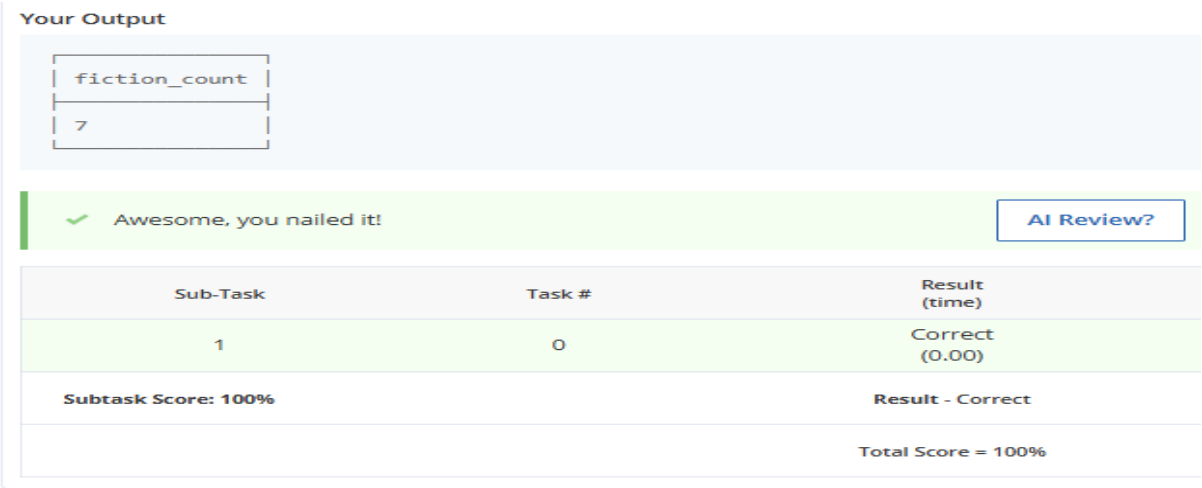
Expected column

- fiction_count

INPUT:

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

OUTPUT:



List of Movies with Ratings

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: Cinema

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5

Movie_id	Movie_name	Description	Rating
10	Galaxy War	Action	8.7

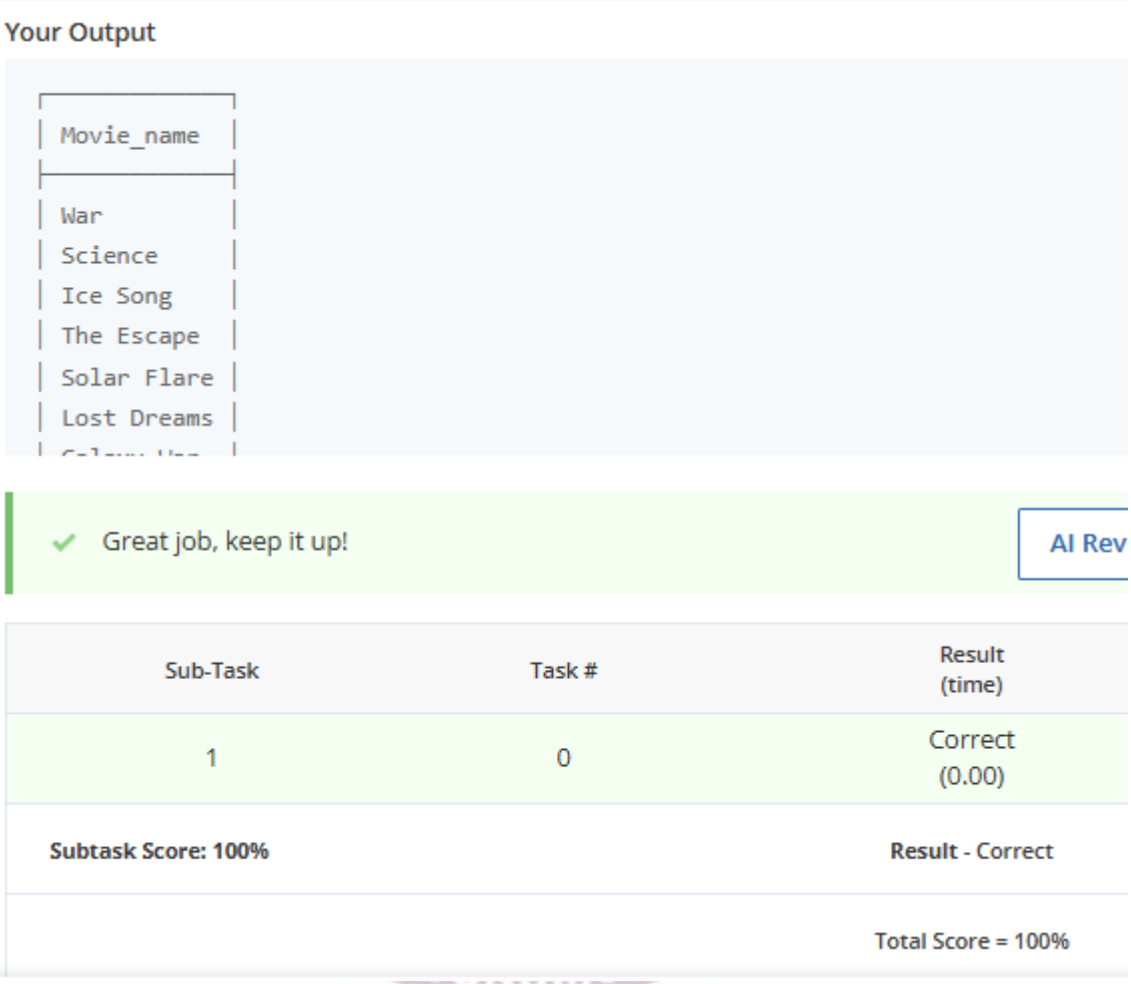
Expected column

- **Movie_name**

INPUT:

SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;

OUTPUT:



Handling NULL Values

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

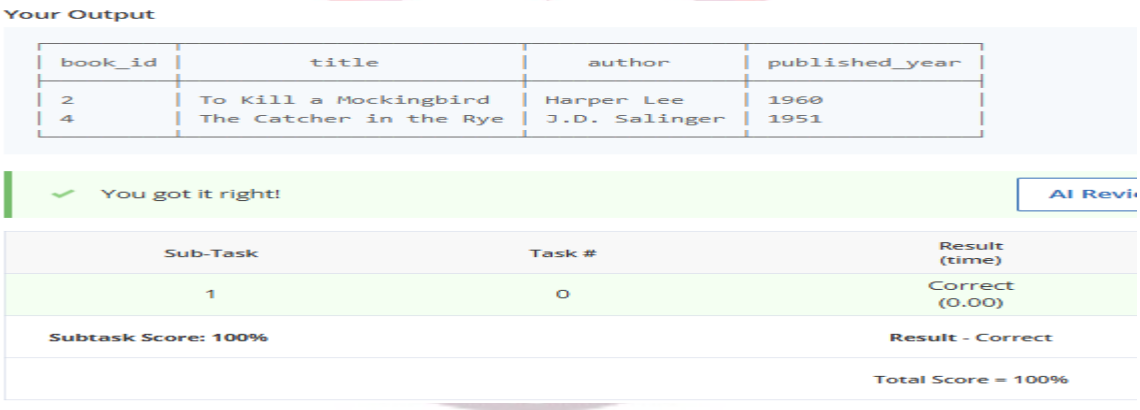
Expected column

- book_id
- title
- author
- published_year

INPUT:

SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;

OUTPUT:



Salary of Employees

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in descending order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

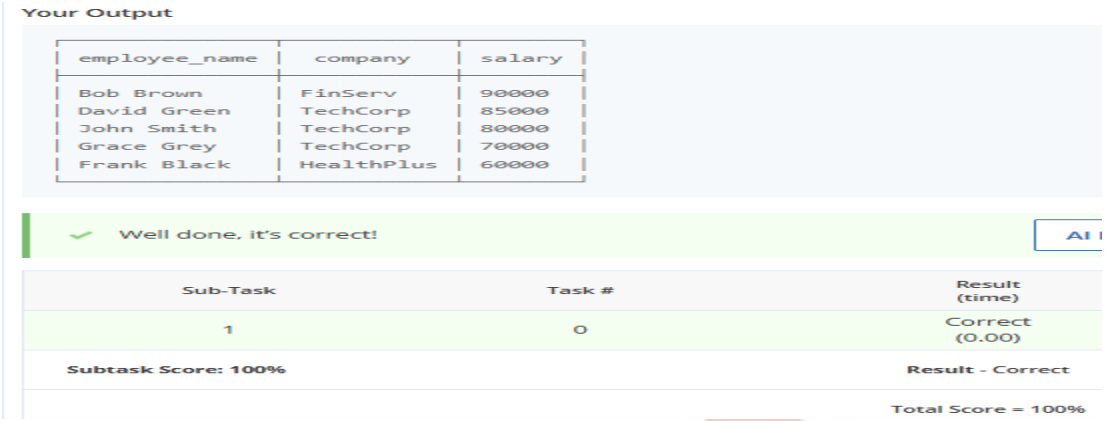
Expected column

- employee_name
- company
- salary

INPUT:

SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;

OUTPUT:



Department of Each Employee

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000

employee_id	employee_name	company	category	department	salary
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Column

- department
- total_employees

INPUT:

SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;

OUTPUT:

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

Hooray, you did it!

AI R

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Article views

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in ascending order.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected column

- author_id
- author_name
- publication_name

INPUT:

SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE view_count = 0 ORDER BY author_id ASC;

OUTPUT:

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

✔ Perfect answer! AI Re

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Subtask Score: 100% **Result** - Correct

Total Score = 100%

Player Performance Insights

Task

Write a query to find the names of the top 3 distinct players by highest score who have won matches, including their scores.

Table 1: Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: Matches

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

Expected Output Columns:

player_name	score
David	1600
Bob	1500
Charlie	1300

INPUT:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```

OUTPUT:

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Great job, keep it up!

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Player Details

Task

Write a query to retrieve the details of the last five matches played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named Players and Matches.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

- match_id
- player_1
- player_2
- winner
- match_date
- score

Expected Output:

match_id	player_1	player_2	winner	match_date	score

106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

INPUT:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT:

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

✔ Great job, keep it up!

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

PRACTICAL PROGRAMS:

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles.

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

(iv) To display the customer and the corresponding name of the vehicle hired by them.

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

(iii) SELECT COUNT(*), Make FROM CABHUB;

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

CABHUB

Vcode	VehicleName	Make	Color	Capacity	Charges
100	Innova	Toyota	WHITE	7	15
102	SX4	Suzuki	BLUE	4	14
104	C-Class	Mercedes	RED	4	35
105	A-Star	Suzuki	WHITE	3	14
108	Indigo	Tata	SILVER	3	12

CUSTOMER

Code	CName	VCode
1	HemantSahu	101
2	Raj Lal	108
3	Feroza Shah	105
4	Ketan Dhal	104

SYNTAX:

CREATE TABLE CABHUB(Vcode INT PRIMARY KEY,VehicleName VARCHAR(20),Make VARCHAR(20),Color VARCHAR(10),Capacity INT,Charges INT);

INSERT INTO CABHUB VALUES(100,'Innova','Toyota','WHITE',7,15);

INSERT INTO CABHUB VALUES(102,'SX4','Suzuki','BLUE',4,14);

INSERT INTO CABHUB VALUES(104,'C-Class','Mercedes','RED',4,35);

INSERT INTO CABHUB VALUES(105,'A-Star','Suzuki','WHITE',3,14);

INSERT INTO CABHUB VALUES(108,'Indigo','Tata','SILVER',3,12);

CREATE TABLE CUSTOMER(Code INT PRIMARY KEY,CName VARCHAR(20),VCode INT);

INSERT INTO CUSTOMER VALUES(1,'Hemant Sahu',101);

INSERT INTO CUSTOMER VALUES(2,'Raj Lal',108);

INSERT INTO CUSTOMER VALUES(3,'Feroza Shah',105);

INSERT INTO CUSTOMER VALUES(4,'Ketan Dhal',104);

QUESTION:

(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles.

INPUT:

SELECT VehicleName FROM CABHUB WHERE Color='WHITE';

OUTPUT:



QUESTION:

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

INPUT:

SELECT VehicleName,Make,Capacity FROM CABHUB ORDER BY Capacity ASC;

OUTPUT:



Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
A-Star	Suzuki	3
Indigo	Tata	3
C-Class	Mercedes	4
SX4	Suzuki	4
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

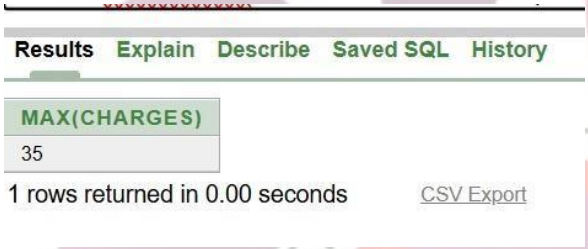
QUESTION:

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

INPUT:

SELECT MAX(Charges) FROM CABHUB;

OUTPUT:



Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) To display the customer and the corresponding name of the vehicle hired by them.

INPUT:

SELECT CUSTOMER.CName,CABHUB.VehicleName FROM CUSTOMER,CABHUB
WHERE CUSTOMER.VCode=CABHUB.VCode;

OUTPUT:



Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.02 seconds CSV Export

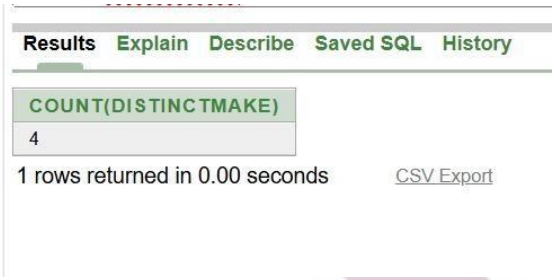
QUESTION:

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

INPUT:

SELECT COUNT(DISTINCT Make) FROM CABHUB;

OUTPUT:



QUESTION:

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

INPUT:

SELECT MAX(Charges),MIN(Charges) FROM CABHUB;

OUTPUT:



QUESTION:

(iii) SELECT COUNT(*), Make FROM CABHUB;

INPUT:

SELECT COUNT(*),Make FROM CABHUB GROUP BY Make;

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

INPUT:

SELECT VehicleName FROM CABHUB WHERE Capacity=4;

OUTPUT:



Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

(a) Display all the details of vehicles that use Petrol as the fuel type.

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

(e) Count the number of vehicles that use Diesel as the fuel type.

(f) Insert the following record into the CO2_EMISSION table. 108, 'MG', 'Hector', 'Petrol', 1.5, 160

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol'; (iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

VehicleID	Company	Model	FuelType	EngineSize	CO2Emission
101	Toyota	Corolla	Petrol	1.8	165
102	Hyundai	Creta	Diesel	1.5	145
103	Honda	City	Petrol	1.5	152
104	Tata	Nexon	Diesel	1.5	138
105	Maruti	Swift	Petrol	1.2	125
106	Kia	Seltos	Diesel	1.5	150
107	Mahindra	XUV700	Diesel	2.2	185

SYNTAX:

CREATE TABLE CO2_EMISSION(VehicleID INT PRIMARY KEY,Company VARCHAR(20),Model VARCHAR(20),FuelType VARCHAR(10),EngineSize DECIMAL(3,1),CO2Emission INT);

INSERT INTO CO2_EMISSION VALUES(101,'Toyota','Corolla','Petrol',1.8,165);

INSERT INTO CO2_EMISSION VALUES(102,'Hyundai','Creta','Diesel',1.5,145);

INSERT INTO CO2_EMISSION VALUES(103,'Honda','City','Petrol',1.5,152);

INSERT INTO CO2_EMISSION VALUES(104,'Tata','Nexon','Diesel',1.5,138);

INSERT INTO CO2_EMISSION VALUES(105,'Maruti','Swift','Petrol',1.2,125);

INSERT INTO CO2_EMISSION VALUES(106,'Kia','Seltos','Diesel',1.5,150); INSERT

INTO CO2_EMISSION VALUES(107,'Mahindra','XUV700','Diesel',2.2,185);

QUESTION:

(a) Display all the details of vehicles that use Petrol as the fuel type.

INPUT:

SELECT * FROM CO2_EMISSION WHERE FuelType='Petrol';

OUTPUT:



Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

INPUT:

SELECT Company,Model FROM CO2_EMISSION WHERE EngineSize>1.5; **OUTPUT:**



Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

INPUT:

SELECT Company,Model FROM CO2_EMISSION WHERE EngineSize>1.5;

OUTPUT:



Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

INPUT:

SELECT Company,Model,CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

OUTPUT:

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
MG	Hector	160
Toyota	Corolla	165
Mahindra	XUV700	185

8 rows returned in 0.01 seconds CSV Export

QUESTION:

(e) Count the number of vehicles that use Diesel as the fuel type.

INPUT:

SELECT * FROM CO2_EMISSION WHERE CO2Emission>150;

OUTPUT:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) Insert the following record into the CO2_EMISSION table. 108, 'MG', 'Hector', 'Petrol', 1.5, 160

INPUT:

INSERT INTO CO2_EMISSION VALUES(108,'MG','Hector','Petrol',1.5,160);

OUTPUT:



QUESTION:

(g) Write the output of the following SQL commands.

SELECT MAX(CO2Emission)FROM CO2_EMISSION;

INPUT:

SELECT MAX(CO2Emission) FROM CO2_EMISSION;

OUTPUT:



QUESTION:

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

INPUT:

SELECT MIN(EngineSize) FROM CO2_EMISSION;

OUTPUT:

Results Explain Describe Saved SQL History

MIN(ENGINESIZE)
1.2

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

INPUT:

SELECT AVG(CO2Emission) FROM CO2_EMISSION WHERE FuelType='Petrol';

OUTPUT:

Results Explain Describe Saved SQL History

AVG(CO2EMISSION)
150.5

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

INPUT:

SELECT COUNT(*) FROM CO2_EMISSION WHERE EngineSize>=1.5;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

(b) display the names of applicants, who are paying fee more than 30000.

(c) To display name of all applicants in ascending order of their joinyear.

(d) To display the year and the total number of applicants joined in each YEAR from thetable
APPLICANTS.

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in thecourse from the APPLICANTS table.

(f) To display the applicant's name with their respective course's name from thetables APPLICANTS and COURSES.

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

APPLICANTS

NO	NAME	FEE	GENDER	C_ID	JOINYEAR
1012	Amandeep	30000	M	A01	2012
1102	Avisha	25000	F	A02	2009
1103	Ekant	30000	M	A02	2011
1049	Arun	30000	M	A03	2009
1025	Amber	40000	M	A02	2011
1106	Ela	40000	F	A05	2010
1017	Nikita	35000	F	A03	2012
1108	Arleena	30000	F	A03	2012
2109	Shakti	35000	M	A04	2011
1101	Kirat	25000	M	A01	2012

COURSES

C_ID	COURSE
A01	Fashion Design
A02	Networking
A03	Hotel Management
A04	Event Management
A05	Office Management

SYNTAX:

```
CREATE TABLE APPLICANTS(NO INT PRIMARY KEY,NAME VARCHAR(30),FEE
INT,GENDER CHAR(1),C_ID VARCHAR(5),JOINYEAR INT);

INSERT INTO APPLICANTS VALUES(1012,'Amandeep',30000,'M','A01',2012);

INSERT INTO APPLICANTS VALUES(1102,'Avisha',25000,'F','A02',2009);

INSERT INTO APPLICANTS VALUES(1103,'Ekant',30000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES(1049,'Arun',30000,'M','A03',2009);

INSERT INTO APPLICANTS VALUES(1025,'Amber',40000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES(1106,'Ela',40000,'F','A05',2010);

INSERT INTO APPLICANTS VALUES(1017,'Nikita',35000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(1108,'Arleena',30000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(2109,'Shakti',35000,'M','A04',2011);

INSERT INTO APPLICANTS VALUES(1101,'Kirat',25000,'M','A01',2012);

CREATE TABLE COURSES(C_ID VARCHAR(5) PRIMARY KEY,COURSE
VARCHAR(30));

INSERT INTO COURSES VALUES('A01','Fashion Design');

INSERT INTO COURSES VALUES('A02','Networking');

INSERT INTO COURSES VALUES('A03','Hotel Management');

INSERT INTO COURSES VALUES('A04','Event Management'); INSERT

INTO COURSES VALUES('A05','Office Management');
```

QUESTION:

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

INPUT:

```
SELECT NAME,FEE,GENDER,JOINYEAR FROM APPLICANTS WHERE
JOINYEAR<2010;
```

Results Explain Describe Saved SQL History

NAME	FEE	GENDER	JOINYEAR
Avisha	25000	F	2009
Arun	30000	M	2009

2 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) display the names of applicants, who are paying fee more than 30000.

INPUT:

SELECT NAME FROM APPLICANTS WHERE FEE>30000;

OUTPUT:

Results Explain Describe Saved SQL History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(c) To display name of all applicants in ascending order of their joinyear.

INPUT:

SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME
Arun
Avisha
Ela
Ekant
Amber
Shakti
Nikita
Arleena
Amandeep
Kirat

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the year and the total number of applicants joined in each YEAR from the table
APPLICANTS.

INPUT:

SELECT JOINYEAR,COUNT(*) FROM APPLICANTS GROUP BY JOINYEAR;

OUTPUT:

Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.01 seconds CSV Export

QUESTION:

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

INPUT:

SELECT C_ID,COUNT(*) FROM APPLICANTS GROUP BY C_ID;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

C_ID	COUNT(*)
A03	3
A04	1
A02	3
A05	1
A01	2

5 rows returned in 0.00 seconds CSV Export

QUESTION:

To display the applicant's name with their respective course's name from thetables APPLICANTS and COURSES.

INPUT:

SELECT APPLICANTS.NAME,COURSES.COURSE FROM APPLICANTS,COURSES WHERE APPLICANTS.C_ID=COURSES.C_ID;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME	COURSE
Amandeep	Fashion Design
Avisha	Networking
Ekant	Networking
Arun	Hotel Management
Amber	Networking
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
Shakti	Event Management
Kirat	Fashion Design

10 rows returned in 0.01 seconds CSV Export

QUESTION:

(f) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= ‘F’ AND C_ID= ‘02’;

INPUT:

SELECT NAME,JOINYEAR FROM APPLICANTS WHERE GENDER='F' AND C_ID='02';

OUTPUT:



QUESTION:

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= ‘M’;

INPUT:

SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER='M';

OUTPUT:



QUESTION:

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= ‘A01’ OR C_ID= ‘A05’;

INPUT:

SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID='A01' OR C_ID='A05';

OUTPUT:

Results Explain Describe Saved SQL History

AVG(FEE)
31666.666666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

INPUT:

SELECT SUM(FEE),C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*)=2;

OUTPUT:

Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
55000	A01

1 rows returned in 0.00 seconds CSV Export

4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables PRODUCTS AND SUPPLIERS.

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 S02 2 S03 1

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

(f) To display the name of the companies and the name of the products in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

PRODUCTS

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01
106	LED SCREEN	70	28000	DISEXPERTS	S02
105	CAR GPS SYSSTEM	60	12000	MOVEON	S03

SUPPLIERS

SUPCODE	SNAME	CITY
S01	GET ALL INC	KOLKATA
S03	EASY MARKET CORP	DELHI
S02	DIGI BUSY GROUP	CHENNAI

SYNTAX:

CREATE TABLE PRODUCTS(PID INT PRIMARY KEY,SNAME VARCHAR(30),QTY INT,PRICE INT,COMPANY VARCHAR(30),SUPCODE VARCHAR(10));
INSERT INTO PRODUCTS VALUES(101,'DIGITAL CAMERA 14X',120,12000,'RENIX','S01');
INSERT INTO PRODUCTS VALUES(102,'DIGITAL PAD 11i',100,22000,'DIGI POP','S02');
INSERT INTO PRODUCTS VALUES(104,'PEN DRIVE 16 GB',500,1100,'STOREKING','S01');
INSERT INTO PRODUCTS VALUES(106,'LED SCREEN',70,28000,'DISEXPERTS','S02');
INSERT INTO PRODUCTS VALUES(105,'CAR GPS SYSTEM',60,12000,'MOVEON','S03');
CREATE TABLE SUPPLIERS(SUPCODE VARCHAR(10) PRIMARY KEY,SNAME VARCHAR(30),CITY VARCHAR(20));
INSERT INTO SUPPLIERS VALUES('S01','GET ALL INC','KOLKATA');
INSERT INTO SUPPLIERS VALUES('S03','EASY MARKET CORP','DELHI'); INSERT INTO SUPPLIERS VALUES('S02','DIGI BUSY GROUP','CHENNAI');

QUESTION:

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

INPUT:

SELECT * FROM PRODUCTS ORDER BY SNAME;

OUTPUT:

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA 14X	120	12000	RENIX	S01
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

INPUT:

SELECT SNAME,PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

OUTPUT:

Results Explain Describe Saved SQL History

SNAME	PRICE
DIGITAL CAMERA 14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 S02 2 S03 1

INPUT:

SELECT SUPCODE,COUNT(*) FROM PRODUCTS GROUP BY SUPCODE;

OUTPUT:

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	1
S03	1

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

INPUT:

SELECT PRICE,SNAME,QTY FROM PRODUCTS WHERE QTY>100;

OUTPUT:



Results Explain Describe Saved SQL History

PRICE	SNAME	QTY
12000	DIGITAL CAMERA 14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

INPUT:

SELECT SNAME FROM SUPPLIERS WHERE CITY='DELHI' OR CITY='CHENNAI';

OUTPUT:



Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To display the name of the companies and the name of the products in descending order of company names.

INPUT:

SELECT P.COMPANY,P.SNAME FROM PRODUCTS P ORDER BY P.COMPANY DESC;

OUTPUT:

Results Explain Describe Saved SQL History

COMPANY	SNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA 14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

INPUT:

SELECT DISTINCT SUPCODE FROM PRODUCTS;

OUTPUT:

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

INPUT:

SELECT MAX(PRICE),MIN(PRICE) FROM PRODUCTS; INPUT:

OUTPUT:

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

INPUT:

SELECT PRICE*QTY AS AMOUNT FROM PRODUCTS WHERE PID=104;

OUTPUT:



Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

INPUT:

SELECT P.SNAME,S.SNAME FROM PRODUCTS P,SUPPLIERS S WHERE P.SUPCODE=S.SUPCODE AND QTY>100;

OUTPUT:



Results Explain Describe Saved SQL History

SNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA 14X	GET ALL INC

2 rows returned in 0.00 seconds CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

(ii) To display Id and SName of all Shoppe in Nehru Place.

(iii) To display Minimum and Maximum Price of each Name of accessories.

(iv) To display Name, Price of all accessories and their respective SName where they are available.

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;*

(iii) *SELECT COUNT (DISTINCT Area) FROM SHOPPE;*

(iv) *SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN ('S02,

'S03');

SHOPPE

ID	SName	Area
S0001	ABC Computeronics	CP
S0002	All Infotech Media	GK II
S0003	Tech Shoppe	CP
S0004	Greeks Techno Soft	Nehru Place
S0005	Hitech Tech Store	Nehru Place

ACCESSORIES

No	Name	Price	ID
A01	Mother Board	12000	S01
A02	Hard Disk	5000	S01
A03	Keyboard	500	S02
A04	Mouse	300	S01
A05	Mother Board	13000	S02
A06	Keyboard	400	S03
A07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

SYNTAX:

```
CREATE TABLE SHOPPE(ID VARCHAR(10) PRIMARY KEY,SName VARCHAR(30),Area VARCHAR(20));
INSERT INTO SHOPPE VALUES('S0001','ABC Computeronics','CP');
INSERT INTO SHOPPE VALUES('S0002','All Infotech Media','GK II');
INSERT INTO SHOPPE VALUES('S0003','Tech Shoppe','CP');
INSERT INTO SHOPPE VALUES('S0004','Greeks Techno Soft','Nehru Place');
INSERT INTO SHOPPE VALUES('S0005','Hitech Tech Store','Nehru Place');
CREATE TABLE ACCESSORIES(No VARCHAR(10) PRIMARY KEY,Name VARCHAR(30),Price INT,ID VARCHAR(10));
INSERT INTO ACCESSORIES VALUES('A01','Mother Board',12000,'S01');
INSERT INTO ACCESSORIES VALUES('A02','Hard Disk',5000,'S01');
INSERT INTO ACCESSORIES VALUES('A03','Keyboard',500,'S02');
INSERT INTO ACCESSORIES VALUES('A04','Mouse',300,'S01');
INSERT INTO ACCESSORIES VALUES('A05','Mother Board',13000,'S02');
INSERT INTO ACCESSORIES VALUES('A06','Keyboard',400,'S03');
INSERT INTO ACCESSORIES VALUES('A07','LCD',6000,'S04');
INSERT INTO ACCESSORIES VALUES('T08','LCD',5500,'S05');
INSERT INTO ACCESSORIES VALUES('T09','Mouse',350,'S05');
INSERT INTO ACCESSORIES VALUES('T10','Hard Disk',4500,'S03');
```

QUESTION:

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

INPUT:

SELECT Name,Price FROM ACCESSORIES ORDER BY Price ASC; **OUTPUT:**

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(ii) To display Id and SName of all Shoppe in Nehru Place.

INPUT:

SELECT ID,SName FROM SHOPPE WHERE Area='Nehru Place';

OUTPUT:

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display Minimum and Maximum Price of each Name of accessories.

INPUT:

SELECT Name,MIN(Price),MAX(Price) FROM ACCESSORIES GROUP BY Name;

OUTPUT:



Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

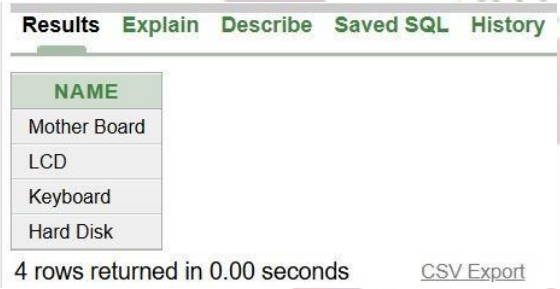
QUESTION:

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

INPUT:

SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

OUTPUT:



Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) SELECT Area, COUNT *() FROM GROUP BY Area;* **INPUT:**

SELECT Area,COUNT(*) FROM SHOPPE GROUP BY Area;

OUTPUT:

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) *SELECT COUNT (DISTINCT Area) FROM SHOPPE;*

INPUT:

SELECT COUNT(DISTINCT Area) FROM SHOPPE**;**

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) *SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES
WHERE SNo IN (‘S02,
‘S03’);

INPUT:

SELECT Name,Price*0.05 AS DISCOUNT FROM ACCESSORIES WHERE ID IN('S02','S03');

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

6.Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL .

(a) To select all the information of patients of cardiology department.
(b) To list the names of female patients who are in ENT department.
(c) To list name of all patients with their date of admission in ascending order.
(d) To display Patient's Name, Charges, Age for only female patients.
(e) To count the number of patients with Age<30.
(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M".
(g) Give the output of following SQL statements:
(i) Select COUNT (DISTINCT charges) from HOSPITAL;
(ii) Select MIN (Age) from HOSPITAL where Gender = "F".
(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT".
(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

No.	Name	Age	Department	Date of Admission	Charges	Gender
1	Arpit	62	Surgery	21/01/98	300	M
2	Zarina	22	ENT	12/12/97	250	F
3	Kareem	32	Orthopedic	19/02/98	200	M
4	Arun	12	Surgery	11/01/98	300	M
5	Zubin	30	ENT	24/02/98	250	M
6	Ketaki	16	ENT	12/01/98	250	M
7	Ankita	29	Cardiology	20/02/98	800	F
8	Zareen	45	Gynecology	22/02/98	300	F
9	Kush	19	Cardiology	13/01/98	800	M
10	Shilpa	23	Nuclear Medicine	21/02/98	400	F

SYNTAX:

```
CREATE TABLE HOSPITAL(No INT PRIMARY KEY,Name VARCHAR(30),Age
INT,Department VARCHAR(30),Datofadm DATE,Charges INT,Gender CHAR(1));
INSERT INTO HOSPITAL VALUES(1,'Arpit',62,'Surgery',DATE '1998-01-21',300,'M');
INSERT INTO HOSPITAL VALUES(2,'Zarina',22,'ENT',DATE '1997-12-12',250,'F');
INSERT INTO HOSPITAL VALUES(3,'Kareem',32,'Orthopedic',DATE '1998-02-19',200,'M');
INSERT INTO HOSPITAL VALUES(4,'Arun',12,'Surgery',DATE '1998-01-11',300,'M');
INSERT INTO HOSPITAL VALUES(5,'Zubin',30,'ENT',DATE '1998-02-24',250,'M');
INSERT INTO HOSPITAL VALUES(6,'Ketaki',16,'ENT',DATE '1998-01-12',250,'M');
INSERT INTO HOSPITAL VALUES(7,'Ankita',29,'Cardiology',DATE '1998-02-20',800,'F');
```

INSERT INTO HOSPITAL VALUES(8,'Zareen',45,'Gynecology',DATE '1998-02-22',300,'F');
INSERT INTO HOSPITAL VALUES(9,'Kush',19,'Cardiology',DATE '1998-01-13',800,'M');
INSERT INTO HOSPITAL VALUES(10,'Shilpa',23,'Nuclear Medicine',DATE '1998-02-21',400,'F');

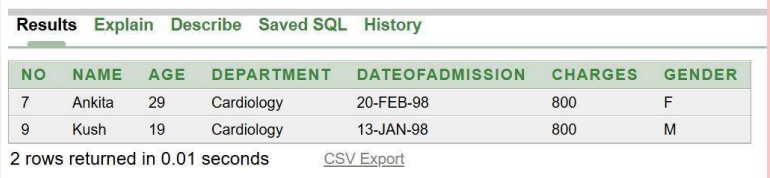
QUESTION:

(a) To select all the information of patients of cardiology department.

INPUT:

SELECT * FROM HOSPITAL WHERE Department='Cardiology';

OUTPUT:



Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADMISSION	CHARGES	GENDER
7	Ankita	29	Cardiology	20-FEB-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.01 seconds CSV Export

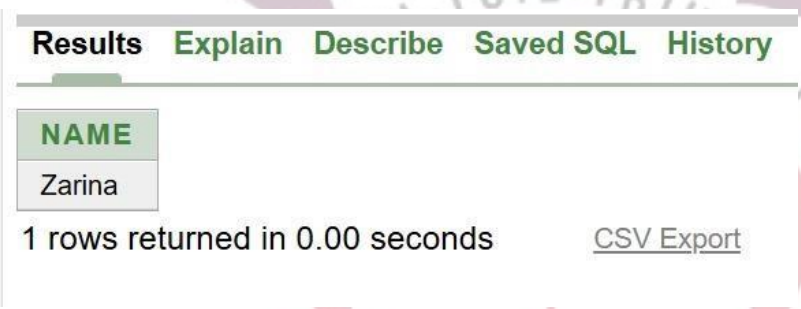
QUESTION:

(b) To list the names of female patients who are in ENT department.

INPUT:

SELECT Name FROM HOSPITAL WHERE Gender='F' AND Department='ENT';

OUTPUT:



Results Explain Describe Saved SQL History

NAME
Zarina

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To list name of all patients with their date of admission in ascending order.

INPUT:

SELECT Name,DateofAdmission FROM HOSPITAL ORDER BY DateofAdmission ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DATEOFADMISSION
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display Patient's Name, Charges, Age for only female patients.

INPUT:

SELECT Name,Charges,Age FROM HOSPITAL WHERE Gender='F';

OUTPUT:

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(e) To count the number of patients with Age<30.

INPUT:

SELECT COUNT(*) FROM HOSPITAL WHERE Age<30;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

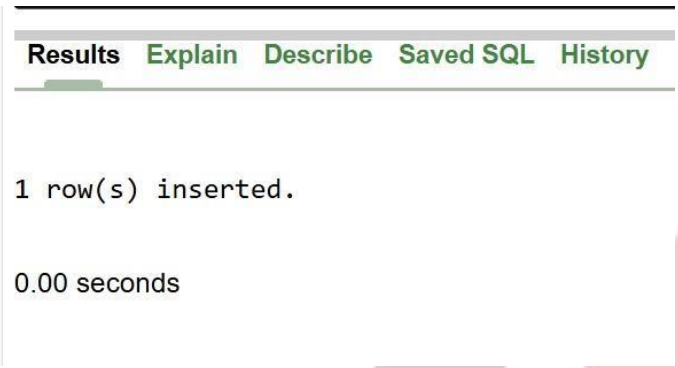
QUESTION:

(f) To inset in a new row in the HOSPITAL table with the following data: 11, “Aftab”, 24, “Surgery”, {25/02/98}, 300, “M”.

INPUT:

INSERT INTO HOSPITAL VALUES(11,'Aftab',24,'Surgery',DATE '1998-02-25',300,'M');

OUTPUT:



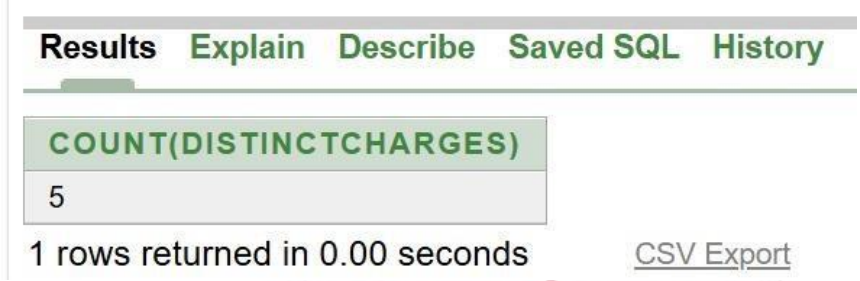
QUESTION:

(g) Give the output of following SQL statements: (i) Select COUNT (DISTINCT charges) from HOSPITAL;

INPUT:

SELECT COUNT(DISTINCT Charges) FROM HOSPITAL;

OUTPUT:



QUESTION:

(ii) Select MIN (Age) from HOSPITAL where Gender = “F”.

INPUT:

SELECT MIN(Age) FROM HOSPITAL WHERE Gender='F';

OUTPUT:



QUESTION:

(iii) Select SUM (Charges) from HOSPITAL where Department = “ENT”.

(iv) **INPUT:**

SELECT SUM(Charges) FROM HOSPITAL WHERE Department='ENT';

OUTPUT:



QUESTION:

(v) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

INPUT:

SELECT AVG(Charges) FROM HOSPITAL WHERE DateofAdmission<TO_DATE('12-AUG1998','DD-MON-YYYY');

OUTPUT:



7.Answer the questions (a) and (b) on the basis of the following tables STORE and ITEM

(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

(ii) To display SNo and SName of all store location in CP.

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

(iv) To display IName, Price of all items and their respective SName where they areavailable.

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area; (iii)

SELECT COUNT (DISTINCT Area) FROM STORE:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN (‘S02’, ‘S03’);

STORE

SNo	SName	Area
S01	ABC Computronics	GK II
S02	All InfoTech Media	CP
S03	Tech Shoppe	Nehru Place
S04	Geeks Techno Soft	Nehru Place
S05	Hitech Tech Store	CP

ITEM

INo	IName	Price	SNo
T01	Mother Board	12000	S01
T02	Hard Disk	5000	S01
T03	Keyboard	500	S02
T04	Mouse	300	S01
T05	Mother Board	13000	S02
T06	Keyboard	400	S03
T07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

SYNTAX:

```
CREATE TABLE STORE(SNo VARCHAR(10) PRIMARY KEY,SName VARCHAR(30),Area VARCHAR(20));
INSERT INTO STORE VALUES('S01','ABC Computeronics','GK II');
INSERT INTO STORE VALUES('S02','All InfoTech Media','CP');
INSERT INTO STORE VALUES('S03','Tech Shoppe','Nehru Place');
INSERT INTO STORE VALUES('S04','Geeks Techno Soft','Nehru Place');
INSERT INTO STORE VALUES('S05','Hitech Tech Store','CP');
CREATE TABLE ITEM(INo VARCHAR(10) PRIMARY KEY,IName VARCHAR(30),Price INT,SNo VARCHAR(10));
INSERT INTO ITEM VALUES('T01','Mother Board',12000,'S01');
INSERT INTO ITEM VALUES('T02','Hard Disk',5000,'S01');
INSERT INTO ITEM VALUES('T03','Keyboard',500,'S02');
INSERT INTO ITEM VALUES('T04','Mouse',300,'S01');
INSERT INTO ITEM VALUES('T05','Mother Board',13000,'S02');
INSERT INTO ITEM VALUES('T06','Keyboard',400,'S03');
```

INSERT INTO ITEM VALUES('T07','LCD',6000,'S04');
INSERT INTO ITEM VALUES('T08','LCD',5500,'S05');
INSERT INTO ITEM VALUES('T09','Mouse',350,'S05'); INSERT
INTO ITEM VALUES('T10','Hard Disk',4500,'S03');

QUESTION:

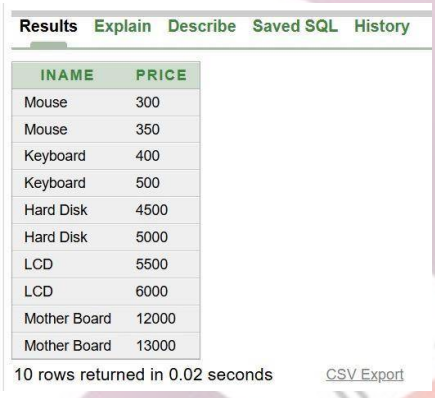
(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

INPUT:

SELECT IName,Price FROM ITEM ORDER BY Price ASC;

OUTPUT:



Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(ii) To display SNo and SName of all store location in CP.

INPUT:

SELECT SNo,SName FROM STORE WHERE Area='CP';

OUTPUT:



Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

INPUT:

SELECT IName,MIN(Price),MAX(Price) FROM ITEM GROUP BY IName;

OUTPUT:

Results Explain Describe Saved SQL History

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv) To display IName, Price of all items and their respective SName where they are available.

INPUT:

SELECT I.IName,I.Price,S.SName FROM ITEM I,STORE S WHERE I.SNo=S.SNo;

OUTPUT:

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) Write the output of the following SQL commands (i) to (iv): (i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

INPUT:

SELECT DISTINCT IName FROM ITEM WHERE Price>=5000;

OUTPUT:



Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.01 seconds CSV Export

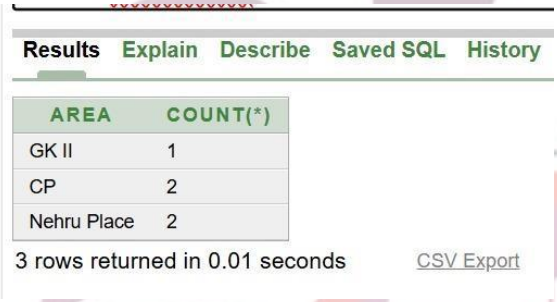
QUESTION:

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

INPUT:

SELECT Area,COUNT(*) FROM STORE GROUP BY Area;

OUTPUT:



Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.01 seconds CSV Export

QUESTION:

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

INPUT:

SELECT COUNT(DISTINCT Area) FROM STORE;

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

INPUT:

SELECT IName,Price*0.05 AS DISCOUNT FROM ITEM WHERE SNo IN('S02','S03');

OUTPUT:

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.02 seconds CSV Export

8.Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

(a) To show all information about the teacher of history department

(b) To list the names of female teacher who are in Hindi department

(c) To list names of all teachers with their date of joining in ascending order. (d) To display teacher's Name, Salary, Age for male teacher only (e) To count the number of teachers with Age>23.

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

(g) Give the output of following SQL statements:

(i) Select COUNT (distinct department) from TEACHER;

(II) Select MAX (Age) from TEACHER where Gender= "F"

(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96}; (iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

No.	Name	Age	Department	Date of Join	Salary	Gender
1	Jugal	34	Computer	10/01/97	12000	M
2	Sharmila	31	History	24/03/98	20000	F
3	Sandeep	32	Maths	12/12/96	30000	M
4	Sangeeta	35	History	01/07/99	40000	F
5	Rakesh	42	Maths	05/09/97	25000	M
6	Shyam	50	History	27/06/98	30000	M
7	Shiv Om	44	Computer	25/02/97	21000	M
8	Shalakha	33	Maths	31/07/97	20000	F

SYNTAX:

CREATE TABLE TEACHER(No INT PRIMARY KEY,Name VARCHAR(20),Age INT,Department VARCHAR(20),DateofJoin DATE,Salary INT,Gender CHAR(1));

```
INSERT INTO TEACHER VALUES(1,'Jugal',34,'Computer',TO_DATE('1997-01-10','YYYYMM-DD'),12000,'M');

INSERT INTO TEACHER VALUES(2,'Sharmila',31,'History',TO_DATE('1998-03-24','YYYYMM-DD'),20000,'F');

INSERT INTO TEACHER VALUES(3,'Sandeep',32,'Maths',TO_DATE('1996-12-12','YYYYMM-DD'),30000,'M');

INSERT INTO TEACHER VALUES(4,'Sangeeta',35,'History',TO_DATE('1999-07-01','YYYYMM-DD'),40000,'F');

INSERT INTO TEACHER VALUES(5,'Rakesh',42,'Maths',TO_DATE('1997-09-05','YYYYMM-DD'),25000,'M');

INSERT INTO TEACHER VALUES(6,'Shyam',50,'History',TO_DATE('1998-06-27','YYYYMM-DD'),30000,'M');

INSERT INTO TEACHER VALUES(7,'Shiv Om',44,'Computer',TO_DATE('1997-02-25','YYYY-MM-DD'),21000,'M');

INSERT INTO TEACHER VALUES(8,'Shalakha',33,'Maths',TO_DATE('1997-07-31','YYYYMM-DD'),20000,'F');

INSERT INTO TEACHER VALUES(9,'Raja',26,'Computer',TO_DATE('1995-05-13','YYYYMM-DD'),2300,'M');
```

QUESTION:

(a) To show all information about the teacher of history department

INPUT:

```
SELECT * FROM TEACHER WHERE Department='History';
```

OUTPUT:



Results | Explain | Describe | Saved SQL | History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To list the names of female teacher who are in Hindi department

INPUT:

SELECT Name FROM TEACHER WHERE Gender='F' AND Department='Hindi'

OUTPUT:



QUESTION:

(c) To list names of all teachers with their date of joining in ascending order.

INPUT:

SELECT Name,DateofJoin FROM TEACHER ORDER BY DateofJoin ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DATEOFJOIN
Raja	13-MAY-95
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98
Sangeeta	01-JUL-99

9 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display teacher's Name, Salary, Age for male teacher only

INPUT:

SELECT Name,Salary,Age FROM TEACHER WHERE Gender='M';

OUTPUT:

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44
Raja	2300	26

6 rows returned in 0.02 seconds CSV Export

QUESTION:

(e) To count the number of teachers with Age>23.

INPUT:

SELECT COUNT(*) FROM TEACHER WHERE Age>23;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(*)
9

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To inset a new row in the TEACHER table with the following data: 9, “Raja”, 26, “Computer”, {13/05/95}, 2300, “M”

INPUT:

SELECT COUNT(DISTINCT Department) FROM TEACHER; **OUTPUT:**

Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.00 seconds CSV Export

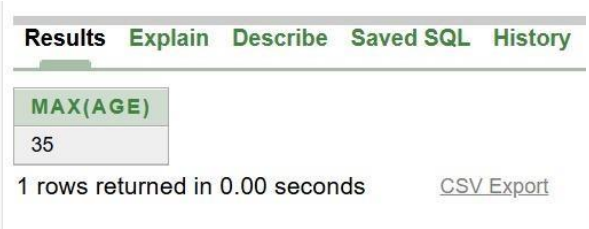
QUESTION:

(g) Give the output of following SQL statements: (i) Select COUNT (distinct department) from TEACHER;

INPUT:

SELECT MAX(Age) FROM TEACHER WHERE Gender='F';

OUTPUT:



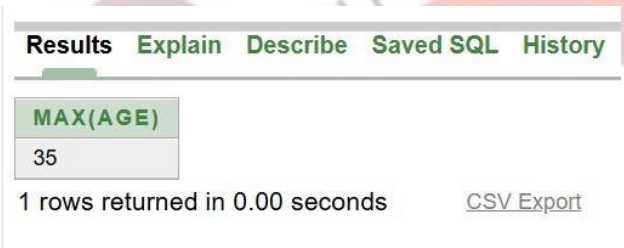
QUESTION:

(II) Select MAX (Age) from TEACHER where Gender= "F"

INPUT:

SELECT MAX(Age) FROM TEACHER WHERE Gender='F';

OUTPUT:



QUESTION:

(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

INPUT:

SELECT AVG(Salary) FROM TEACHER WHERE DateofJoin<TO_DATE('1996-07-12','YYYY-MM-DD');

OUTPUT:


Results Explain Describe Saved SQL History

AVG(SALARY)
2300

1 rows returned in 0.02 seconds CSV Export

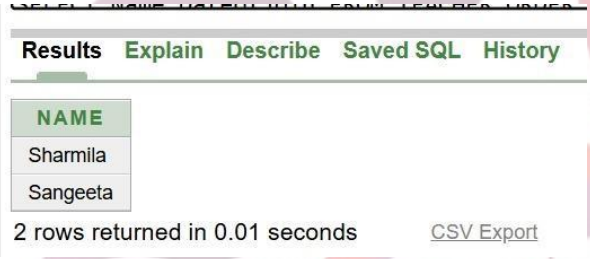
QUESTION:

(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

INPUT:

SELECT SUM(Salary) FROM TEACHER WHERE DateofJoin<TO_DATE('1996-07-12','YYYY-MM-DD');

OUTPUT:


Results Explain Describe Saved SQL History

NAME
Sharmila
Sangeeta

2 rows returned in 0.01 seconds CSV Export

9.Given the following tables for a database LIBRARY: Write SQL queries for (a) to (f):

(a) To show Book name, Author name and Price of books of EPB publishers.
(b) To list the names of the books of Fiction Type.
(c) To display the names and price of the books in descending order of their price.
(d) To increase the price of all books of first publisher by 50.
(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).
(f) To insert a new row in the table Issued following the data: “F0002”,4.
(g) Give the output of the following queries based on the above tables:
(i) SELECT COUNT(DISTINCT Publishers) FROM Books;
(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;
(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;
(iv) SELECT COUNT (*) FROM Books;

Books

Book Id	Book Name	Author Name	Publishers	Price	Type	Qty.
F0001	The Tears	William Hopkins	First Publ.	750	Fiction	10
F0002	Thunderbolts	Anna Roberts	First Publ.	700	Fiction	5
T0001	My First C++	Briian & Brooke	EPB	250	Text	10
T0002	C++ Brainworks	A. W. Rossaine	TDH	325	Text	5
C0001	Fast Cook	LataKapoor	EPB	350	Cookery	8

Issued

Book Id	Quantity Issued
F0001	3
T0001	1
C0001	5

SYNTAX:

```
CREATE TABLE Books(Book_Id VARCHAR(10) PRIMARY KEY,Book_Name VARCHAR(30),Author_Name VARCHAR(30),Publishers VARCHAR(20),Price INT,Type VARCHAR(20),Qty INT);
INSERT INTO Books VALUES('F0001','The Tears','William Hopkins','First Publ.',750,'Fiction',10);
INSERT INTO Books VALUES('F0002','Thunderbolts','Anna Roberts','First Publ.',700,'Fiction',5);
INSERT INTO Books VALUES('T0001','My First C++','Brian & Brooke','EPB',250,'Text',10);
INSERT INTO Books VALUES('T0002','C++ Brainworks','A. W. Rossaine','TDH',325,'Text',5);
INSERT INTO Books VALUES('C0001','Fast Cook','Lata Kapoor','EPB',350,'Cookery',8);
CREATE TABLE Issued(Book_Id VARCHAR(10),Quantity_Issued INT);
INSERT INTO Issued VALUES('F0001',3);
INSERT INTO Issued VALUES('T0001',1);
INSERT INTO Issued VALUES('C0001',5);
SELECT Book_Name,Author_Name,Price FROM Books WHERE Publishers='EPB';
SELECT Book_Name FROM Books WHERE Type='Fiction';
SELECT Book_Name,Price FROM Books ORDER BY Price DESC;
UPDATE Books SET Price=Price+50 WHERE Publishers='First Publ.';
SELECT B.Book_Id,B.Book_Name,I.Quantity_Issued FROM Books B,Issued I WHERE B.Book_Id=I.Book_Id;
INSERT INTO Issued VALUES('F0002',4);
SELECT COUNT(DISTINCT Publishers) FROM Books;
SELECT SUM(Price) FROM Books WHERE Qty>5;
SELECT Book_Name,Author_Name FROM Books WHERE Price<500;
SELECT COUNT(*) FROM Books;
```

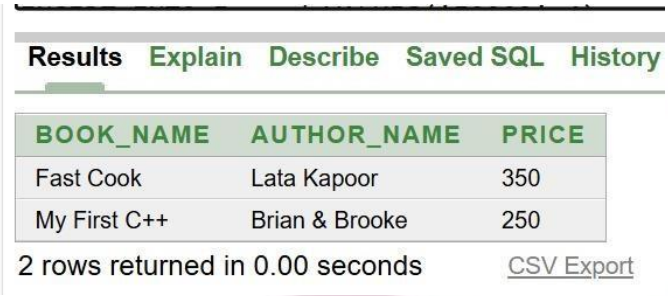
QUESTION:

(a) To show Book name, Author name and Price of books of EPB publishers.

INPUT:

SELECT Book_Name,Author_Name,Price FROM Books WHERE Publishers='EPB';

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
Fast Cook	Lata Kapoor	350
My First C++	Brian & Brooke	250

2 rows returned in 0.00 seconds CSV Export

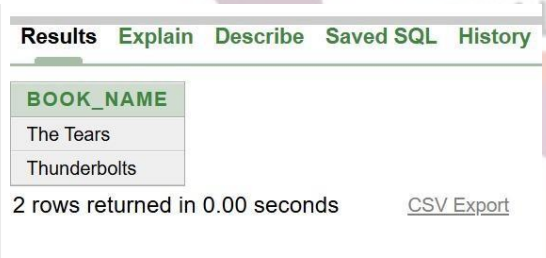
QUESTION:

(b) To list the names of the books of Fiction Type.

INPUT:

SELECT Book_Name FROM Books WHERE Type='Fiction';

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME
The Tears
Thunderbolts

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To display the names and price of the books in descending order of their price.

INPUT:

SELECT Book_Name,Price FROM Books ORDER BY Price DESC;

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To increase the price of all books of first publisher by 50.

INPUT:

UPDATE Books SET Price=Price+50 WHERE Publishers='First Publ.';

OUTPUT:



Results Explain Describe Saved SQL History

2 row(s) updated.

0.02 seconds

QUESTION:

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

INPUT:

SELECT B.Book_Id,B.Book_Name,I.Quantity_Issued FROM Books B,Issued I WHERE B.Book_Id=I.Book_Id;

OUTPUT:

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
C0001	Fast Cook	5
F0001	The Tears	3
F0001	The Tears	3
T0001	My First C++	1

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To insert a new row in the table Issued following the data: "F0002",4.

INPUT:

INSERT INTO Issued VALUES('F0002',4);

OUTPUT:

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUESTION:

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

INPUT:

SELECT COUNT(DISTINCT Publishers) FROM Books;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

INPUT:

SELECT SUM(Price) FROM Books WHERE Qty>5;

OUTPUT:



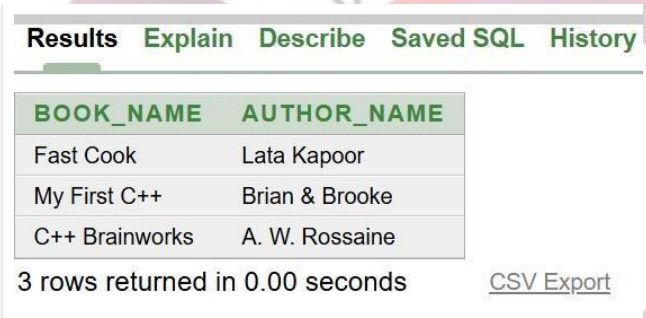
QUESTION:

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

INPUT:

SELECT Book_Name,Author_Name FROM Books WHERE Price<500;

OUTPUT:



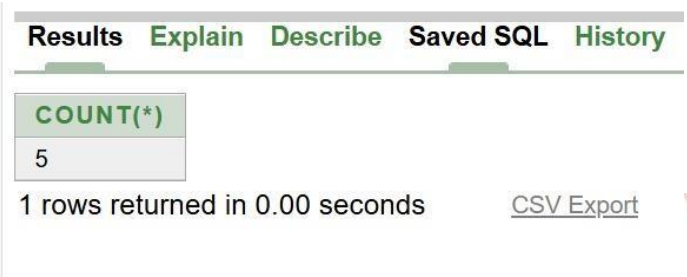
QUESTION:

(iv) SELECT COUNT (*) FROM Books;

INPUT:

SELECT COUNT(*) FROM Books;

OUTPUT:



10.Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the name of all the SILVER colored cars.

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

(iv) To display the customer name and the corresponding name of the cards hired by them. (b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

(iii) SELECT COUNT (*), Make FROM CARDEN;

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

CARDEN

Ccode	CarName	Make	Color	Capacity	Charges
501	A-star	Suzuki	RED	3	14
503	Indigo	Tata	SILVER	3	12
502	Innova	Toyota	WHITE	7	15
509	SX4	Suzuki	SILVER	4	14
510	C-Class	Mercedes	RED	4	35

CUSTOMER

CCode	Cname	Ccode (Car Code)
1001	HamantSahu	501
1002	Raj Lal	509
1003	Feroja Shah	503
1004	Ketan Dhal	502

SYNTAX:

CREATE TABLE CARDEN (Ccode INT PRIMARY KEY, CarName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(10), Capacity INT, Charges INT);

INSERT INTO CARDEN VALUES (501, 'A-star', 'Suzuki', 'RED', 3, 14);
INSERT INTO CARDEN VALUES (503, 'Indigo', 'Tata', 'SILVER', 3, 12);
INSERT INTO CARDEN VALUES (502, 'Innova', 'Toyota', 'WHITE', 7, 15);
INSERT INTO CARDEN VALUES (509, 'SX4', 'Suzuki', 'SILVER', 4, 14);
INSERT INTO CARDEN VALUES (510, 'C-Class', 'Mercedes', 'RED', 4, 35);
CREATE TABLE CUSTOMER (CCCode INT PRIMARY KEY, CName VARCHAR(30), Ccode INT);
INSERT INTO CUSTOMER VALUES (1001, 'HemantSahu', 501);
INSERT INTO CUSTOMER VALUES (1002, 'Raj Lal', 509);
INSERT INTO CUSTOMER VALUES (1003, 'Feroja Shah', 503);
INSERT INTO CUSTOMER VALUES (1004, 'Ketan Dhal', 502);

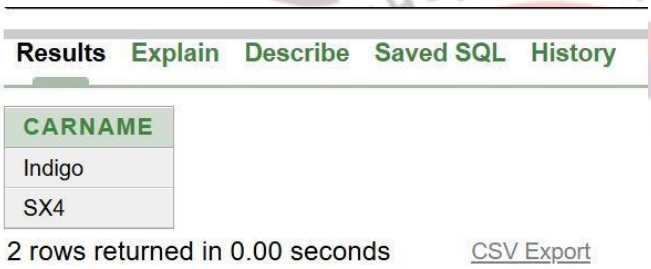
QUESTION:

(a) Write SQL commands for the following statements:
(i) To display the name of all the SILVER colored cars.

INPUT:

```
SELECT CarName FROM CARDEN WHERE Color = 'SILVER';
```

OUTPUT:



QUESTION:

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

INPUT:

```
SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;
```

OUTPUT:

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

INPUT:

SELECT MAX(Charges) FROM CARDEN;

OUTPUT:

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) To display the customer name and the corresponding name of the cards hired by them.

INPUT:

SELECT CUSTOMER.CName,CARDEN.CarName FROM CUSTOMER,CARDEN WHERE CUSTOMER.Ccode=CARDEN.Ccode; **OUTPUT:**

Results Explain Describe Saved SQL History

ORA-00904: "CUSTOMER"."CCODE": invalid identifier

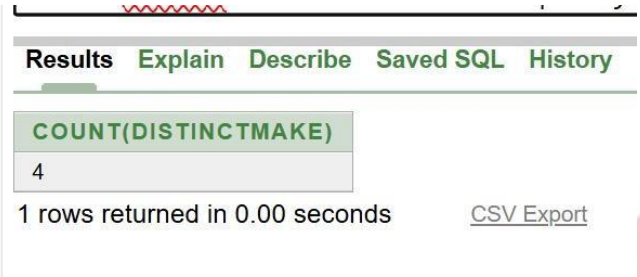
QUESTION:

(b) Give the output of the following SQL queries: (i)
SELECT COUNT (DISTINCT Make) FROM CARDEN;

INPUT:

SELECT COUNT(DISTINCT Make) FROM CARDEN;

OUTPUT:



QUESTION:

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

INPUT:

SELECT MAX(Charges),MIN(Charges) FROM CARDEN;

OUTPUT:



QUESTION:

(iii) SELECT COUNT (*), Make FROM CARDEN;

INPUT:

SELECT COUNT(*),Make FROM CARDEN GROUP BY Make;

OUTPUT:



Results | Explain | Describe | Saved SQL | History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds CSV Export

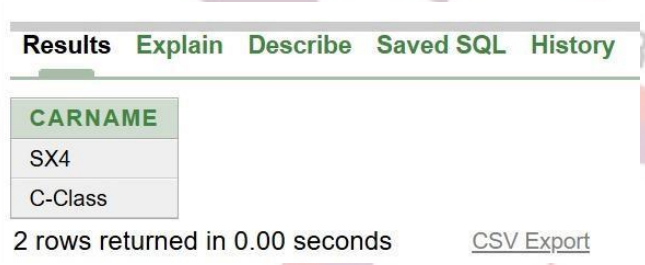
QUESTION:

(iv)SELECT CarName FROM CARDEN WHERE Capacity = 4;

(v) **INPUT:**

SELECT CarName FROM CARDEN WHERE Capacity=4;

OUTPUT:



Results | Explain | Describe | Saved SQL | History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

11.Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB2006' and '08-AUG-2009'.

(iv) TO add a new row in the EMPLOYEE table with the following data: 109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

EMPLOYEE

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-2003	13-JAN-1980
102	Ravi Chander	HEAD-IT	S02	12-FEB-2010	22-JUL-1987
103	John Ken	Receptionist	S03	24-JUN-2009	24-FEB-1983
105	NazarAmeen	GM	S02	11-AUG-2006	03-MAR-1984
108	PriyamSen	CEO	S01	29-DEC-2004	19-JAN-1982

SALGRADE

SGRADE	SALARY	HRA
S01	56000	18000
S02	32000	12000
S03	24000	8000

SYNTAX:

```
CREATE TABLE EMPLOYEE(ECODE INT PRIMARY KEY,NAME VARCHAR(30),DESIG VARCHAR(30),SGRADE VARCHAR(5),DOJ DATE,DOB DATE);

INSERT INTO EMPLOYEE VALUES(101,'Abdul Ahmad','EXECUTIVE','S03',DATE '2003-03-23',DATE '1980-01-13');
INSERT INTO EMPLOYEE VALUES(102,'Ravi Chander','HEAD-IT','S02',DATE '2010-02-12',DATE '1987-07-22');
INSERT INTO EMPLOYEE VALUES(103,'John Ken','RECEPTIONIST','S03',DATE '2009-06-24',DATE '1983-02-24');
INSERT INTO EMPLOYEE VALUES(105,'Nazar Ameen','GM','S02',DATE '2006-08-11',DATE '1984-03-03');
INSERT INTO EMPLOYEE VALUES(108,'Priyam Sen','CEO','S01',DATE '2004-12-29',DATE '1982-01-19');
CREATE TABLE SALGRADE(SGRADE VARCHAR(5) PRIMARY KEY,SALARY INT,HRA INT);
INSERT INTO SALGRADE VALUES('S01',56000,18000);
INSERT INTO SALGRADE VALUES('S02',32000,12000);
INSERT INTO SALGRADE VALUES('S03',24000,8000);
SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;
SELECT NAME,DESIG FROM EMPLOYEE WHERE SGRADE IN('S02','S03');
SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN DATE '2006-02-09' AND DATE '2009-08-08';
SELECT COUNT(SGRADE),SGRADE FROM EMPLOYEE GROUP BY SGRADE;
```

SELECT MIN(DOB),MAX(DOJ) FROM EMPLOYEE;
SELECT NAME,SALARY FROM EMPLOYEE E,SALGRADE S WHERE E.SGRADE=S.SGRADE AND E.ECODE<103;
SELECT SGRADE,SALARY+HRA FROM SALGRADE WHERE SGRADE='S02';
INSERT INTO EMPLOYEE VALUES(109,'Harish Roy','HEAD-IT','S02',DATE '2007-0909',DATE '1983-04-21');

QUESTION:

(a) Write SQL commands for the following statements:
(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

INPUT:

SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;

OUTPUT:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	RECEPTIONIST	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84
108	Priyam Sen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

INPUT:

SELECT NAME,DESIG FROM EMPLOYEE WHERE SGRADE IN('S02','S03');

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DESIG
Ravi Chander	HEAD-IT
Abdul Ahmad	EXECUTIVE
John Ken	RECEPTIONIST
Nazar Ameen	GM

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB2006' and '08-AUG-2009'.

INPUT:

SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN DATE '2006-02-09' AND DATE '2009-08-08';

OUTPUT:



Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	RECEPTIONIST	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.02 seconds CSV Export

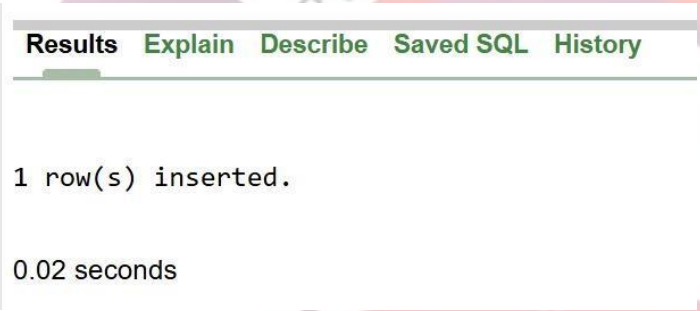
QUESTION:

(iv)TO add a new row in the EMPLOYEE table with the following data: 109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

INPUT:

INSERT INTO EMPLOYEE VALUES(109,'Harish Roy','HEAD-IT','S02',DATE '2007-0909',DATE '1983-04-21');

OUTPUT:



Results Explain Describe Saved SQL History

1 row(s) inserted.

0.02 seconds

QUESTION:

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

INPUT:

SELECT COUNT(SGRADE),SGRADE FROM EMPLOYEE GROUP BY SGRADE;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S02
2	S03
1	S01

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

INPUT:

SELECT MIN(DOB),MAX(DOJ) FROM EMPLOYEE;

OUTPUT:

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103; **INPUT:**

SELECT NAME,SALARY FROM EMPLOYEE E,SALGRADE S WHERE E.SGRADE=S.SGRADE AND E.ECODE<103;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.02 seconds CSV Export

QUESTION:

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

INPUT:

SELECT SGRADE,SALARY+HRA FROM SALGRADE WHERE SGRADE='S02';

OUTPUT:



12.Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

(iv) To add a new row with the following: 19, 'Daya Kishore', 'Operator', 'P003', '19-JUN2008', '11-JUL-1984'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL ='P003';

WORKER

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
11	RadheShyam	Supervisor	P001	13-SEP-2004	23-AUG-1981
12	ChanderNath	Operator	P003	22-FEB-2010	12-JUL-1987
13	Fizza	Operator	P003	14-JUN-2009	14-OCT-1983
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-2006	13-MAR-1984
18	Sanya	Clerk	P002	19-DEC-2005	09-JUN-1983

PAYLAVEL

PLEVEL	PAY	ALLOWANCE
P001	26000	12000
P002	22000	10000
P003	12000	6000

SYNTAX:

CREATE TABLE WORKER(ECODE INT,NAME VARCHAR(20),DESIGN VARCHAR(20),PLEVEL VARCHAR(5),DOJ DATE,DOB DATE);
INSERT INTO WORKER VALUES(11,'RadheShyam','Supervisor','P001','13-SEP-2004','23-AUG-1981');
INSERT INTO WORKER VALUES(12,'ChanderNath','Operator','P003','22-FEB-2010','12-JUL1987');
INSERT INTO WORKER VALUES(13,'Fizza','Operator','P003','14-JUN-2009','14-OCT-1983');
INSERT INTO WORKER VALUES(15,'Ahmeen Ahmad','Mechanic','P002','21-AUG-2006','13MAR-1984');
INSERT INTO WORKER VALUES(18,'Sanya','Clerk','P002','19-DEC-2005','09-JUN-1983');
CREATE TABLE PAYLEVEL(PLEVEL VARCHAR(5),PAY INT,ALLOWANCE INT);
INSERT INTO PAYLEVEL VALUES('P001',26000,12000);
INSERT INTO PAYLEVEL VALUES('P002',22000,10000);
INSERT INTO PAYLEVEL VALUES('P003',12000,6000);

QUESTION:

(a) Write SQL commands for the following statements:
(i) To display the detail of all WORKER in descending order of DOB.

INPUT:

SELECT * FROM WORKER ORDER BY DOB DESC;

OUTPUT:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.01 seconds CSV Export

QUESTION:

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

INPUT:

SELECT NAME,DESIGN FROM WORKER WHERE PLEVEL='P001' OR PLEVEL='P002';

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.02 seconds CSV Export

QUESTION:

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

INPUT:

SELECT * FROM WORKER WHERE DOB BETWEEN '19-JAN-1984' AND '18-JAN-1987';

OUTPUT:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv)To add a new row with the following: 19, 'Daya Kishore', 'Operator', 'P003', '19-JUN2008', '11-JUL-1984'.

INPUT:

INSERT INTO WORKER VALUES(19,'Daya Kishore','Operator','P003','19-JUN-2008','11JUL-1984');

OUTPUT:



QUESTION:

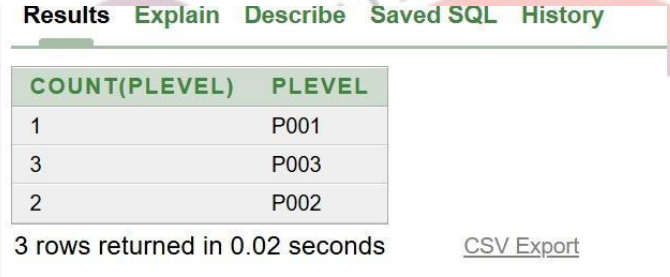
(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

INPUT:

SELECT COUNT(PLEVEL),PLEVEL FROM WORKER GROUP BY PLEVEL;

OUTPUT:



QUESTION:

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

INPUT:

SELECT MAX(DOB),MIN(DOJ) FROM WORKER; **OUTPUT:**



QUESTION:

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

INPUT:

SELECT NAME,PAY FROM WORKER W,PAYLEVEL P WHERE W.PLEVEL=P.PLEVEL AND W.ECODE<13;

OUTPUT:



Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL ='P003';

INPUT:

SELECT PLEVEL,PAY+ALLOWANCE FROM PAYLEVEL WHERE PLEVEL='P003';

OUTPUT:



Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.01 seconds CSV Export

13.Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy. (ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

(iv) SELECT MAX(LastBuy) FROM STORE;

STORE

ItemNo	Item	Scode	Qty	Rate	LastBuy
2005	Sharpener Classic	23	60	8	31-JUN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

SUPPLIERS

Scode	Sname
21	Premium Stationers
23	Soft Plastics
22	Tetra Supply

SYNTAX:

```
DROP TABLE STORE CASCADE CONSTRAINTS;
DROP TABLE SUPPLIERS CASCADE CONSTRAINTS;
CREATE TABLE STORE(ItemNo INT PRIMARY KEY,Item VARCHAR(30),Scode INT,Qty INT,Rate INT,LastBuy DATE);
INSERT INTO STORE VALUES(2001,'Eraser Small',22,220,6,TO_DATE('19-JAN-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2002,'Gel Pen Premium',21,150,12,TO_DATE('24-FEB-2010','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2003,'Ball Pen 0.25',22,50,25,TO_DATE('01-FEB-2010','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2004,'Eraser Big',22,110,8,TO_DATE('02-DEC-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2005,'Sharpener Classic',23,60,8,TO_DATE('30-JUN-
```

2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2006,'Gel Pen Classic',21,250,20,TO_DATE('11-MAR-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2009,'Ball Pen 0.5',21,180,18,TO_DATE('03-NOV-2009','DD-MON-YYYY'));
CREATE TABLE SUPPLIERS(Scode INT PRIMARY KEY,Sname VARCHAR(30));
INSERT INTO SUPPLIERS VALUES(21,'Premium Stationers');
INSERT INTO SUPPLIERS VALUES(22,'Tetra Supply');
INSERT INTO SUPPLIERS VALUES(23,'Soft Plastics');

QUESTION:

(a) Write SQL commands for the following statements:
(i) To display details of all the items in the Store table in ascending order of LastBuy.

INPUT:

SELECT * FROM STORE ORDER BY LastBuy ASC;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2006	Gel Pen Classic	21	250	20	11-MAR-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2003	Ball Pen 0.25	22	50	25	01-FEB-10
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

INPUT:

SELECT ItemNo,Item FROM STORE WHERE Rate>15;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.01 seconds CSV Export

QUESTION:

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

INPUT:

SELECT * FROM STORE WHERE Scode=22 OR Qty>110;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2003	Ball Pen 0.25	22	50	25	01-FEB-10
2004	Eraser Big	22	110	8	02-DEC-09
2006	Gel Pen Classic	21	250	20	11-MAR-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

INPUT:

SELECT Scode,MIN(Rate) FROM STORE GROUP BY Scode;

OUTPUT:

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds CSV Export

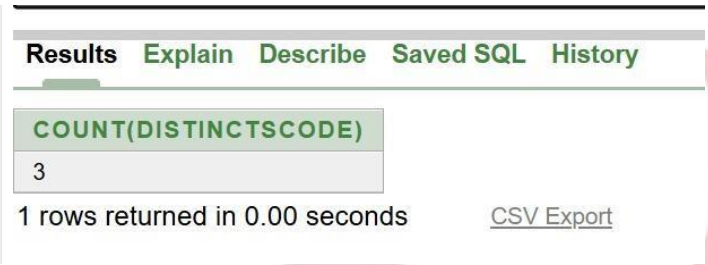
QUESTION:

(b) Give the output of the following SQL queries: (i)
SELECT COUNT (DISTINCT Scode) FROM STORE;

INPUT:

SELECT COUNT(DISTINCT Scode) FROM STORE;

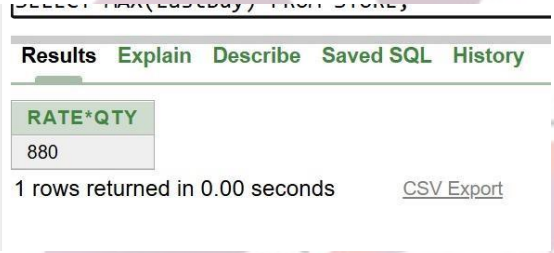
OUTPUT:



QUESTION:

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004; **INPUT:**

SELECT Rate*Qty FROM STORE WHERE ItemNo=2004; **OUTPUT:**



QUESTION:

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

INPUT:

SELECT Item,Sname FROM STORE S,SUPPLIERS P WHERE S.Scode=P.Scode AND S.ItemNo=2006;

OUTPUT:



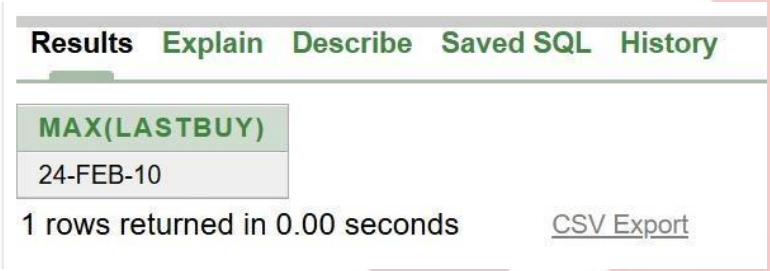
QUESTION:

(iv) SELECT MAX(LastBuy) FROM STORE;

INPUT:

SELECT MAX(LastBuy) FROM STORE;

OUTPUT:



14.Consider the following table GARMENT and FABRIC, Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

(ii) To display the details of all the GARMENT, which have READYDATE in between 08DEC-07 and 16-JUN-08 (inclusive if both the dates).

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

(vii) SELECT MAX (FCODE) FROM FABRIC;

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

GARMENT

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10023	Pencil Skirt	1150	F03	19-DEC-08
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10024	Baby Top	750	F03	07-APR-07
10090	Tulip Skirt	850	F02	31-MAR-07
10019	Evening Gown	850	F03	06-JUN-08
10009	Informal Pant	1500	F02	20-OCT-08
10007	Formal Pant	1350	F01	09-MAR-08
10020	Frock	850	F04	09-SEP-07
10089	Slacks	750	F03	20-OCT-08

FABRIC

FCODE	TYPE
F04	Polyester
F02	Cotton
F03	Silk
F01	Terelene

SYNTAX:

CREATE TABLE GARMENT(GCODE INT,DESCRIPTION VARCHAR(30),PRICE INT,FCODE VARCHAR(5),READYDATE DATE);

INSERT INTO GARMENT VALUES(10023,'Pencil Skirt',1150,'F03','19-DEC-08');

INSERT INTO GARMENT VALUES(10001,'Formal Shirt',1250,'F01','12-JAN-08');

INSERT INTO GARMENT VALUES(10012,'Informal Shirt',1550,'F02','06-JUN-08');

INSERT INTO GARMENT VALUES(10024,'Baby Top',750,'F03','07-APR-07');

INSERT INTO GARMENT VALUES(10090,'Tulip Skirt',850,'F02','31-MAR-07');

INSERT INTO GARMENT VALUES(10019,'Evening Gown',850,'F03','06-JUN-08');

INSERT INTO GARMENT VALUES(10009,'Informal Pant',1500,'F02','20-OCT-08');

INSERT INTO GARMENT VALUES(10007,'Formal Pant',1350,'F01','09-MAR-08');

INSERT INTO GARMENT VALUES(10020,'Frock',850,'F04','09-SEP-07');

INSERT INTO GARMENT VALUES(10089,'Slacks',750,'F03','20-OCT-08');

CREATE TABLE FABRIC(FCODE VARCHAR(5),TYPE VARCHAR(20));

INSERT INTO FABRIC VALUES('F04','Polyester');

INSERT INTO FABRIC VALUES('F02','Cotton');

INSERT INTO FABRIC VALUES('F03','Silk'); INSERT

INTO FABRIC VALUES('F01','Terelene');

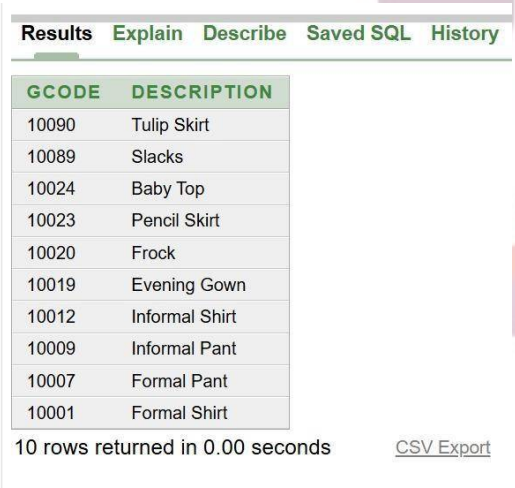
QUESTION:

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

INPUT:

SELECT GCODE,DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

OUTPUT:



Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display the details of all the GARMENT, which have READYDATE in between 08DEC-07 and 16-JUN-08 (inclusive if both the dates).

INPUT:

SELECT * FROM GARMENT WHERE READYDATE BETWEEN '08-DEC-07' AND '16-JUN-08';

OUTPUT:

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10012	Informal Shirt	1550	F02	06-JUN-08
10001	Formal Shirt	1250	F01	12-JAN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

INPUT:

SELECT AVG(PRICE) FROM GARMENT WHERE FCODE='F03';

OUTPUT:

Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

INPUT:

SELECT FCODE,MAX(PRICE),MIN(PRICE) FROM GARMENT GROUP BY FCODE;

OUTPUT:

Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.00 seconds CSV Export

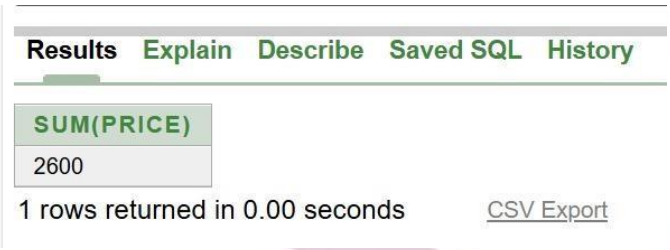
QUESTION:

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = ‘F01’;

INPUT:

SELECT SUM(PRICE) FROM GARMENT WHERE FCODE='F01';

OUTPUT:



Results Explain Describe Saved SQL History

SUM(PRICE)
2600

1 rows returned in 0.00 seconds CSV Export

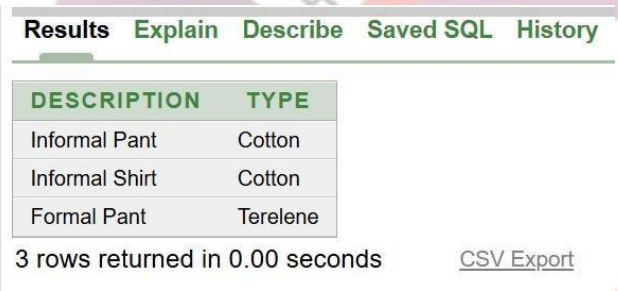
QUESTION:

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

INPUT:

SELECT DESCRIPTION,TYPE FROM GARMENT G,FABRIC F WHERE G.FCODE=F.FCODE AND G.PRICE>=1260;

OUTPUT:



Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(vii) SELECT MAX (FCODE) FROM FABRIC;

INPUT:

SELECT MAX(FCODE) FROM FABRIC;

OUTPUT:

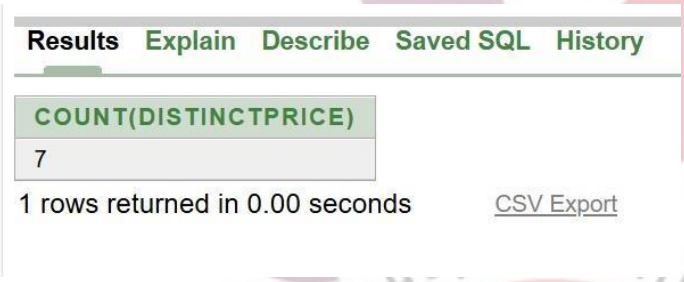


QUESTION:

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

INPUT:

SELECT COUNT(DISTINCT PRICE) FROM GARMENT; **OUTPUT:**



15.Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables FURNITURE AND ARRIVALS. (a) To show all information about the baby cots from the FURNITURE table.

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table. (f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30.

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

FURNITURE

No.	Item Name	Type	Date of Stock	Price	Discount
1	White lotus	Double Bed	23/02/02	30000	25
2	Pink feather	Baby cot	20/01/02	7000	20
3	Dolphin	Baby cot	19/02/02	9500	20
4	Decent	Office Table	01/01/02	25000	30
5	Comfort zone	Double Bed	12/01/02	25000	25
6	Donald	Baby cot	24/02/02	6500	15
7	Royal Finish	Office Table	20/02/02	18000	30
8	Royal tiger	Sofa	22/02/02	31000	30
9	Econo sitting	Sofa	13/12/01	9500	25
10	Eating paradise	Dining Table	19/02/02	11500	25

ARRIVALS

No.	Item Name	Type	Date of Stock	Price	Discount
11	Wood Comfort	Double Bed	23/03/03	25000	25
12	Old Fox	Sofa	20/02/03	17000	20
13	Micky	Baby cot	21/02/02	7500	15

SYNTAX:

```
CREATE TABLE FURNITURE(NO INT PRIMARY KEY,ITEMNAME
VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT
INT);
INSERT INTO FURNITURE VALUES(1,'White lotus','Double Bed',DATE '2002-
0223',30000,25);
INSERT INTO FURNITURE VALUES(2,'Pink feather','Baby cot',DATE '2002-01-20',7000,20);
INSERT INTO FURNITURE VALUES(3,'Dolphin','Baby cot',DATE '2002-02-19',9500,20);
INSERT INTO FURNITURE VALUES(4,'Decent','Office Table',DATE '2002-01-01',25000,30);
INSERT INTO FURNITURE VALUES(5,'Comfort zone','Double Bed',DATE '2002-
0112',25000,25);
INSERT INTO FURNITURE VALUES(6,'Donald','Baby cot',DATE '2002-02-24',6500,15);
INSERT INTO FURNITURE VALUES(7,'Royal Finish','Office Table',DATE '2002-
0220',18000,30);
INSERT INTO FURNITURE VALUES(8,'Royal tiger','Sofa',DATE '2002-02-22',31000,30);
INSERT INTO FURNITURE VALUES(9,'Econo sitting','Sofa',DATE '2001-12-13',9500,25);
INSERT INTO FURNITURE VALUES(10,'Eating paradise','Dining Table',DATE '2002-
0219',11500,25);
CREATE TABLE ARRIVALS(NO INT PRIMARY KEY,ITEMNAME VARCHAR(30),TYPE
VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);
```

INSERT INTO ARRIVALS VALUES(11,'Wood Comfort','Double Bed',DATE '2003-0323',25000,25);
INSERT INTO ARRIVALS VALUES(12,'Old Fox','Sofa',DATE '2003-02-20',17000,20);
INSERT INTO ARRIVALS VALUES(13,'Micky','Baby cot',DATE '2002-02-21',7500,15);

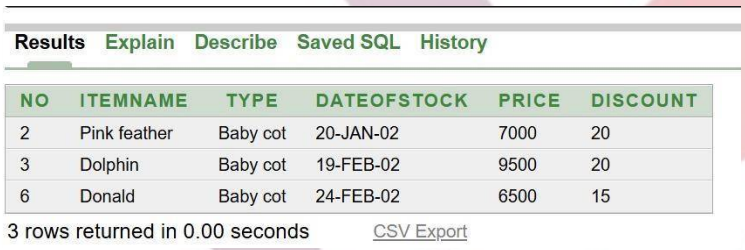
QUESTION:

(a) To show all information about the baby cots from the FURNITURE table.

INPUT:

SELECT * FROM FURNITURE WHERE TYPE='Baby cot';

OUTPUT:



Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.00 seconds CSV Export

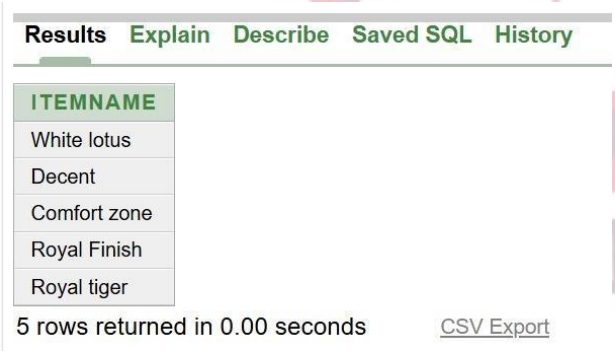
QUESTION:

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

INPUT:

SELECT ITEMNAME FROM FURNITURE WHERE PRICE>15000;

OUTPUT:



Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

INPUT:

SELECT ITEMNAME,TYPE FROM FURNITURE WHERE DATEOFSTOCK<DATE '2002-01-01' ORDER BY ITEMNAME DESC;

OUTPUT:



Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Econo sitting	Sofa

1 rows returned in 0.01 seconds CSV Export

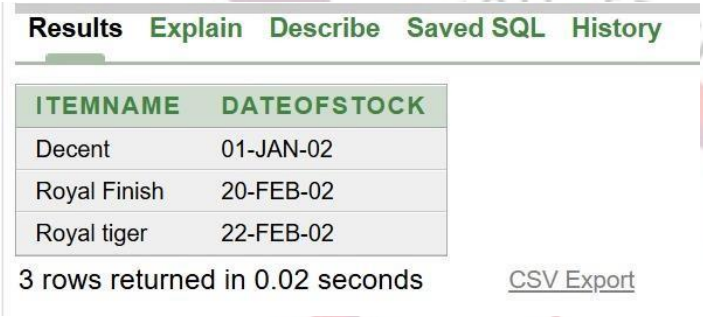
QUESTION:

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

INPUT:

SELECT ITEMNAME,DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT>25;

OUTPUT:



Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.02 seconds CSV Export

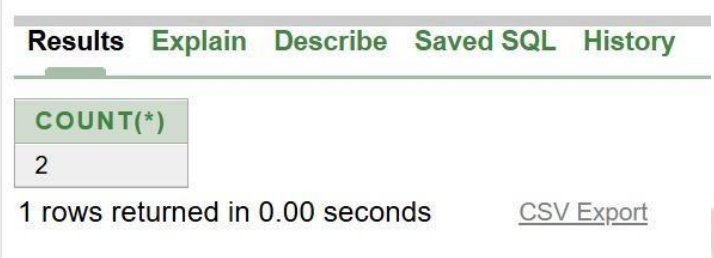
QUESTION:

(e) To count the number of items, whose TYPE is “Sofa” from FURNITURE table.

INPUT:

SELECT COUNT(*) FROM FURNITURE WHERE TYPE='Sofa';

OUTPUT:



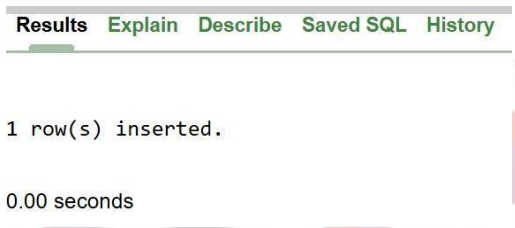
QUESTION:

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30.

INPUT:

INSERT INTO ARRIVALS VALUES(14,'Velvet touch','Double bed',DATE '2003-0325',25000,30);

OUTPUT:



QUESTION:

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

INPUT:

SELECT COUNT(DISTINCT TYPE) FROM FURNITURE;

OUTPUT:



QUESTION:

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

INPUT:

SELECT MAX(FURNITURE.DISCOUNT) FROM FURNITURE,ARRIVALS;

OUTPUT:



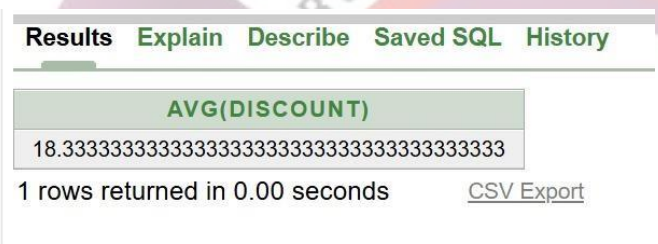
QUESTION:

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = “Baby cot”;

INPUT:

SELECT AVG(DISCOUNT) FROM FURNITURE WHERE TYPE='Baby cot';

OUTPUT:



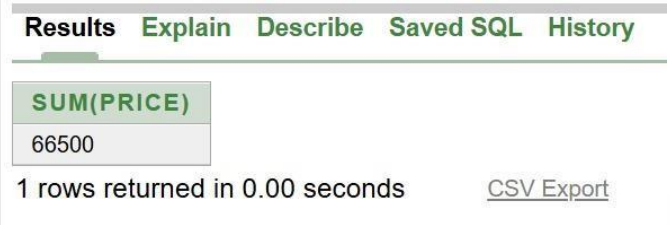
QUESTION:

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

INPUT:

SELECT SUM(PRICE) FROM FURNITURE WHERE DATEOFSTOCK<DATE '2002-02-12';

OUTPUT:



16.Given the following COVID19 relation, write SQL commands for questions (a) to (f). a) Display all details of patients from Tamil Nadu.

b) Display the names of patients whose Status is Recovered. c) Display PatientName and Age in ascending order of Age.

d) Display the names of patients who are Vaccinated = 'Yes'.

e) Count the number of patients whose status is Active.

f) Insert the following record into the COVID19 table: 108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

(g) Write the output of the following SQL commands. a.

SELECT COUNT*() FROM COVID19;*

b. SELECT MAX(Age) FROM COVID19;

c. SELECT COUNT() FROM COVID19 WHERE Vaccinated='Yes';

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

PatientID	PatientName	State	Age	Status	Vaccinated
101	Arun	Tamil Nadu	35	Recovered	Yes
102	Priya	Kerala	42	Active	Yes
103	Karthik	Karnataka	28	Recovered	No
104	Divya	Tamil Nadu	31	Active	Yes
105	Rahul	Andhra Pradesh	55	Deceased	No
106	Meena	Kerala	24	Recovered	Yes
107	Suresh	Tamil Nadu	48	Active	No

SYNTAX:

```
CREATE TABLE COVID19 (PatientID INT PRIMARY KEY, PatientName VARCHAR(20),
State VARCHAR(30), Age INT, Status VARCHAR(15), Vaccinated VARCHAR(5));
INSERT INTO COVID19 VALUES (101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');
INSERT INTO COVID19 VALUES (102, 'Priya', 'Kerala', 42, 'Active', 'Yes');
INSERT INTO COVID19 VALUES (103, 'Karthik', 'Karnataka', 28, 'Recovered', 'No');
INSERT INTO COVID19 VALUES (104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');
INSERT INTO COVID19 VALUES (105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');
INSERT INTO COVID19 VALUES (106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');
```

INSERT INTO COVID19 VALUES (107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');

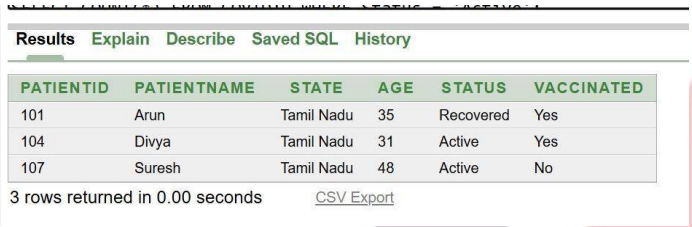
QUESTION:

a) Display all details of patients from Tamil Nadu.

INPUT:

SELECT * FROM COVID19 WHERE State = 'Tamil Nadu';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.00 seconds CSV Export

QUESTION:

b) Display the names of patients whose Status is Recovered.

INPUT:

SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.00 seconds CSV Export

QUESTION:

c) Display PatientName and Age in ascending order of Age.

INPUT:

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds CSV Export

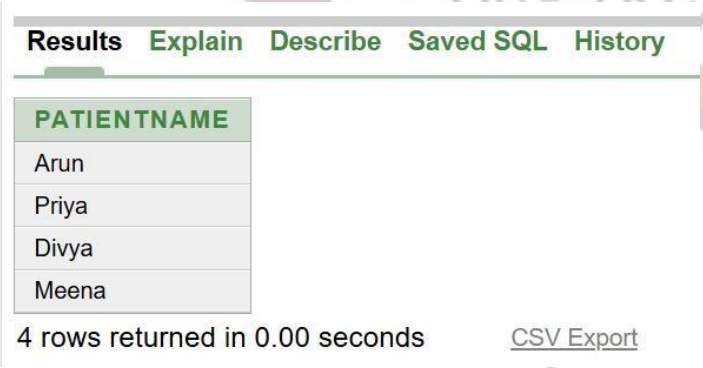
QUESTION:

d) Display the names of patients who are Vaccinated = 'Yes'.

INPUT:

```
SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';
```

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.00 seconds CSV Export

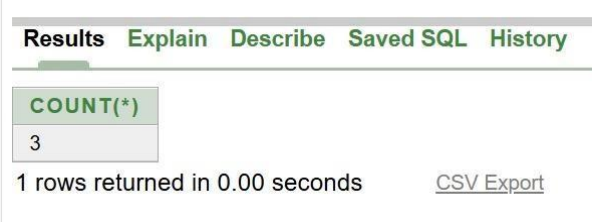
QUESTION:

e) Count the number of patients whose status is Active.

INPUT:

```
SELECT COUNT(*) FROM COVID19 WHERE Status = 'Active';
```

OUTPUT:



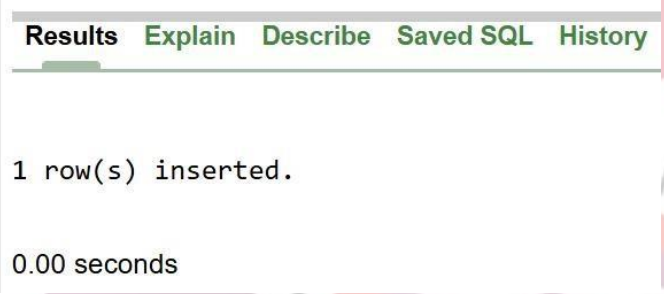
QUESTION:

f) Insert the following record into the COVID19 table: 108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INPUT:

INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

OUTPUT:



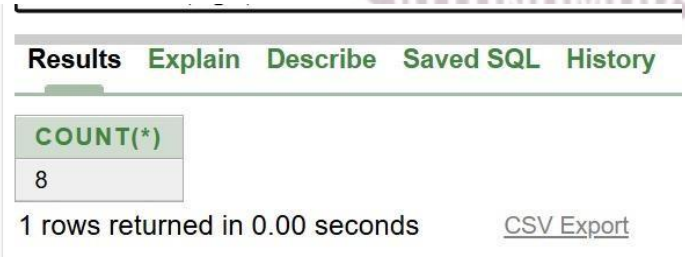
QUESTION:

(h) Write the output of the following SQL commands. a. SELECT COUNT*() FROM COVID19;*

INPUT:

SELECT COUNT(*) FROM COVID19;

OUTPUT:



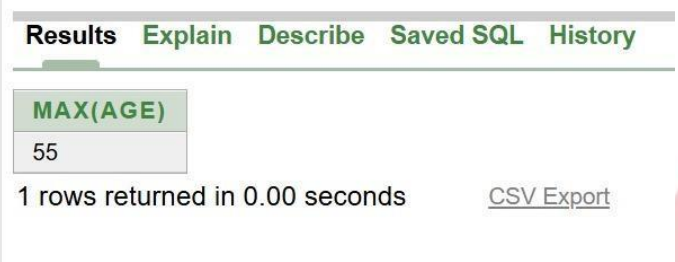
QUESTION:

b. *SELECT MAX(Age) FROM COVID19;*

INPUT:

SELECT MAX(Age) FROM COVID19;

OUTPUT:



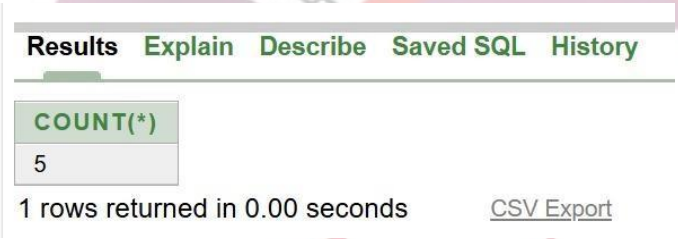
QUESTION:

SELECT COUNT() FROM COVID19 WHERE Vaccinated='Yes';

INPUT:

SELECT COUNT(*) FROM COVID19 WHERE Vaccinated = 'Yes';

OUTPUT:



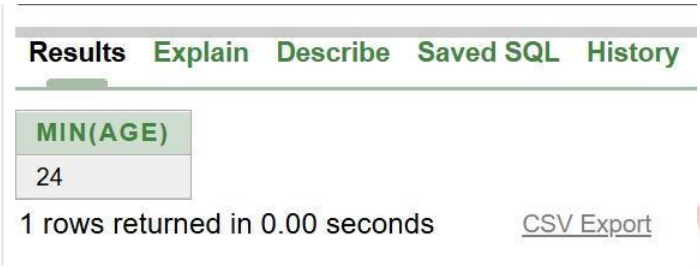
QUESTION:

c. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

INPUT:

SELECT MIN(Age) FROM COVID19 WHERE Status = 'Recovered';

OUTPUT:



17.Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).

a) Display all houses located in Chennai.

b) Display the Location and Price of houses having 3 Bedrooms. c) Display Location, AreaSqft, and Price in descending order of Price.

d) Display the details of houses costing more than ₹70,00,000.

e) Count the number of Apartments.

f) Insert the following record: 208, 'Vellore', 3, 1700, 7300000, 'Villa' (g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

HouseID	Location	Bedrooms	AreaSqft	Price	HouseType
201	Chennai	2	1100	5200000	Apartment
202	Coimbatore	3	1450	6800000	Villa
203	Madurai	2	980	4100000	Apartment
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
206	Salem	2	1200	5000000	Apartment
207	Erode	3	1800	7600000	Independent

SYNTAX:

CREATE TABLE HOUSEPRICE (HouseID INT PRIMARY KEY, Location VARCHAR(20), Bedrooms INT, AreaSqft INT, Price INT, HouseType VARCHAR(20));
INSERT INTO HOUSEPRICE VALUES (201, 'Chennai', 2, 1100, 5200000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES (202, 'Coimbatore', 3, 1450, 6800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES (203, 'Madurai', 2, 980, 4100000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES (204, 'Chennai', 4, 2200, 9800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES (205, 'Trichy', 3, 1650, 7200000, 'Independent');
INSERT INTO HOUSEPRICE VALUES (206, 'Salem', 2, 1200, 5000000, 'Apartment'); INSERT INTO HOUSEPRICE VALUES (207, 'Erode', 3, 1800, 7600000, 'Independent');

QUESTION:

a) Display all houses located in Chennai.

INPUT:

SELECT * FROM HOUSEPRICE WHERE Location = 'Chennai';

OUTPUT:



Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.01 seconds CSV Export

QUESTION:

b) Display the Location and Price of houses having 3 Bedrooms.

INPUT:

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

OUTPUT:



Results Explain Describe Saved SQL History

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.00 seconds CSV Export

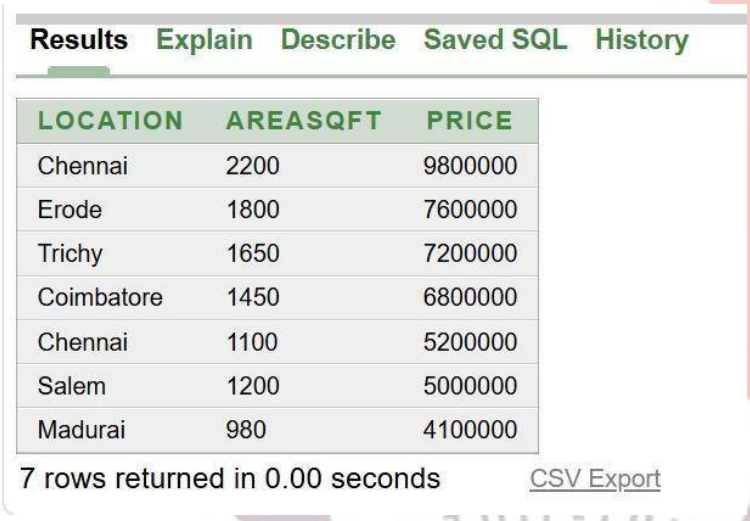
QUESTION:

c) Display Location, AreaSqft, and Price in descending order of Price.

INPUT:

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

OUTPUT:



Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1650	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.00 seconds CSV Export

QUESTION:

d) Display the details of houses costing more than ₹70,00,000.

INPUT:

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

OUTPUT:



Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.00 seconds CSV Export

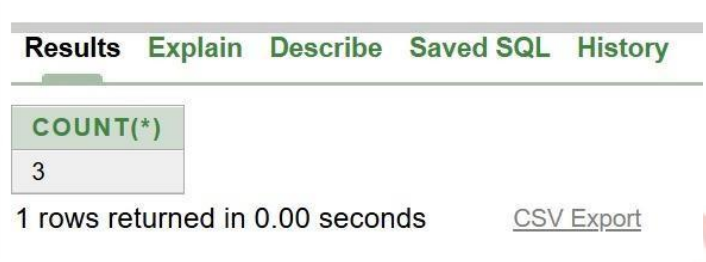
QUESTION:

e) Count the number of Apartments.

INPUT:

SELECT COUNT(*) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

OUTPUT:



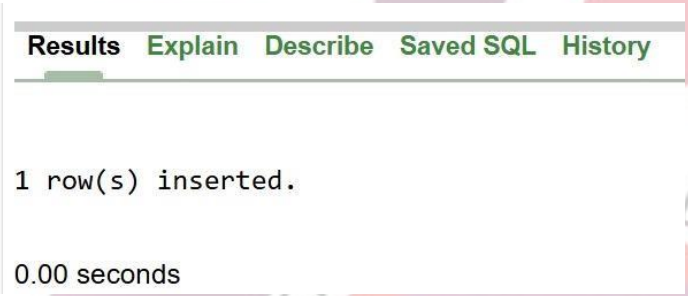
QUESTION:

f) Insert the following record: 208, 'Vellore', 3, 1700, 7300000, 'Villa'

INPUT:

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

OUTPUT:



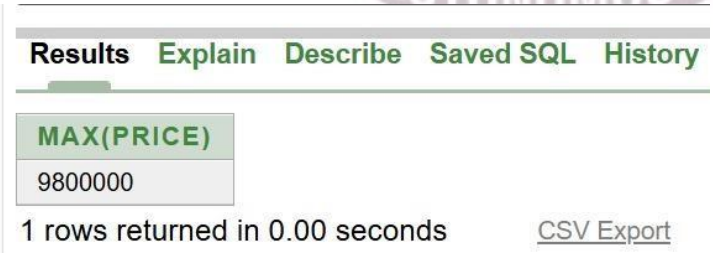
QUESTION:

(g) Write the output of the following SQL commands.
(i) SELECT MAX(Price)FROM HOUSEPRICE;

INPUT:

SELECT MAX(Price) FROM HOUSEPRICE;

OUTPUT:



QUESTION:

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

INPUT:

SELECT MIN(AreaSqft) FROM HOUSEPRICE;

OUTPUT:



QUESTION:

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

INPUT:

SELECT AVG(Price) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

OUTPUT:



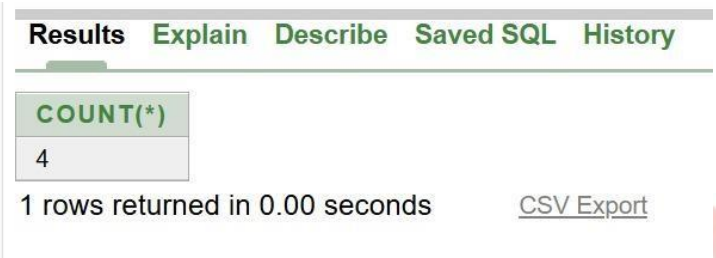
QUESTION:

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

INPUT:

SELECT COUNT(*) FROM HOUSEPRICE WHERE Bedrooms = 3;

OUTPUT:



18.Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).

a) Display all patients diagnosed with Breast Cancer.
b) Display the names of patients whose Stage is III.
c) Display PatientName and Age in ascending order of Age.
d) Display the names of patients having SurvivalYears greater than 5.
e) Count the number of patients diagnosed with Lung Cancer.
f) Insert the following record: 308, 'Anand', 'Leukemia', 50, 'II', 7 (g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;
(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;
(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';
(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

PatientID	PatientName	CancerType	Age	Stage	SurvivalYears
301	Kavya	Breast	45	II	8
302	Arun	Lung	60	III	4
303	Divya	Breast	38	I	10
304	Hari	Colon	52	II	6
305	Meena	Lung	57	IV	2
306	Ravi	Prostate	64	III	5
307	Priya	Breast	41	II	9

SYNTAX:

CREATE TABLE CANCERDATA (PatientID INT PRIMARY KEY, PatientName VARCHAR(20), CancerType VARCHAR(20), Age INT, Stage VARCHAR(5), SurvivalYears INT);
INSERT INTO CANCERDATA VALUES (301, 'Kavya', 'Breast', 45, 'II', 8); INSERT INTO CANCERDATA VALUES (302, 'Arun', 'Lung', 60, 'III', 4);
INSERT INTO CANCERDATA VALUES (303, 'Divya', 'Breast', 38, 'I', 10);
INSERT INTO CANCERDATA VALUES (304, 'Hari', 'Colon', 52, 'II', 6);

INSERT INTO CANCERDATA VALUES (305, 'Meena', 'Lung', 57, 'IV', 2);
INSERT INTO CANCERDATA VALUES (306, 'Ravi', 'Prostate', 64, 'III', 5);
INSERT INTO CANCERDATA VALUES (307, 'Priya', 'Breast', 41, 'II', 9);

QUESTION:

a) Display all patients diagnosed with Breast Cancer.

INPUT:

SELECT * FROM CANCERDATA WHERE CancerType='Breast';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.00 seconds CSV Export

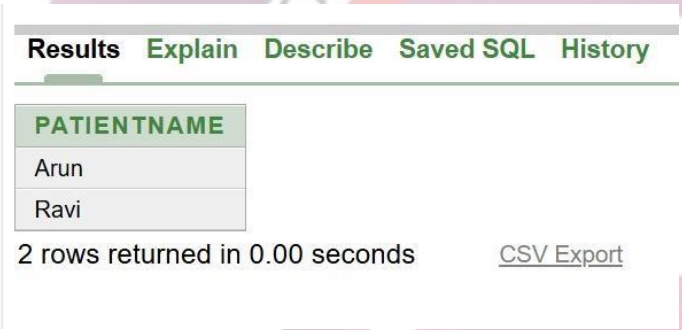
QUESTION:

b) Display the names of patients whose Stage is III.

INPUT:

SELECT PatientName FROM CANCERDATA WHERE Stage='III';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.00 seconds CSV Export

QUESTION:

c) Display PatientName and Age in ascending order of Age.

INPUT:

SELECT PatientName,Age FROM CANCERDATA ORDER BY Age ASC;

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

7 rows returned in 0.00 seconds CSV Export

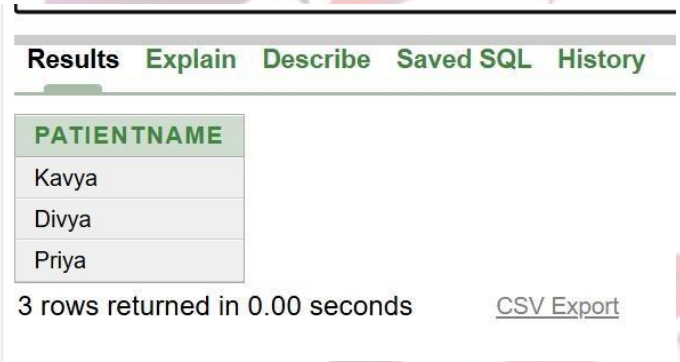
QUESTION:

d) Display the names of patients having SurvivalYears greater than 5.

INPUT:

```
SELECT PatientName FROM CANCERDATA WHERE SurvivalYears>6;
```

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Priya

3 rows returned in 0.00 seconds CSV Export

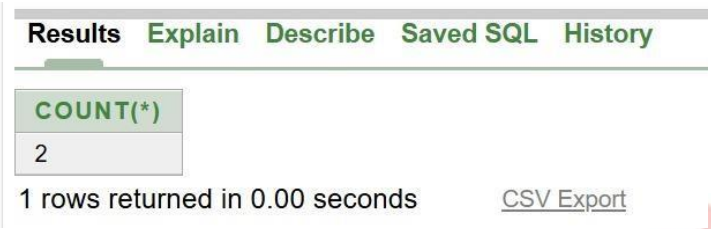
QUESTION:

e) Count the number of patients diagnosed with Lung Cancer.

INPUT:

```
SELECT COUNT(*) FROM CANCERDATA WHERE CancerType='Lung';
```

OUTPUT:



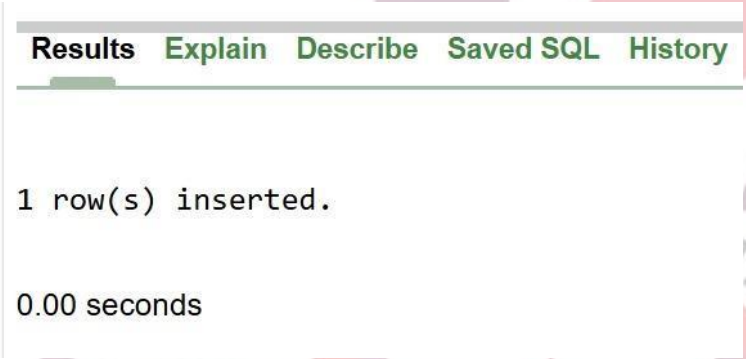
QUESTION:

f) Insert the following record: 308, 'Anand', 'Leukemia', 50, 'II', 7

INPUT:

```
INSERT INTO CANCERDATA VALUES (308, 'Anand', 'Leukemia', 50, 'II', 7);
```

OUTPUT:



QUESTION:

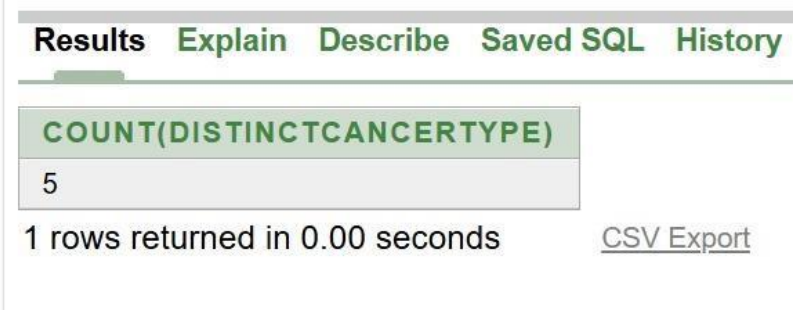
(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

INPUT:

```
SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;
```

OUTPUT:



QUESTION:

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

INPUT:

SELECT MAX(SurvivalYears) FROM CANCERDATA;

OUTPUT:



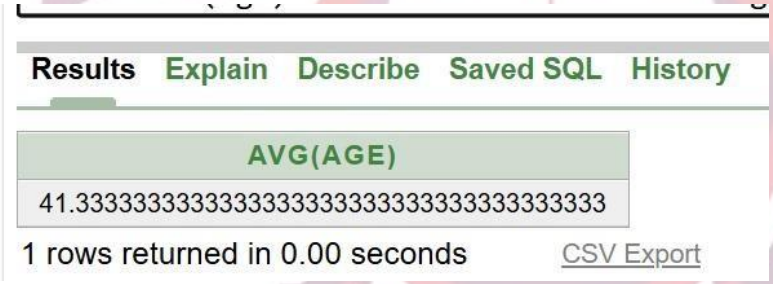
QUESTION:

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

INPUT:

SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

OUTPUT:



QUESTION:

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

INPUT:

SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

OUTPUT:



19.Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

(a) To display the details of all the items in ascending order of item names (i.e. INAME). (b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

(c) To display the number of items, which are traded by each trader. The expected output of this query should be T01 2 T02 2 T03 1

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

(f) To display the name of the companies and the bane of the items in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

ITEMS

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

TRADERS

TCODE	TNAME	CITY
T01	ELECTRONIC SALES	MUMBAI
T03	BUSY STORE CORP	DELHI
T02	DISP HOUSE INC	CHENNAI

SYNTAX:

CREATE TABLE ITEMS (CODE INT PRIMARY KEY, INAME VARCHAR(30), QTY INT, PRICE INT, COMPANY VARCHAR(20), TCODE VARCHAR(5));
INSERT INTO ITEMS VALUES (1001, 'DIGITAL PAD12i', 120, 11000, 'XENITA', 'T01');
INSERT INTO ITEMS VALUES (1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T03');
INSERT INTO ITEMS VALUES (1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T02');
INSERT INTO ITEMS VALUES (1003, 'DIGITAL CAMERA 12X', 160, 8000, 'DIGICLICK', 'T01');
INSERT INTO ITEMS VALUES (1005, 'PEN DRIVE 32 GB', 600, 1200, 'STOREHOME', 'T02');
CREATE TABLE TRADERS (TCODE VARCHAR(5) PRIMARY KEY, TNAME VARCHAR(30), CITY VARCHAR(20));
INSERT INTO TRADERS VALUES ('T01', 'ELECTRONIC SALES', 'MUMBAI');
INSERT INTO TRADERS VALUES ('T03', 'BUSY STORE CORP', 'DELHI'); INSERT INTO TRADERS VALUES ('T02', 'DISP HOUSE INC', 'CHENNAI');

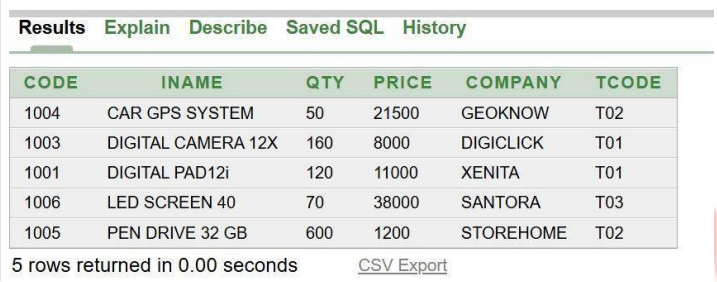
QUESTION:

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

INPUT:

SELECT * FROM ITEMS ORDER BY INAME ASC;

OUTPUT:



Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T02
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T01
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T03
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T02

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

INPUT:

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

OUTPUT:



Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds CSV Export

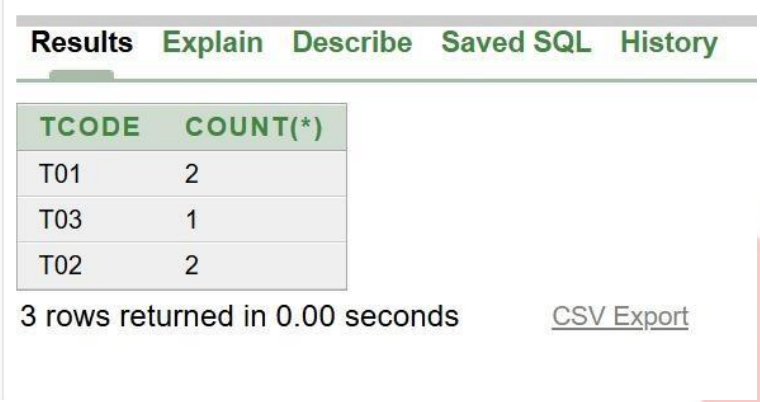
QUESTION:

(c) To display the number of items, which are traded by each trader. The expected output of this query should be T01 2 T02 2 T03 1

INPUT:

SELECT TCODE, COUNT(*) FROM ITEMS GROUP BY TCODE;

OUTPUT:



Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

INPUT:

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY>150;

OUTPUT:



Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

INPUT:

SELECT TNAME FROM TRADERS WHERE CITY='DELHI' OR CITY='MUMBAI';

OUTPUT:



Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

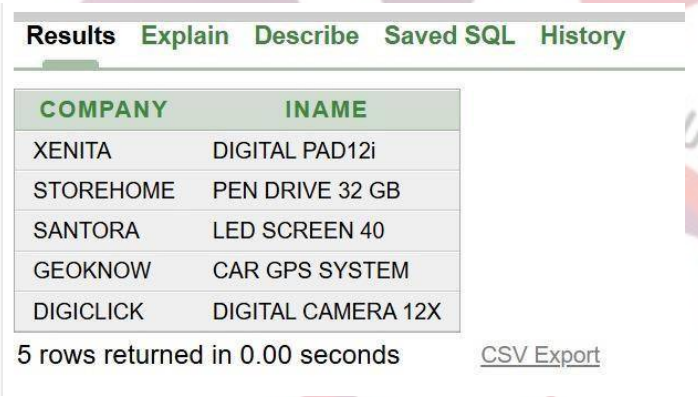
QUESTION:

(f) To display the name of the companies and the bane of the items in descending order of company names.

INPUT:

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

OUTPUT:



Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

INPUT:

SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

OUTPUT:



Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

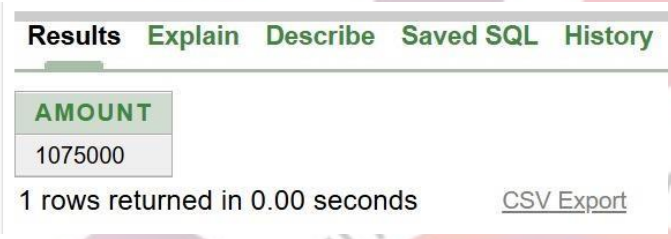
QUESTION:

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

INPUT:

SELECT PRICE*QTY AS AMOUNT FROM ITEMS WHERE CODE=1004;

OUTPUT:



Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

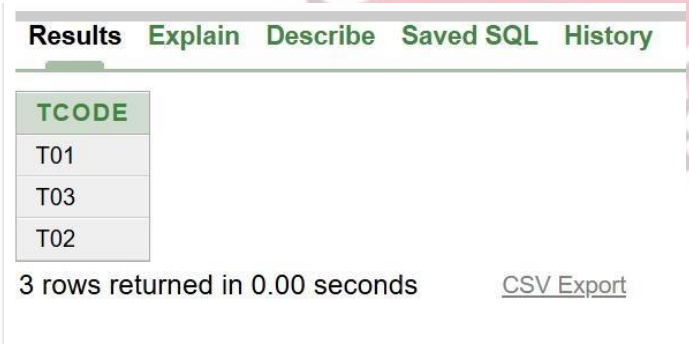
QUESTION:

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

INPUT:

SELECT DISTINCT TCODE FROM ITEMS;

OUTPUT:



Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

INPUT:

SELECT INAME, TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE=T.TCODE AND QTY<100;

OUTPUT:



Results Explain Describe Saved SQL History

INAME	TNAME
LED SCREEN 40	BUSY STORE CORP
CAR GPS SYSTEM	DISP HOUSE INC

2 rows returned in 0.01 seconds CSV Export

TRUTH-TOIL-TRIUMPH
AUTONOMOUS